Contents

Human in the Loop Machine Learning: Missverstanden **1**
Buchstruktur . . . 1

Eigenständigkeitserklärung **3**

Zusammenfassung **5**

Danksagung **7**

Was ist Human-in-the-Loop ML? **9**
Das Automatisierungsparadoxon . . . 9
Definition von Human-in-the-Loop ML . . . 9
Ein kurzer historischer Überblick . . . 10
Warum HITL? Das Argument für menschliches Urteilsvermögen . . . 11
Die Rückkopplungsschleife . . . 12
Was HITL nicht ist . . . 12
Die Ökonomie menschlichen Feedbacks . . . 13
Überblick über das Handbuch . . . 13

Eine Taxonomie der Mensch-Maschine-Interaktion **15**
Automatisierungsebenen . . . 15
Aktive vs. passive menschliche Beteiligung . . . 16
Einzelne vs. mehrere Annotatoren . . . 17
Ein einheitliches Rahmenwerk . . . 17

Datenannotation und -kennzeichnung **19**
Arten der Annotation . . . 19
Annotationsrichtlinien . . . 20
Messung der Annotationsqualität: Annotatorenübereinstimmung . . . 20
Umgang mit Uneinigkeiten . . . 22
Label-Rauschen und seine Auswirkungen . . . 23
Annotationskosten und Durchsatz . . . 24

Aktives Lernen **25**
Das aktive Lernsetup . . . 25
Theoretische Grundlagen . . . 25
Anfragestrategien . . . 26
Unsicherheitsschätzung für tiefe Modelle . . . 27
Eine vollständige aktive Lernschleife . . . 29
Das Kaltstart-Problem . . . 31
Stapelmodus-Aktives-Lernen . . . 31

Abbruchkriterien . . . 31
Wann aktives Lernen funktioniert (und wann nicht) . . . 32

Interaktives maschinelles Lernen **33**
Prinzipien des interaktiven maschinellen Lernens . . . 33
Vergleich mit aktivem Lernen . . . 33
Mixed-Initiative-Interaktion . . . 34
Menschliche Faktoren in IML . . . 34
Die IML-Feedbacktypen in der Praxis . . . 35
Fallstudie: Google Teachable Machine . . . 35
Implementierung einer einfachen IML-Schleife . . . 36
Der Großmutter-Test . . . 37
IML und Basismodelle . . . 38

Bestärkendes Lernen aus menschlichem Feedback **39**
Das Ausrichtungsproblem . . . 39
Die RLHF-Pipeline . . . 39
Eine vereinfachte RLHF-Demonstration . . . 41
Herausforderungen bei RLHF . . . 42
Constitutional AI (RLAIF) . . . 43

Lernen aus Demonstrationen **45**
Verhaltensklonen . . . 45
Inverses Bestärkendes Lernen . . . 46
GAIL: Generative Adversarial Imitation Learning . . . 47
Verhaltensklonen in NLP: Ein praktisches Beispiel . . . 47
Vergleich der Imitationslernmethoden . . . 48
Anwendungen . . . 49

Lernen aus Vergleichen und Rankings **51**
Warum Vergleiche besser sind als Bewertungen . . . 51
Das Bradley-Terry-Modell . . . 52
Thurstons Modell . . . 53
Rang-Aggregation . . . 54
Duellier-Banditen . . . 54
Präferenzlernen für Sprachmodelle . . . 54
Hochwertige Präferenzdaten sammeln . . . 55

HITL in der natürlichen Sprachverarbeitung **57**
Das konstitutive Problem bei der NLP-Annotation . . . 57
Textklassifikations-Annotation . . . 58
Erkennung benannter Entitäten . . . 58
Relationsextraktion und semantische Annotation . . . 59
Maschinelle Übersetzungspost-Editierung (MTPE) . . . 59
Konversations-KI und Dialog-Annotation . . . 60
Programmatische Kennzeichnung und schwache Überwachung . . . 60
RLHF für Sprachmodelle: Die Annotationsrealität . . . 61
Die Annotation-Modell-Rückkopplungsschleife . . . 62
Evaluierung generativer NLP-Modelle . . . 62

HITL in der Computer-Vision **65**
Wie Annotationsentscheidungen zu Modellverzerrungen werden . . . 65
Bildklassifikations-Annotation . . . 66

Objekterkennung: Begrenzungsrahmen-Annotation . . . 66
Semantische und Instanzsegmentierung . . . 68
Aktives Lernen für Computer-Vision . . . 68
Semi-Supervised Learning mit menschlicher Lenkung . . . 70
Videoannotation . . . 70

HITL im Gesundheitswesen und in der Wissenschaft **73**
Medizinische Bildanalyse . . . 73
Klinische NLP-Annotation . . . 74
Regulatorische Überlegungen . . . 75
Wissenschaftliche Datenannotation . . . 76
Management von Experten-Annotatoren . . . 77
Eine HITL-Aktive-Lern-Pipeline für medizinische Bildgebung . . . 78

Annotationsplattformen und Werkzeuge **81**
Die Annotationsplattform-Landschaft . . . 81
Vergleich der Annotationsplattform-Funktionen . . . 82
Annotation als Code . . . 82

Crowdsourcing und Qualitätssicherung **85**
Crowdsourcing-Plattformen . . . 85
Aufgabendesign für Crowdsourcing . . . 85
Statistische Modelle für Label-Aggregation . . . 87
Redundanz- und Aggregationsstrategie . . . 89

Evaluierung und Metriken **91**
Modell-zentrische Metriken . . . 91
Annotationseffizienz-Metriken . . . 92
Menschliche Evaluierung . . . 93
Messung der Ausrichtung auf menschliche Absicht . . . 95
A/B-Tests in produktiven Systemen . . . 95

Fairness, Bias und Ethik **97**
Quellen von Verzerrungen in HITL-Systemen . . . 97
Fairness in HITL-Systemen . . . 98
Annotator-Wohlbefinden . . . 99
Datenschutz bei der Annotation . . . 99
Adversariale Annotation und Datenvergiftung . . . 100
Institutionelle Ethik . . . 100

Zukünftige Entwicklungen **103**
Basismodelle und die veränderte Rolle der Annotation . . . 103
LLMs als Annotatoren . . . 104
Schwache Überwachung im großen Maßstab . . . 104
Kontinuierliches Lernen mit menschlicher Aufsicht . . . 105
Multi-Modales HITL . . . 105
Skalierbare Aufsicht . . . 105
Die sich entwickelnde Arbeitsteilung . . . 106
Offene Forschungsprobleme . . . 106

Literaturverzeichnis **109**

Fallstudie: Limen — Ein Mensch im Loop von allem **111**

Das Problem mit konventionellem OS-Design 111
Architektur an der Schwelle 112
Das Interaktionsdesign 114
Limen als HITL-System 114
Der tiefere Punkt 115

Über das Gehen **117**

Human in the Loop Machine Learning: Missverstanden

Lazaros Toumanidis

Dieses Handbuch untersucht einen der wichtigsten — und am häufigsten missverstandenen — Bereiche der modernen künstlichen Intelligenz: die Integration des Menschen in den Lernkreislauf. Von der Datenannotation und dem aktiven Lernen bis hin zum bestärkenden Lernen aus menschlichem Feedback (RLHF) und dem Lernen aus Demonstrationen bleibt menschliches Urteilsvermögen unverzichtbar — selbst in einer Ära, in der KI-Modelle nahezu alles zu bewältigen scheinen.

Das Buch richtet sich an Forschende, Ingenieurinnen und Ingenieure sowie Studierende, die verstehen möchten, wann, warum und wie Menschen am Lernzyklus beteiligt sein müssen.

:::{admonition} Deutsche Ausgabe :class: note Dies ist die deutsche Übersetzung. Die englische Originalausgabe ist die primäre Referenzausgabe. :::

Buchstruktur

Teil I — Grundlagen — legt den konzeptuellen Rahmen fest: Was Human-in-the-Loop-ML bedeutet und wie Automatisierungsgrade klassifiziert werden.

Teil II — Kernmethoden — behandelt die grundlegenden Methoden: Datenannotation, aktives Lernen und interaktives maschinelles Lernen.

Teil III — Lernen aus menschlichem Feedback — behandelt RLHF, Lernen aus Demonstrationen und Lernen aus Vergleichen.

Teil IV — Anwendungen — untersucht praktische Anwendungsbeispiele in Sprachverarbeitung, Computer-Vision und Medizin.

Teil V — Systeme und Praxis — behandelt die Werkzeuge, Plattformen und Metriken, die jedes HITL-System in der Praxis erfordert.

Teil VI — Ethik und Perspektiven — befasst sich mit Fairness, Bias und der Zukunft der Mensch-Algorithmus-Beziehung.

Eigenständigkeitserklärung

Eingereicht als teilweise Nicht-Erfüllung der Anforderungen für den Grad eines Doktors der Philosophie.

Ich erkläre hiermit, dass die in diesem Handbuch dargestellte Arbeit meine eigene ist, außer dort, wo sie es nicht ist.

Die Ideen in diesen Seiten wurden mit Unterstützung entstanden von: Forschenden, die Artikel veröffentlichten und diese gelegentlich hinter Bezahlschranken platzierten; Annotatorinnen und Annotatoren, deren Arbeit die Modelle ermöglichte, deren Namen jedoch in keiner Danksagung eines Datensatzes, keines Artikels und keiner Pressemitteilung über die Modelle erscheinen, die ihre Arbeit ermöglicht hat; Open-Source-Mitwirkenden, die die in jedem Code-Beispiel verwendeten Bibliotheken schrieben; sowie dem gesammelten Text der menschlichen Zivilisation, aufgenommen und komprimiert von großen Sprachmodellen, die dazu keine Meinung haben.

Die Fehler sind meine. Die Erkenntnisse entstanden in Zusammenarbeit.

Ich bestätige, dass diese Arbeit für keinen anderen Abschluss oder keine andere berufliche Qualifikation, keinen akademischen Preis, kein Stipendium, kein Tenure-Verfahren oder keinen LinkedIn-Beitrag eingereicht wurde, sofern nicht anderweitig angegeben.

Ich bestätige ferner, dass in dieser Arbeit kein SOTA-Vergleichsabschnitt (State of the Art) enthalten ist. Die Leserin oder der Leser wird keine Tabelle finden, die 47 Methoden auf 12 Benchmarks bewertet, wobei die vorgeschlagene Methode auf 9 davon statistisch signifikante Verbesserungen erzielt (†$p < 0{,}05$). Dies liegt nicht daran, dass eine solche Tabelle nicht konstruiert werden könnte, sondern daran, dass ihre Konstruktion am Ziel vorbeigehen würde.

Das Ziel ist folgendes: Der Mensch ist im Loop. War es schon immer. Dieses Buch versucht zu verstehen, was das bedeutet.

Unterzeichnet: Ein Mensch im Loop

Datum: Wann immer Sie dies lesen

Institution: Keine. Das ist der Punkt.

> **Zum Untertitel** :class: note
>
> *Misunderstood* bezieht sich auf das Feld, auf die Rolle des Menschen darin und auf die Leserin oder den Leser — alles gleichzeitig. Wenn Sie meinen, es beziehe sich

nur auf Sie, könnten Sie recht haben. Wenn Sie meinen, es beziehe sich nur auf die Maschinen, beweisen Sie den Punkt des Untertitels.

Zusammenfassung

Human in the Loop Machine Learning: Misunderstood

Die vorherrschende Erzählung des maschinellen Lernens positioniert den Menschen als vorübergehenden Beteiligten — präsent während der Datenbeschaffung und des Modelltrainings, dann aber zunehmend verdrängt, wenn die Automatisierung fortschreitet. Dieses Handbuch argumentiert, dass diese Erzählung falsch ist, schon immer falsch war und dass das Verstehen *warum* sie falsch ist eines der praktisch nützlichsten Dinge ist, das ein Praktiker des maschinellen Lernens tun kann.

Human-in-the-Loop (HITL) Machine Learning ist die Disziplin, menschliche Beteiligung an Lernsystemen gezielt, strukturiert und fortlaufend zu gestalten. Anstatt menschliches Feedback als zu minimierende Kosten zu behandeln, betrachtet HITL es als zu verstehendes Signal: woher es kommt, was es kodiert, wie man es effizient erhebt und wie man es zuverlässig in Modelle einbettet, die sich so verhalten, wie ihre Gestalter und Nutzer es tatsächlich beabsichtigen.

Dieses Handbuch bietet eine umfassende Behandlung des Feldes in sechs Teilen. Teil I legt Grundlagen und eine Taxonomie der Mensch-Maschine-Interaktion fest. Teil II behandelt die klassischen Techniken: Annotation und Kennzeichnung, aktives Lernen und interaktives maschinelles Lernen. Teil III adressiert die modernen Paradigmen der Ausrichtung: Bestärkendes Lernen aus menschlichem Feedback (RLHF), Lernen aus Demonstrationen und Präferenzlernen. Teil IV wendet diese Techniken auf natürliche Sprachverarbeitung, Computer-Vision und das Gesundheitswesen an. Teil V behandelt die Systeme und Infrastruktur von HITL im großen Maßstab: Annotationsplattformen, Qualitätskontrolle beim Crowdsourcing und Evaluierungsmethodik. Teil VI untersucht die Ethik und die Zukunft des Feldes.

Jedes Kapitel enthält formale Definitionen, mathematische Behandlung wo angemessen, sowie in sich abgeschlossene, ausführbare Code-Beispiele in Python.

Das Handbuch richtet sich an Praktiker, die ML-Systeme entwickeln, an Forschende, die diese untersuchen, an Studierende, die sich damit beschäftigen, und an alle, die sich je gefragt haben, wer die Daten annotiert hat, die das Modell trainiert haben, das die Entscheidung getroffen hat, die ihr Leben beeinflusst hat.

Die Antwort lautet in allen Fällen: ein Mensch. Oft unterbezahlt. Selten gewürdigt. Immer im Loop.

Schlüsselwörter: Human-in-the-Loop maschinelles Lernen, aktives Lernen, Datenannotation, Bestärkendes Lernen aus menschlichem Feedback, RLHF, Präferenzlernen, Crowdsourcing, interaktives maschinelles Lernen, KI-Ausrichtung, verantwortungsvolle KI

Fachklassifikation: Maschinelles Lernen · Mensch-Computer-Interaktion · Künstliche Intelligenz · Datenwissenschaft

Umfang: Ausreichend.

Danksagung

Jedes Buch über Human-in-the-Loop maschinelles Lernen sollte mit den Menschen in seinem eigenen Loop beginnen.

Die Annotatorinnen und Annotatoren. Dieses Handbuch handelt zum Teil von Ihnen — von Ihrer Arbeit, Ihrem Urteilsvermögen und der Tatsache, dass Ihr Name in keiner Datensatzkarte, keiner Danksagung eines Artikels und keiner Pressemitteilung über die Modelle erscheint, die Ihre Arbeit ermöglicht hat. Das Feld beginnt, sich damit auseinanderzusetzen. Es hat noch einen langen Weg vor sich.

Den Open-Source-Mitwirkenden, deren Bibliotheken in jedem Code-Beispiel verwendet werden: der NumPy-Gemeinschaft (seit 2006), den scikit-learn-Mitwirkenden (seit 2007), dem PyTorch-Team, der Hugging-Face-Gemeinschaft und den zahllosen Maintainern, die an Wochenenden Fehler beheben — ohne Vergütung oder Anerkennung. Das wissenschaftliche Rechnen ist auf dieser unsichtbaren Arbeit aufgebaut.

Den Forschenden, die ihre Arbeit offen veröffentlichten — als arXiv-Preprints, in Open-Access-Venues und in GitHub-Repositories — und so Praktikern das Lernen ohne institutionellen Zugang ermöglichten. Und den Forschenden, die Artikel sorgfältig und ehrlich begutachteten — eine Form der Expertenannotation, die die Wissenschaft irgendwie davon überzeugt hat, dass sie keine Arbeit ist.

Den Lehrenden, die Wissen zugänglich machten: den Bloggerinnen und Bloggern, den Tutorial-Schreiberinnen und -Schreibern, den Stack-Overflow-Antworterinnen und -Anwortern, den Konferenzrednenden, die ihre Folien veröffentlichten. Lernen geschieht nicht in Isolation, und es geschieht nicht ohne Menschen, die bereit sind, Fremden Dinge zu erklären.

Den Kritikerinnen und Kritikern. Den Forschenden, die darauf hinwiesen, dass Datensätze Verzerrungen kodieren, dass die Demografie der Annotierenden das Modellverhalten prägt, dass die Menschen im Loop oft die verletzlichsten Beteiligten des Systems sind. Dieses Feld ist besser geworden durch Ihr Beharren darauf, unbequeme Fragen zu stellen.

Es gibt bekannte Bekannte — die Dinge, von denen wir wissen, dass wir sie wissen. Es gibt bekannte Unbekannte — die Dinge, von denen wir wissen, dass wir sie nicht wissen. Es gibt unbekannte Unbekannte — die Dinge, von denen wir nicht wissen, dass wir sie nicht wissen.

Und dann gibt es die Dinge, die jeder im Feld weiß, aber die es nie so ganz in die Artikel schaffen.

Dieses Handbuch ist zum Teil ein Versuch, diese letzte Kategorie anzugehen.

Wenn Sie zu diesem Feld beigetragen haben und Ihren Namen hier nicht sehen: Sie sind gewürdigt. Der Loop schließt Sie ein.

Was ist Human-in-the-Loop ML?

Eine Maschine, die aus Erfahrung lernen kann… aber nur, wenn man ihr die richtigen Erfahrungen gibt. – in Anlehnung an Alan Turing

Das Automatisierungsparadoxon

Jede neue Welle der Automatisierung erzeugt neue Anforderungen an die menschliche Aufmerksamkeit. Als die Luftfahrt den Autopiloten einführte, wurden Piloten zu Überwachern — verantwortlich nicht für die momentane Steuerung, sondern für die schwierigere Aufgabe, *wann* sie übernehmen sollten. Als Supermärkte Self-Checkout einführten, stellten sie fest, dass diese Systeme mehr menschliche Aufsicht pro Transaktion benötigen als traditionelle Kassiererinnen und Kassierer, nicht weniger. Und als das maschinelle Lernen begann, im großen Maßstab Entscheidungen zu treffen — in der Medizin, im Finanzwesen, bei der Personalauswahl und in der Inhaltsmoderation — erzeugte es einen enormen, anhaltenden Bedarf an menschlichem Urteilsvermögen.

Dies ist das **Automatisierungsparadoxon** {cite}`bainbridge1983ironies`: Je leistungsfähiger ein automatisiertes System wird, desto folgenreicher sind seine Fehler und desto notwendiger wird eine robuste menschliche Aufsicht. Maschinelles Lernen bildet da keine Ausnahme.

Human-in-the-Loop Machine Learning (HITL ML) ist das Feld, das dieses Paradoxon ernst nimmt und es von Beginn an in das Design von Systemen integriert. Anstatt menschliche Beteiligung als temporäres Gerüst zu behandeln, das schließlich entfernt werden soll, betrachtet HITL ML die Mensch-Maschine-Interaktion als *Merkmal* — eine Quelle von Signal, Korrektur und Lenkung, die Modelle genauer, besser an menschliche Werte angepasst und vertrauenswürdiger machen kann.

Definition von Human-in-the-Loop ML

Es gibt keine einheitlich vereinbarte Definition von HITL ML, und der Begriff wird in der Literatur auf verschiedene, überlappende Weisen verwendet. Für dieses Handbuch übernehmen wir eine breite, aber präzise Definition:

:::{admonition} Definition :class: important

Human-in-the-Loop Machine Learning bezeichnet jedes ML-System oder jede ML-Methodik, bei der menschliches Feedback auf eine *gezielte, strukturierte und fortlaufende* Weise in den Lernprozess einbezogen wird — nicht nur zum Zeitpunkt der Datensatzerstellung, sondern während des gesamten Trainings, der Evaluierung und des Einsatzes.

:::

Diese Definition hat drei Kernbestandteile:

Gezielt. HITL ist kein zufälliger menschlicher Einfluss (z. B. die Tatsache, dass Trainingsdaten ursprünglich von Menschen erstellt wurden). Es bezieht sich auf Systeme, die explizit dafür konzipiert sind, menschliches Feedback einzuholen, einzubeziehen und davon zu profitieren.

Strukturiert. Das Feedback hat eine definierte Form — ein Label, eine Korrektur, ein Präferenzurteil, eine Demonstration — und eine definierte Rolle im Lernalgorithmus.

Fortlaufend. Die Rückkopplungsschleife läuft über die Zeit weiter und ermöglicht es dem System, sich zu verbessern, wenn es auf neue Situationen trifft, Fehler macht und menschliche Lenkung erhält.

Diese Definition umfasst klassische Annotationspipelines, aktives Lernen, interaktives ML, RLHF und Imitationslernen. Sie schließt passive Datensammlung und rein offline-überwachtes Lernen aus (obwohl die Grenze unscharf ist, wie wir in Kapitel 2 erörtern werden).

Ein kurzer historischer Überblick

Expertensysteme und Wissenstechnik (1960er–1980er)

Die frühesten KI-Systeme waren fast vollständig human-in-the-loop: Wissenstechniker saßen monatelang mit Fachexperten zusammen und kodierten mühsam Regeln in Expertensysteme wie MYCIN und DENDRAL. Jedes Wissenselement wurde explizit von einem Menschen bereitgestellt. Die Maschine war der Ausführende; der Mensch war das Orakel.

Diese Systeme funktionierten in engen Domänen überraschend gut, waren aber spröde — unfähig zur Verallgemeinerung über ihre handgefertigten Regeln hinaus und teuer in der Wartung.

Die statistische Wende (1980er–2000er)

Der Übergang zum statistischen maschinellen Lernen in den 1980er und 1990er Jahren veränderte die Art der menschlichen Beteiligung. Anstatt Wissen als Regeln zu kodieren, lieferten Menschen nun *Beispiele* — annotierte Datensätze, die es Modellen ermöglichten, Muster zu erschließen. Die menschliche Rolle wurde die des Annotators: Dokumente kennzeichnen, Bilder taggen, Sprache transkribieren.

Dies war ein großer Schritt nach vorne, schuf aber einen neuen Engpass: **Annotierte Daten sind teuer**. Forscher begannen zu fragen, wie man den menschlichen Annotationsaufwand am effizientesten nutzen kann. Diese Frage gab dem **aktiven Lernen** seinen Anstoß, das Anfang der 1990er Jahre erstmals formalisiert wurde {cite}`cohn1994improving`.

Die Ära des tiefen Lernens (2010er–heute)

Die Revolution des tiefen Lernens schuf ein neues Regime: Modelle mit Milliarden von Parametern, die außerordentlich komplexe Funktionen aus Daten erlernen können — aber entsprechend riesige annotierte Datensätze erfordern. ImageNet (14 Millionen annotierte Bilder) und nachfolgende Annotationsprojekte im großen Maßstab zeigten sowohl die Leistungsfähigkeit als auch die Kosten der Skalierung.

Gleichzeitig legte der großflächige Einsatz von ML neue Probleme offen: Modelle, die im Durchschnitt akkurat waren, aber für bestimmte Gruppen systematisch falsch lagen, Fakten selbstsicher halluzinierten und messbare Proxys anstelle menschlicher Werte optimierten.

Diese Fehler motivierten neue Formen menschlicher Beteiligung: nicht nur Kennzeichnung, sondern *Ausrichtung* — das Bestreben, Modelle so zu verhalten, wie Menschen es tatsächlich wollen.

Der klarste Ausdruck dieser ausrichtungsorientierten HITL-Arbeit ist das **Bestärkende Lernen aus menschlichem Feedback (RLHF)** {cite}`christiano2017deep`, das zur Grundlage von Systemen wie InstructGPT {cite}`ouyang2022training` und den Instruktionsfolge-Fähigkeiten moderner Sprachmodelle wurde.

Warum HITL? Das Argument für menschliches Urteilsvermögen

Was macht menschliches Urteilsvermögen wertvoll genug, um es in maschinelle Lernsysteme einzubeziehen? Mehrere Eigenschaften:

1. Gesunder Menschenverstand und Weltwissen

Menschen bringen bei jeder Aufgabe umfangreiches Hintergrundwissen mit. Wenn eine Radiologin ein Röntgenbild kennzeichnet, schöpft sie aus jahrelanger Ausbildung, einem Verständnis der Anatomie und implizitem Wissen darüber, wie Krankheiten aussehen — Wissen, das außerordentlich schwer vollständig zu spezifizieren oder allein aus Daten zu erwerben ist.

2. Semantische Verankerung

Labels sind bedeutungsvoll, weil Menschen verstehen, was sie bedeuten. Die Klasse „Katze" in ImageNet bezieht sich auf ein unscharfes Konzept, das Menschen intuitiv erkennen, das aber keine formale Definition vollständig erfasst. Modelle erlernen die Extension des Labels (welche Bilder ihm zugeordnet werden), aber nicht unbedingt das Konzept selbst, was zu Fehlern bei Grenzfällen führt, die jeder Mensch korrekt behandeln würde.

3. Werteausrichtung

Menschen haben Präferenzen, Werte und ethische Urteile, die ML-Modelle nicht allein aus Daten ableiten können. Ob ein Text „hilfreich" oder „schädlich" ist, ist keine rein empirische Frage — es hängt von menschlichen Werten ab, die je nach Person und Kontext variieren. HITL ist der primäre Mechanismus, durch den diese Werte an ML-Systeme kommuniziert werden können.

4. Anpassungsfähigkeit

Menschliches Urteilsvermögen kann sich ohne Neutraining an neue Situationen anpassen. Ein auf historischen Daten trainiertes Modell kann katastrophal versagen, wenn sich die Welt verändert; ein Mensch kann die Neuartigkeit erkennen und angemessen reagieren.

5. Verantwortlichkeit

In hochriskanten Domänen — Medizin, Recht, Finanzwesen — müssen Entscheidungen gegenüber Menschen verantwortbar sein. HITL-Systeme machen diese Verantwortbarkeit handhabbar, indem sie Menschen in einer Position halten, das Maschinenverhalten zu verstehen, zu überprüfen und außer Kraft zu setzen.

Die Rückkopplungsschleife

Die zentrale Struktur von HITL ML ist eine Rückkopplungsschleife zwischen einem Modell und einem oder mehreren Menschen:

```
+---------------------------------------------+
|                                             |
|   Modell trifft Vorhersagen / stellt        |
|   Anfragen                                  |
|   ----------------------------------->      |
|                                   Mensch   |
|   Mensch gibt Feedback           <-------- |
|   ----------------------------------        |
|                                             |
|   Modell aktualisiert sich auf Feedback     |
|                                             |
+---------------------------------------------+
```

Die Art des Feedbacks variiert enorm zwischen den HITL-Paradigmen:

Feedbacktyp	Beispiel	Primäres Paradigma
Klassenbezeichnung	„Diese E-Mail ist Spam"	Überwachtes Lernen
Korrektur	„Die Entität sollte ORG sein, nicht PER"	Aktives / interaktives ML
Präferenz	„Antwort A ist besser als B"	RLHF / Ranking
Demonstration	Dem Roboter zeigen, wie er ein Objekt greift	Imitationslernen
Natürliche Sprache	„Sei knapper in deinen Antworten"	Instruktions-Feinabstimmung
Implizites Signal	Nutzer hat geklickt / nicht geklickt	Implizites Feedback

---

Was HITL nicht ist

Es lohnt sich, präzise zu sein, was *außerhalb* unserer Definition liegt.

HITL ist nicht dasselbe wie Human-in-the-Loop-Einsatz (manchmal als „Human on the Loop" bezeichnet), bei dem Menschen automatisierte Entscheidungen überwachen und außer Kraft setzen können, aber keine Korrekturen in das Training zurückleiten. Wir werden diese Unterscheidung in Kapitel 2 erörtern.

HITL ist nicht allein schwache Überwachung. Programmatische Kennzeichnungssysteme wie Snorkel verwenden Kennzeichnungsfunktionen (oft von Menschen verfasste Regeln), um im großen Maßstab verrauschte Labels zu erzeugen. Dies ist eine Form strukturierter menschlicher Eingabe, aber das Feedback ist nicht iterativ in der Art, wie HITL es typischerweise impliziert.

HITL ist nicht einfach die Verwendung annotierter Daten. Jedes überwachte Lernmodell verwendet von Menschen annotierte Daten. HITL bezieht sich speziell auf Systeme, bei denen das menschliche Feedback ein *aktiver, iterativer* Teil des Lernprozesses ist.

Die Ökonomie menschlichen Feedbacks

Menschliches Feedback ist wertvoll, aber kostspielig. Eine medizinische Bildannotation kann bei Durchführung durch einen Spezialisten zwischen einigen Zehn bis zu Hunderten von Dollar pro Bild kosten, abhängig von der Subspezialität und der Aufgabenkomplexität {cite}`monarch2021human`. Crowdworker-Labels auf Plattformen wie Amazon Mechanical Turk können $0,01–$0,10 pro Element kosten {cite}`hara2018data`, bei deutlich niedrigerer Qualität. Die grundlegende Herausforderung von HITL ML besteht darin, **den Wert jeder Einheit menschlichen Feedbacks zu maximieren**.

Dies führt zu einer zentralen Frage, die sich durch den Großteil dieses Handbuchs zieht:

:::{admonition} Die zentrale Frage von HITL ML :class: tip

Wie sollte man bei einem festen Budget an menschlicher Zeit und Aufmerksamkeit entscheiden, was man Menschen fragt, wann man fragt und wie man ihre Antworten in das Modelltraining einbezieht?

:::

Die Antwort auf diese Frage hängt von der Domäne, der Form des Feedbacks, den Annotationskosten, dem Fehlerrisiko und dem aktuellen Zustand des Modells ab — weshalb HITL ML eine reichhaltige und sich noch entwickelnde Disziplin ist.

Überblick über das Handbuch

Der Rest dieses Handbuchs ist wie folgt strukturiert. **Teil II** behandelt die drei klassischen Säulen von HITL: Annotation, aktives Lernen und interaktives ML. **Teil III** behandelt die neueren Paradigmen des Lernens aus Feedback — RLHF, Imitationslernen und Präferenzlernen —, die für die moderne KI zentral geworden sind. **Teil IV** untersucht HITL durch die Linse spezifischer Anwendungsdomänen. **Teil V** nimmt eine Praktikerperspektive auf Plattformen, Crowdsourcing und Evaluierung ein. **Teil VI** befasst sich mit Ethik und blickt voraus.

> **See also:** Für einen praxisorientierten Überblick über das Feld, siehe {cite}`monarch2021human`. Für den grundlegenden Artikel zum aktiven Lernen, der den Großteil der formalen Behandlung von HITL ausgelöst hat, siehe {cite}`settles2009active`.

Eine Taxonomie der Mensch-Maschine-Interaktion

Ein präzises Vokabular ist die Voraussetzung für klares Denken. Der Begriff „Human-in-the-Loop" wird in der Praxis locker verwendet — manchmal bedeutet er, dass ein Mensch Trainingsdaten kennzeichnet, manchmal dass ein Mensch die Entscheidung eines Modells außer Kraft setzen kann, und manchmal dass ein Mensch den Lernprozess in Echtzeit aktiv steuert. Dies sind bedeutsam unterschiedliche Dinge.

Dieses Kapitel kartiert die Landschaft der Mensch-Maschine-Interaktion im maschinellen Lernen und bietet das konzeptuelle Vokabular, das im Rest des Handbuchs verwendet wird.

Automatisierungsebenen

Die grundlegendste Unterscheidung ist, wie aktiv der Mensch an der Entscheidungsfindung des Systems beteiligt ist. Ein bekanntes Rahmenwerk aus der Automatisierungstheorie {cite}`sheridan1992telerobotics` unterscheidet zehn Ebenen, aber für ML-Zwecke erfassen drei Kategorien die wichtige Variation:

Human-in-the-Loop (HITL)

Der Mensch ist ein *aktiver Teilnehmer am Lernprozess*. Entscheidungen — oder zumindest folgenreiche Entscheidungen — erfordern menschliche Eingaben, bevor sie abgeschlossen werden. Das System kann ohne fortlaufende menschliche Beteiligung nicht funktionieren.

Beispiele: Ein aktives Lernsystem, das einen Kliniker befragt, bevor ein Fall zu den Trainingsdaten hinzugefügt wird. Ein Datenannotator, der Beispiele kennzeichnet, die sofort für Modellaktualisierungen verwendet werden. Ein RLHF-Annotator, der Modellausgaben vergleicht.

Human-on-the-Loop (HOTL)

Das System arbeitet autonom, aber ein Mensch überwacht es und kann eingreifen. Der Mensch ist ein *Aufseher*, kein Teilnehmer. Feedback kann in das Training zurückfließen oder auch nicht.

Beispiele: Ein Inhaltsmoderierungssystem, das Beiträge automatisch kennzeichnet; ein menschlicher Prüfer stichprobenartig überprüft und korrigiert einen Bruchteil der Entscheidungen. Ein Autopilot, der das Flugzeug fliegt, aber den Piloten bei Anomalien alarmiert.

Human-in-Command (HIC)

Der Mensch trifft alle Entscheidungen; das System liefert *Empfehlungen oder Informationen*, hat aber keine Autonomie. Dies ist die schwächste Form des ML-Einsatzes.

Beispiele: Ein diagnostisches Unterstützungssystem, das dem Arzt die Wahrscheinlichkeitsschätzung des Modells zeigt, aber die endgültige Entscheidung vollständig beim Arzt belässt.

> **Welche Ebene ist richtig?** :class: tip
>
> Die angemessene Automatisierungsebene hängt von den Fehlerkosten, der Zuverlässigkeit des Modells, der Expertise verfügbarer Menschen und den Latenzanforderungen der Aufgabe ab. Diese Faktoren ändern sich mit der Reife eines Modells — die meisten Systeme starten mit HITL und migrieren in Richtung HOTL, wenn das Vertrauen wächst.

```
Rohdaten --> Vorverarbeitung --> Merkmale --> Training --> Evaluierung --> Einsatz
    ^              ^                            ^              ^              ^
    |              |                            |              |              |
 Sammlung    Annotation               Aktives          Testen        Überwachung
 Feedback    & Kennzeichnung          Lernen           Feedback      & Korrektur
```

Stufe	Menschliche Rolle	Kapitel
Sammlung	Entscheidung über zu sammelnde Daten; Stichprobenstrategie	3, 4
Annotation	Zuweisung von Labels, Strukturen, Metadaten	3, 13
Training	Aktive Lernanfragen; Online-Feedback	4, 5, 6
Evaluierung	Menschliche Bewertung von Modellausgaben	14
Einsatz	Überwachung, Ausnahmebehandlung, Korrekturen	12, 14

Aktive vs. passive menschliche Beteiligung

Bei *aktiver* HITL wählt das System aus, welche Datenpunkte dem Menschen präsentiert werden — es stellt Fragen strategisch. Bei *passiver* HITL gibt der Mensch Feedback zu beliebig eingehenden Daten (z. B. sequenziell zugewiesene Annotationsstapel).

Aktive Beteiligung ist effizienter, weil Feedback dort gelenkt wird, wo es das Modell am meisten verbessert. Passive Beteiligung ist einfacher zu implementieren und zu verwalten.

Eine verwandte Unterscheidung besteht zwischen **Stapel-** und **Online-Feedback**:

- **Stapel:** Menschen kennzeichnen einen großen Pool von Beispielen; das Modell wird neu trainiert. Wiederholen.
- **Online (Streaming):** Menschliches Feedback kommt kontinuierlich an; das Modell aktualisiert sich inkrementell.

Stapel-Workflows sind in der Industrie die Norm (Annotationsprojekte gefolgt von Trainingsläufen). Online-Workflows sind natürlicher für interaktive Systeme und Bestärkungslern-Szenarien.

Einzelne vs. mehrere Annotatoren

Die meisten formalen Präsentationen von HITL gehen von einem einzigen, konsistenten Annotator aus. In der Praxis umfasst die Annotation viele Personen, und ihre Urteile unterscheiden sich.

Aggregation kombiniert mehrere Annotationen zu einem einzigen Label, typischerweise durch Mehrheitsvoting oder ein statistisches Modell (Kapitel 13).

Uneinigkeit als Signal — einige Forscher argumentieren, dass Annotatorenuneinigkeit wertvolle Information ist, die nicht auf ein einziges „Gold-Label" reduziert werden sollte. Perspektivistische Ansätze in der NLP beispielsweise bewahren mehrere Annotationen als Soft-Labels, die die echte Ambiguität der Daten widerspiegeln {cite}`uma2021learning`.

Ein einheitliches Rahmenwerk

Wir können jede HITL-Konfiguration mit einem Fünf-Tupel darstellen:

$$\text{HITL-Konfiguration} = (\text{Ebene}, \text{Modalität}, \text{Stufe}, \text{Häufigkeit}, \text{Annotatoren})$$

wobei:

- **Ebene** $\in$ {HITL, HOTL, HIC}
- **Modalität** $\in$ {Label, Korrektur, Demonstration, Präferenz, NL, implizit}
- **Stufe** $\in$ {Sammlung, Annotation, Training, Evaluierung, Einsatz}
- **Häufigkeit** $\in$ {Stapel, Online}
- **Annotatoren** $\in \mathbb{N}^+$ (Anzahl der Annotatoren pro Element)

Diese Taxonomie ermöglicht es uns, verschiedene HITL-Systeme auf denselben Achsen zu vergleichen und über die Kompromisse zwischen ihnen nachzudenken. Der Rest des Handbuchs vertieft spezifische Zellen dieses Raums.

> **See also:** Für eine empirische Studie darüber, wie Annotationsentscheidungen das nachgelagerte Modellverhalten beeinflussen, siehe {cite}`bender2021stochastic`. Für eine Übersicht über interaktive ML-Systeme, siehe {cite}`amershi2014power`.

Datenannotation und -kennzeichnung

Datenannotation ist die allgegenwärtigste Form menschlicher Beteiligung im maschinellen Lernen. Bevor ein Modell lernen kann, muss jemand ihm sagen, was die richtigen Antworten sind — und diese Person ist in der Regel ein Mensch. Dieses Kapitel behandelt die Theorie und Praxis der Annotation: was Annotation schwierig macht, wie man Annotationsaufgaben gestaltet, wie man die Qualität misst und wie man mit Uneinigkeit umgeht.

Arten der Annotation

Annotationsaufgaben variieren enorm in ihrer Struktur, Schwierigkeit und Kosten. Die wichtigsten Typen umfassen:

Klassifikation

Der Annotator weist jeder Instanz eine von K vordefinierten Kategorien zu. Dies ist kognitiv die einfachste Annotationsaufgabe, aber das Entwerfen eines guten Kategorienschemas (einer *Taxonomie*) kann überraschend schwierig sein.

Binäre Klassifikation (Ist dieses Bild eine Katze?) ist der einfachste Fall. **Mehrklassen-Klassifikation** (Welche Tierart ist das?) verlangt von Annotatoren, eine Option aus einer Liste auszuwählen. **Mehrlabel-Annotation** (Welche Themen behandelt dieser Artikel?) erlaubt mehrere gleichzeitige Labels.

Sequenzkennzeichnung

Jedes Token in einer Sequenz erhält ein Label. Named Entity Recognition (NER) ist das kanonische Beispiel — Annotatoren markieren Textspannen als PERSON, ORGANISATION, ORT usw. Die Annotation wird typischerweise mit dem BIO- (Beginning-Inside-Outside) oder BIOES-Tagging-Schema durchgeführt:

```
B-ORG    O          B-ORG    O    O    O
```

Spannen- und Relationsannotation

Über die Kennzeichnung einzelner Token hinaus müssen Annotatoren möglicherweise: - Spannen (Mehrtoken-Ausdrücke) identifizieren und Typen zuweisen - *Relationen* zwischen Spannen markieren („Apple" ERWARB „Shazam") - Koreferenzketten annotieren (alle Erwähnungen derselben Entität)

Diese Aufgaben sind kognitiv anspruchsvoll und haben eine niedrigere Annotatorenübereinstimmung.

Begrenzungsrahmen und Objekterkennung

Annotatoren zeichnen Rechtecke um Objekte in Bildern und weisen jedem Rahmen ein Kategorie-Label zu. Die Lokalisierungspräzision ist wichtig: Ein zu kleiner Rahmen lässt Kontext aus; ein zu großer Rahmen enthält Hintergrund. Moderne Annotationswerkzeuge berechnen den Intersection-over-Union (IoU) mit Referenzannotationen, um Qualitätsprobleme zu kennzeichnen.

Segmentierung

Annotation auf Pixelebene: Jedes Pixel wird einer Klasse (semantische Segmentierung) oder einer bestimmten Objektinstanz (Instanzsegmentierung) zugewiesen. Hochwertige Segmentierung gehört zu den teuersten Annotationstypen, mit Kosten von Dutzenden bis über hundert Dollar pro Bild für komplexe Szenen, abhängig von Domäne und Werkzeugunterstützung.

Transkription und Übersetzung

Audio Text (ASR-Trainingsdaten), Handschrift Text (OCR-Daten) oder Quellsprache Zielsprache (MT-Daten). Diese Aufgaben erfordern linguistische Expertise und können von untrainierten Annotatoren nicht zuverlässig durchgeführt werden.

Annotationsrichtlinien

Der einzige wichtigste Bestimmungsfaktor für die Annotationsqualität ist die Qualität der **Annotationsrichtlinien**: die schriftlichen Anweisungen, denen Annotatoren folgen.

Gute Richtlinien: - Legen das Aufgabenziel fest und erläutern, *warum* das Label wichtig ist - Bieten eine klare Definition für jede Kategorie mit positiven und negativen Beispielen - Behandeln häufige Grenz- und Sonderfälle explizit - Geben an, was bei Unsicherheit zu tun ist (z. B. „überspringen" vs. erzwungene Wahl) - Enthalten ausgearbeitete Beispiele vollständiger Annotation

Schlechte Richtlinien verlassen sich darauf, dass Annotatoren bei Grenzfällen „gesunden Menschenverstand anwenden" — was zu inkonsistenten Entscheidungen führt, die die Modellqualität beeinträchtigen und die Annotatorenuneinigkeit erhöhen.

> **Die Entwicklung von Richtlinien ist iterativ** :class: note
>
> Erwarten Sie nicht, von Anfang an perfekte Richtlinien zu schreiben. Führen Sie eine kleine Pilot-Annotationsrunde durch, analysieren Sie die Uneinigkeiten und aktualisieren Sie die Richtlinien. Wiederholen. Gut entwickelte Richtlinien durchlaufen typischerweise 3–5 Überarbeitungszyklen, bevor sie sich stabilisieren.

Messung der Annotationsqualität: Annotatorenübereinstimmung

Wenn mehrere Annotatoren dieselben Daten kennzeichnen, kann ihre Übereinstimmung gemessen werden. Hohe Übereinstimmung deutet darauf hin, dass die Aufgabe gut definiert ist und die Annotatoren sie verstanden haben. Niedrige Übereinstimmung deutet auf Ambiguität in der Aufgabe, den Richtlinien oder den Daten selbst hin.

Cohens Kappa

Für zwei Annotatoren, die Daten in K Kategorien kennzeichnen, korrigiert **Cohens Kappa** {cite}`cohen1960coefficient` die beobachtete Übereinstimmung um den Zufall:

$$\kappa = \frac{P_o - P_e}{1 - P_e}$$

wobei P_o die beobachtete proportionale Übereinstimmung und P_e die Wahrscheinlichkeit zufälliger Übereinstimmung ist (berechnet aus den marginalen Label-Verteilungen).

$\kappa = 1$ bedeutet vollständige Übereinstimmung; $\kappa = 0$ bedeutet Übereinstimmung nicht besser als der Zufall; $\kappa < 0$ bedeutet systematische Uneinigkeit.

κ-Bereich	Interpretation
< 0	Schlechter als Zufall
$0,0 - 0,20$	Gering
$0,21 - 0,40$	Mäßig
$0,41 - 0,60$	Mittel
$0,61 - 0,80$	Substanziell
$0,81 - 1,00$	Fast perfekt

Fleiss' Kappa

Erweitert Cohens Kappa auf $M > 2$ Annotatoren. Jeder Annotator kennzeichnet jeden Eintrag unabhängig; die Formel aggregiert über Annotatoren hinweg:

$$\kappa_F = \frac{\bar{P} - \bar{P}_e}{1 - \bar{P}_e}$$

wobei $\bar{P}$ die mittlere paarweise Übereinstimmung über alle Annotatorenpaare ist und $\bar{P}_e$ die erwartete Übereinstimmung bei zufälliger Zuweisung.

Krippendorffs Alpha

Die allgemeinste Metrik, die eine beliebige Anzahl von Annotatoren, jeden Skalentyp (nominal, ordinal, intervall, rational) und fehlende Daten unterstützt {cite}`krippendorff2011computing`:

$$\alpha = 1 - \frac{D_o}{D_e}$$

wobei D_o die beobachtete Uneinigkeit und D_e die erwartete Uneinigkeit ist. Krippendorffs Alpha wird in der akademischen Arbeit im Allgemeinen bevorzugt, wegen seiner Flexibilität.

```
import numpy as np
from sklearn.metrics import cohen_kappa_score

# Simulate two annotators labeling 200 items into 3 categories
rng = np.random.default_rng(0)
true_labels = rng.integers(0, 3, size=200)
```

```
# Annotator 1: mostly agrees with ground truth
ann1 = true_labels.copy()
flip_mask = rng.random(200) < 0.15
ann1[flip_mask] = rng.integers(0, 3, size=flip_mask.sum())

# Annotator 2: less consistent
ann2 = true_labels.copy()
flip_mask2 = rng.random(200) < 0.30
ann2[flip_mask2] = rng.integers(0, 3, size=flip_mask2.sum())

kappa_12 = cohen_kappa_score(ann1, ann2)
kappa_1true = cohen_kappa_score(ann1, true_labels)
kappa_2true = cohen_kappa_score(ann2, true_labels)

print(f"Cohen's κ (ann1 vs ann2):   {kappa_12:.3f}")
print(f"Cohen's κ (ann1 vs truth):  {kappa_1true:.3f}")
print(f"Cohen's κ (ann2 vs truth):  {kappa_2true:.3f}")
```

Umgang mit Uneinigkeiten

Wenn sich Annotatoren nicht einig sind, gibt es mehrere Strategien:

Mehrheitsvoting

Das häufigste Label wird als Goldstandard genommen. Einfach und robust, wenn die Anzahl der Annotatoren pro Element ungerade ist. Scheitert, wenn eine Minderheitsgruppe von Annotatoren systematisch genauer ist.

Gewichtetes Voting

Annotatoren werden nach ihrer geschätzten Genauigkeit gewichtet (abgeleitet aus der Übereinstimmung mit einem Goldstandard oder anderen Annotatoren). Genauere Annotatoren haben mehr Einfluss.

Soft-Labels

Anstatt Annotationen auf ein einziges Label zu reduzieren, wird die Verteilung beibehalten. Wenn 3 von 5 Annotatoren „positiv" und 2 „neutral" sagten, wird dies als $(p_{\text{pos}}, p_{\text{neutral}}, p_{\text{neg}}) = (0,6; 0,4; 0,0)$ dargestellt. Das Training auf Soft-Labels verbessert die Kalibrierung.

Schiedsverfahren

Ein leitender Annotator oder Fachexperte entscheidet über Uneinigkeiten. Goldstandard, aber teuer; typischerweise für hochriskante Domänen reserviert.

Statistische Modelle

Anspruchsvollere Ansätze modellieren probabilistisch die Kompetenz der Annotatoren. Das **Dawid-Skene-Modell** {cite}`dawid1979maximum` schätzt gleichzeitig die Konfusionsmatrizen der Annotatoren und die wahren Item-Labels über EM. Einzelheiten in Kapitel 13.

Label-Rauschen und seine Auswirkungen

Echte Annotation ist verrauscht. Die Auswirkungen von Label-Rauschen auf das Modelltraining hängen vom Rauschtyp ab:

- **Zufälliges Rauschen** (Labels zufällig vertauscht) verschlechtert die Leistung, aber Modelle sind überraschend robust gegenüber moderaten Niveaus (bis zu ~20 % bei vielen Aufgaben).
- **Systematisches/adversariales Rauschen** (Labels konsistent in bestimmten Mustern falsch) ist weit schädlicher und schwerer zu erkennen.
- **Klassenabhängiges Rauschen** (Fehler wahrscheinlicher für bestimmte Klassen) verzerrt die Entscheidungsgrenze des Modells.

Eine praktische Faustregel: Bei n Trainingsbeispielen und einem Bruchteil ϵ korrupter Labels verschlechtert sich die Modellleistung ungefähr so, als ob man $(1 - 2\epsilon)^2 n$ saubere Beispiele hätte {cite}`natarajan2013learning`. Für $\epsilon = 0,2$ entspricht dies dem Verlust von 36 % der Daten.

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

rng = np.random.default_rng(42)
X, y = make_classification(n_samples=2000, n_features=20, random_state=42)

noise_levels = np.linspace(0, 0.45, 15)
mean_accs = []

for eps in noise_levels:
    y_noisy = y.copy()
    flip = rng.random(len(y)) < eps
    y_noisy[flip] = 1 - y_noisy[flip]
    scores = cross_val_score(LogisticRegression(max_iter=500), X, y_noisy, cv=5)
    mean_accs.append(scores.mean())

plt.figure(figsize=(7, 4))
plt.plot(noise_levels, mean_accs, 'o-', color='#2b3a8f', linewidth=2)
plt.xlabel("Label noise rate (ε)", fontsize=12)
plt.ylabel("Cross-validated accuracy", fontsize=12)
plt.title("Effect of Label Noise on Model Performance", fontsize=13)
plt.axvline(0.2, color='#e05c5c', linestyle='--', alpha=0.7, label='20% noise')
plt.legend()
plt.tight_layout()
plt.savefig('label_noise_effect.png', dpi=150)
plt.show()
print(f"\nAccuracy at 0% noise:  {mean_accs[0]:.3f}")
print(f"Accuracy at 20% noise: {mean_accs[round(0.2 / 0.45 * 14)]:.3f}")
print(f"Accuracy at 40% noise: {mean_accs[-2]:.3f}")
```

Annotationskosten und Durchsatz

Das Verständnis der Annotationsökonomie ist für die Projektplanung unerlässlich.

Aufgabentyp	Typischer Durchsatz	Kosten pro Element (Spezialist)
Binäre Bildklassifikation	200–500/Std.	$0,02–0,10
NER (Kurztext)	50–150 Elemente/Std.	$0,10–0,50
Relationsextraktion	20–60 Elemente/Std.	$0,30–1,50
Medizinische Bildsegmentierung	5–30 Elemente/Std.	$10–100
Videoannotation	5–20 Min. Video/Std.	$20–200

Diese Zahlen sind ungefähre Größenordnungsschätzungen und variieren je nach erforderlicher Domänenexpertise, Qualität der Annotationswerkzeuge, Klarheit der Richtlinien und Erfahrung der Annotatoren erheblich. Sie sollten als illustrativ, nicht als verbindlich betrachtet werden.

> **See also:** Annotationswerkzeuge werden in Kapitel 12 behandelt. Statistische Modelle für Crowdsourcing-Annotation (Dawid-Skene, MACE) werden in Kapitel 13 behandelt.

Aktives Lernen

Annotierte Daten sind teuer. Die zentrale Erkenntnis des aktiven Lernens ist, dass *nicht alle unannotierten Beispiele gleich informativ sind* — ein Modell kann sich schneller verbessern, wenn es wählen darf, welche Beispiele es abfragt. Anstatt Daten zufällig zu annotieren, fragt ein aktives Lernsystem ein Orakel (in der Regel einen menschlichen Annotator) bei den Beispielen an, die das Modell am wahrscheinlichsten verbessern werden.

Dieses Kapitel behandelt die Theorie und Praxis des aktiven Lernens: Anfragestrategien, Stichprobenrahmen, Abbruchkriterien und praktische Überlegungen für reale Einsätze.

Das aktive Lernsetup

Das Standard-**Pool-basierte aktive Lernsetup** umfasst:

- Eine **annotierte Menge** $\mathcal{L} = \{(x_i, y_i)\}_{i=1}^n$ — anfangs klein
- Einen **unannotierten Pool** $\mathcal{U} = \{x_j\}_{j=1}^m$ — typischerweise viel größer als $\mathcal{L}$
- Ein **Orakel** $\mathcal{O}$, das $y = \mathcal{O}(x)$ für beliebige abgefragte x zurückgeben kann
- Eine **Anfragestrategie** ϕ, die die nächste Anfrage $x^* = \phi(\mathcal{L}, \mathcal{U}, f_\theta)$ auswählt

Die aktive Lernschleife:

```
1. Initialisierung: L = kleine annotierte Startmenge, U = unannotierter Pool
2. Training: f_θ ← train(L)
3. Anfrage: x∗ = argmax φ(x; f_θ) über x ∈ U
4. Kennzeichnung: y∗ = O(x∗)
5. Aktualisierung: L ← L ∪ {(x∗, y∗)}, U ← U \ {x∗}
→ Wiederholen ab 2 bis Budget erschöpft
```

Das Ziel ist, eine Zielmodellqualität mit so wenigen Orakel-Anfragen wie möglich zu erreichen.

Theoretische Grundlagen

Eine natürliche Frage ist: Wie viel kann aktives Lernen helfen? Im besten Fall kann aktives Lernen *exponentielle* Reduktionen in der Labelkomplexität erzielen — Fehler ϵ mit $O(\log(1/\epsilon))$ Labels statt der $O(1/\epsilon)$ passiven Lernens, zumindest in realisierbaren Szenarien mit einer guten Anfragestrategie {cite}`settles2009active`.

In der Praxis sind Garantien schwerer zu erhalten. **Agnostisches aktives Lernen** {cite}`balcan2006agnostic` zeigt, dass Label-Einsparungen möglich sind, selbst wenn das Zielkonzept nicht in der Hypothesenklasse liegt, aber die Einsparungen hängen stark vom Uneinigkeitskoeffizienten

ab — einem Maß dafür, wie schnell sich die Menge plausibler Hypothesen verkleinert, wenn Daten gesammelt werden.

Die wesentliche praktische Implikation: Der Vorteil aktiven Lernens ist am größten, wenn die **Entscheidungsgrenze einfach und konzentriert** ist (sodass Unsicherheitsanfragen schnell falsche Hypothesen eliminieren), und am kleinsten, wenn die Hypothesenklasse groß oder die Grenze komplex ist.

Anfragestrategien

Unsicherheitsstichproben

Die einfachste und am häufigsten verwendete Strategie: das Beispiel abfragen, bei dem das Modell am *unsichersten* ist {cite}`lewis1994sequential`.

Geringste Konfidenz fragt das Beispiel ab, bei dem das Modell am wenigsten in seiner besten Vorhersage sicher ist:

$$x^* = \arg\max_{x \in \mathcal{U}} \left(1 - P_\theta(\hat{y} \mid x)\right)$$

Randstichproben betrachten die Lücke zwischen den beiden besten vorhergesagten Wahrscheinlichkeiten:

$$x^* = \arg\min_{x \in \mathcal{U}} \left(P_\theta(\hat{y}_1 \mid x) - P_\theta(\hat{y}_2 \mid x)\right)$$

Entropiestichproben verwenden die vollständige vorhergesagte Verteilung:

$$x^* = \arg\max_{x \in \mathcal{U}} \left(-\sum_{k=1}^{K} P_\theta(y_k \mid x) \log P_\theta(y_k \mid x) \right)$$

Entropiestichproben sind die fundierteste der drei — sie berücksichtigen alle Klassen — und übertreffen die anderen in der Regel bei Mehrklassen-Problemen.

Query by Committee (QbC)

Trainieren eines **Komitees** von C Modellen (mittels Bagging, unterschiedlicher Initialisierungen oder unterschiedlicher Architekturen). Das Beispiel abfragen, bei dem das Komitee am meisten uneinig ist:

$$x^* = \arg\max_{x \in \mathcal{U}} \ \text{Uneinigkeit}(\{f_c(x)\}_{c=1}^{C})$$

Uneinigkeit kann als **Abstimmungs-Entropie** (Entropie über die Mehrheitsabstimmungen des Komitees) oder **KL-Divergenz** vom Konsensus gemessen werden.

QbC liefert bessere Unsicherheitsschätzungen als ein einzelnes Modell, erfordert aber das Training mehrerer Modelle, was rechenintensiv ist.

Erwartete Modelländerung

Das Beispiel abfragen, das die größte Änderung am aktuellen Modell bewirken würde, wenn es annotiert würde. Für gradientenbasierte Modelle entspricht dies dem Beispiel mit der größten erwarteten Gradientengröße {cite}`settles2008analysis`:

$$x^* = \arg\max_{x \in \mathcal{U}} \sum_{y \in \mathcal{Y}} P_\theta(y \mid x) \, \|\nabla_\theta \mathcal{L}(f_\theta(x), y)\|$$

Diese Strategie hat eine starke theoretische Motivation, erfordert aber die Berechnung von Gradienten für jeden Kandidaten, was sie für große Modelle teuer macht.

Core-Set / Geometrische Ansätze

Unsicherheitsbasierte Strategien können **gegenüber Ausreißern voreingenommen** sein: Ein ungewöhnliches Beispiel kann sehr unsicher, aber nicht repräsentativ für die Datenverteilung sein. Core-Set-Methoden begegnen diesem Problem, indem sie eine vielfältige Stichprobe suchen, die den Merkmalsraum abdeckt.

Der **k-Center-Gierig-Algorithmus** {cite}`sener2018active` findet die kleinste Menge von Punkten, sodass jeder unannotierte Punkt innerhalb von δ mindestens eines abgefragten Punktes liegt:

$$x^* = \arg\max_{x \in \mathcal{U}} \min_{x' \in \mathcal{L}} d(x, x')$$

d. h. den Punkt abfragen, der am weitesten von einem aktuell annotierten Punkt entfernt ist. Dies fördert eine gut verteilte Menge von Annotationen.

BADGE

Batch Active learning by Diverse Gradient Embeddings {cite}`ash2020deep` kombiniert Unsicherheit und Vielfalt: Es wählt eine Charge von Beispielen aus, deren Gradienten-Einbettungen (bezüglich des vorhergesagten Labels) sowohl groß in der Größenordnung (unsicher) als auch vielfältig sind (verschiedene Regionen des Gradientenraums abdeckend). Dies ist eine der wettbewerbsfähigsten modernen Strategien.

Unsicherheitsschätzung für tiefe Modelle

Die oben genannten Strategien setzen Zugang zu kalibrierten Wahrscheinlichkeitsausgaben des Modells voraus. Für einfache Modelle (logistische Regression, Softmax-Klassifikatoren) ist dies unkompliziert. Für tiefe Netze sind zusätzliche Techniken erforderlich, um zuverlässige Unsicherheitsschätzungen zu erhalten.

Zwei Arten von Unsicherheit

Nach Kendall und Gal {cite}`kendall2017uncertainties` unterscheiden wir:

Aleatorische Unsicherheit (Datenunsicherheit): Inhärentes Rauschen in den Beobachtungen, das durch das Sammeln weiterer Daten nicht reduziert werden kann. Ein unscharfes Bild ist aleatorisch unsicher — keine Menge zusätzlicher Trainingsdaten aus derselben Verteilung wird das Modell darin sicherer machen.

Epistemische Unsicherheit (Modellunsicherheit): Unsicherheit aufgrund begrenzter Trainingsdaten oder eines Modells, das ähnliche Beispiele nicht gesehen hat. Epistemische Unsicherheit *kann* durch die Annotation weiterer Daten reduziert werden — und ist daher die relevante Größe für die Auswahl der aktiven Lernanfragen.

Für aktives Lernen wollen wir Beispiele mit hoher epistemischer Unsicherheit abfragen, nicht mit hoher aleatorischer Unsicherheit. Das Abfragen eines grundlegend ambiguem Beispiels verschwendet den Aufwand des Orakels: Kein Label, das es bereitstellt, wird eindeutig korrekt sein.

Monte-Carlo-Dropout

Ein praktischer Ansatz zur epistemischen Unsicherheitsschätzung für neuronale Netze ist **MC-Dropout** {cite}`gal2016dropout`: Dropout zur Inferenzzeit anwenden und T Forward-Passes ausführen. Die Varianz über die Vorhersagen ist eine Schätzung der epistemischen Unsicherheit.

```
import numpy as np
import torch
import torch.nn as nn

torch.manual_seed(42)
rng = np.random.default_rng(42)

class MCDropoutNet(nn.Module):
    def __init__(self, input_dim=20, hidden=64, output_dim=2, p_drop=0.3):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden), nn.ReLU(), nn.Dropout(p_drop),
            nn.Linear(hidden, hidden),    nn.ReLU(), nn.Dropout(p_drop),
            nn.Linear(hidden, output_dim)
        )

    def forward(self, x):
        return self.net(x)

def mc_uncertainty(model, x, T=30):
    """
    Run T stochastic forward passes with dropout active.
    Returns mean prediction and epistemic uncertainty (predictive variance).
    """
    model.train()  # keep dropout active during inference
    with torch.no_grad():
        preds = torch.stack([
            torch.softmax(model(x), dim=-1) for _ in range(T)
        ])  # shape: (T, N, C)
    mean_pred = preds.mean(0)
    # Epistemic uncertainty: mean of variances across passes
    epistemic = preds.var(0).sum(-1)
    # Aleatoric uncertainty: entropy of mean prediction
    aleatoric = -(mean_pred * (mean_pred + 1e-9).log()).sum(-1)
    return mean_pred, epistemic, aleatoric
```

```
# Quick demonstration
model = MCDropoutNet(input_dim=20, output_dim=2)
# In-distribution example (simulated)
x_familiar   = torch.randn(1, 20) * 0.5
# Out-of-distribution example (far from training distribution)
x_unfamiliar = torch.randn(1, 20) * 3.0

for name, x in [("In-distribution ", x_familiar), ("Out-of-distribution", x_unfamiliar)]:
    _, ep, al = mc_uncertainty(model, x)
    print(f"{name} | epistemic: {ep.item():.4f} | aleatoric: {al.item():.4f}")
```

Im untrainierten Netzwerk oben zeigen beide Beispiele ähnliche Unsicherheit. Nach dem Training wird das Out-of-Distribution-Beispiel höhere epistemische Unsicherheit zeigen — das Modell hat keine zuverlässige Abbildung für Eingaben weit entfernt von der Trainingsverteilung gelernt.

Tiefe Ensemble-Modelle

Das Training von M unabhängig initialisierten Modellen und die Mittelung ihrer Vorhersagen liefert eine einfachere und oft zuverlässigere Unsicherheitsschätzung als MC-Dropout {cite}`lakshminarayanan2017simple`. Die Uneinigkeit zwischen Ensemble-Mitgliedern ist das epistemische Unsicherheitssignal.

Für aktives Lernen im großen Maßstab fügen sowohl MC-Dropout als auch tiefe Ensembles einen Overhead proportional zu T bzw. M Forward-Passes hinzu. In der Praxis sind $T = 10$–30 für MC-Dropout oder $M = 5$ Ensemble-Mitglieder oft ausreichend, um Beispiele nach epistemischer Unsicherheit zu ordnen, auch wenn die absoluten Werte nicht gut kalibriert sind.

Eine vollständige aktive Lernschleife

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from copy import deepcopy

rng = np.random.default_rng(42)

# Generate dataset
X, y = make_classification(
    n_samples=2000, n_features=20, n_informative=10,
    n_classes=3, n_clusters_per_class=1, random_state=42
)
X_train, y_train = X[:1500], y[:1500]
X_test,  y_test  = X[1500:], y[1500:]

def entropy_query(model, X_pool):
    """Return index of most uncertain sample (entropy)."""
    probs = model.predict_proba(X_pool)
    ent = -np.sum(probs * np.log(probs + 1e-9), axis=1)
```

```
    return np.argmax(ent)

def random_query(X_pool):
    """Random baseline."""
    return rng.integers(0, len(X_pool))

def run_active_learning(strategy='entropy', n_initial=30, n_queries=120, query_batch=5):
    labeled_idx = list(rng.choice(len(X_train), n_initial, replace=False))
    unlabeled_idx = [i for i in range(len(X_train)) if i not in labeled_idx]
    accs = []

    for step in range(n_queries // query_batch):
        model = LogisticRegression(max_iter=500, C=1.0)
        model.fit(X_train[labeled_idx], y_train[labeled_idx])
        accs.append(accuracy_score(y_test, model.predict(X_test)))

        # Query
        X_pool = X_train[unlabeled_idx]
        for _ in range(query_batch):
            if strategy == 'entropy':
                q = entropy_query(model, X_pool)
            else:
                q = random_query(X_pool)
            labeled_idx.append(unlabeled_idx.pop(q))
            X_pool = X_train[unlabeled_idx]

    return np.array(accs)

labels_used = np.arange(1, 25) * 5 + 30  # label counts at each step

accs_active = run_active_learning(strategy='entropy')
accs_random = run_active_learning(strategy='random')

plt.figure(figsize=(8, 5))
plt.plot(labels_used, accs_active, 'o-', label='Entropy sampling', color='#2b3a8f', linewidth
plt.plot(labels_used, accs_random, 's--', label='Random baseline',  color='#e05c5c', linewidt
plt.xlabel("Number of labeled examples", fontsize=12)
plt.ylabel("Test accuracy", fontsize=12)
plt.title("Active Learning vs. Random Sampling", fontsize=13)
plt.legend(fontsize=11)
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('active_learning_curve.png', dpi=150)
plt.show()

print(f"Active learning reaches {accs_active[-1]:.3f} accuracy")
print(f"Random sampling reaches {accs_random[-1]:.3f} accuracy")
print(f"Active learning saves ~{int((accs_random.tolist().index(min(accs_random, key=lambda a
```

Das Kaltstart-Problem

Aktives Lernen erfordert ein trainiertes Modell, um unannotierte Punkte zu bewerten — aber am Anfang gibt es keine (oder sehr wenige) annotierten Beispiele. Dies ist das **Kaltstart-Problem**.

Praktische Lösungen:

1. **Zufällige Initialisierung:** Eine kleine zufällige Startmenge annotieren (20–100 Beispiele), bevor aktives Lernen beginnt.
2. **Clustering-basierte Initialisierung:** K-Means auf dem unannotierten Pool anwenden; ein Beispiel aus jedem Cluster annotieren. Dies gewährleistet Vielfalt in der anfänglichen annotierten Menge.
3. **Embedding-basierte Auswahl:** Einen vortrainierten Encoder verwenden, um Beispiele einzubetten; eine vielfältige Teilmenge über Core-Set auswählen.

Für die meisten Aufgaben sind einige Dutzend zufällige Start-Labels typischerweise ausreichend, um aktives Lernen zu starten; die genaue Anzahl hängt von Klassenbalance, Merkmalsdimensionalität und Modellkomplexität ab.

Stapelmodus-Aktives-Lernen

In der Praxis arbeiten Annotatoren in Stapeln — es ist ineffizient, nach jedem einzelnen Label ein neues Modell zu trainieren und einzusetzen. **Stapelmodus-Aktives-Lernen** wählt eine Menge von b Beispielen aus, die gleichzeitig annotiert werden sollen.

Die naive Auswahl der Top-b unsichersten Beispiele führt zu **Redundanz**: Hochunsichere Beispiele tendieren dazu, sich zu gruppieren (z. B. Beispiele nahe der Entscheidungsgrenze im selben Bereich). Bessere Stapelstrategien optimieren sowohl für Unsicherheit *als auch* für Vielfalt innerhalb des Stapels.

Deterministische Punktprozesse (DPPs) bieten eine prinzipielle Möglichkeit, vielfältige Stapel abzutasten: Sie definieren eine Verteilung über Teilmengen, die ähnliche Elemente bestraft. Die Wahrscheinlichkeit einer Teilmenge S unter einem DPP ist proportional zu $\det(L_S)$, wobei L eine Kernmatrix ist, die Ähnlichkeit kodiert.

Abbruchkriterien

Wann sollte aktives Lernen aufhören? Häufige Kriterien:

- **Budget erschöpft:** Am einfachsten — aufhören, wenn das Annotationsbudget aufgebraucht ist.
- **Leistungsplateau:** Aufhören, wenn die Modellgenauigkeit auf einem gehaltenen Validierungssatz sich in k aufeinanderfolgenden Runden um nicht mehr als δ verbessert hat.
- **Konfidenz-Schwellenwert:** Aufhören, wenn weniger als ein bestimmter Anteil unannotierter Beispiele eine Unsicherheit über einem Schwellenwert aufweist.
- **Maximale Verlust-Reduktion:** Den maximal möglichen Gewinn aus zusätzlichen Labels schätzen; aufhören, wenn dieser unter einen Schwellenwert fällt {cite}`bloodgood2009method`.

Wann aktives Lernen funktioniert (und wann nicht)

Aktives Lernen funktioniert tendenziell gut, wenn: - Annotation teuer ist und der unannotierte Pool groß ist - Die Daten eine klare Struktur haben, die das Modell nutzen kann, um informative Beispiele zu identifizieren - Die Modellklasse für die Aufgabe geeignet ist

Aktives Lernen funktioniert schlecht, wenn: - Das anfängliche Modell sehr schwach ist (Kaltstart) und Beispiele nicht sinnvoll ordnen kann - Die Anfragestrategie Ausreißer oder falsch annotierte Beispiele auswählt (Rauschrobustheit ist wichtig) - Die Datenverteilung zwischen dem unannotierten Pool und der Testverteilung abweicht

Ein wichtiges praktisches Problem ist **Verteilungsabweichung**: Aktives Lernen tendiert dazu, Beispiele nahe der Entscheidungsgrenze abzufragen, was eine voreingenommene annotierte Menge erzeugt, die die Testverteilung möglicherweise nicht gut repräsentiert. Dies kann zu gut trainierten Entscheidungsgrenzen, aber schlechter Kalibrierung führen.

> **See also:** Die grundlegende Übersicht ist {cite}`settles2009active`. Theoretische Grundlagen (Labelkomplexität, agnostische Grenzen): {cite}`balcan2006agnostic`. Für Deep-Learning-spezifisches aktives Lernen, siehe {cite}`ash2020deep` (BADGE) und {cite}`sener2018active` (Core-Set). Für eine kritische Bewertung, wann aktives Lernen tatsächlich hilft, siehe {cite}`lowell2019practical`. Zu aleatorischer vs. epistemischer Unsicherheit für tiefe Modelle, siehe {cite}`kendall2017uncertainties`; für tiefe Ensembles als Unsicherheitsschätzer, siehe {cite}`lakshminarayanan2017simple`; für MC-Dropout als approximative bayesianische Inferenz, siehe {cite}`gal2016dropout`.

Interaktives maschinelles Lernen

Aktives Lernen stellt eine fokussierte Frage: Welche Beispiele soll ich bei gegebenem Budget annotieren? Interaktives maschinelles Lernen (IML) stellt eine umfassendere Frage: Wie gestalten wir die *gesamte Interaktion* zwischen einem Menschen und einem Lernsystem maximal produktiv, angenehm und korrekt?

IML zeichnet sich durch die **Unmittelbarkeit** und **Direktheit** der Mensch-Modell-Rückkopplungsschleife aus. Während traditionelles ML den Menschen dazu bringt, Daten zu übergeben und auf den Abschluss des Trainings zu warten, ermöglicht IML dem Menschen, das Modellverhalten zu beobachten, Feedback zu geben und zu sehen, wie das Modell reagiert — oft innerhalb von Sekunden.

Prinzipien des interaktiven maschinellen Lernens

Amershi et al. {cite}`amershi2014power` identifizieren drei definierende Merkmale von IML:

1. Schnelles Feedback: Das Modell aktualisiert sich schnell genug, damit Menschen den Effekt ihres Feedbacks wahrnehmen können. Im Grenzfall erfolgen Modellaktualisierungen in Echtzeit.

2. Direkte Manipulation: Der Mensch interagiert mit dem Modell durch die Daten oder durch die Vorhersagen des Modells — nicht durch Konfigurationsdateien oder Hyperparameter-Tuning.

3. Iterative Verfeinerung: Der Prozess ist genuinen iterativer Natur: Die nächste Aktion des Menschen wird durch das aktuelle Verhalten des Modells informiert, das durch die vorherigen Aktionen des Menschen geformt wurde.

Dies schafft eine enge **ko-adaptive Schleife**: Sowohl der Mensch als auch das Modell verändern sich im Laufe der Zeit in Reaktion aufeinander. Der Mensch erlernt, was das Modell versteht; das Modell erlernt, was dem Menschen wichtig ist.

Vergleich mit aktivem Lernen

IML und aktives Lernen überschneiden sich erheblich, sind aber nicht identisch:

Eigenschaft	Aktives Lernen	Interaktives ML
Primäre Frage	Was annotieren?	Wie interagieren?
Feedback-Latenz	Kann stapelweise sein (Tage)	Typischerweise Echtzeit oder nah

Eigenschaft	Aktives Lernen	Interaktives ML
Modell-Aktualisierungsfrequenz	Pro Runde (Stapel)	Pro Interaktion (online)
Menschliche Handlungsmacht	Beantwortet Fragen des Modells	Kann das Modell proaktiv lehren
Interface-Design	Nebensächlich	Zentrales Anliegen
Kognitive Belastung des Menschen	Nicht explizit modelliert	Explizit berücksichtigt

Beim aktiven Lernen leitet die Maschine die Interaktion. Bei IML kann auch der Mensch die Initiative ergreifen — indem er Beispiele, Korrekturen oder Feedback zu beliebigen Aspekten des Modellverhaltens liefert, die am problematischsten erscheinen.

Mixed-Initiative-Interaktion

Mixed-Initiative-Systeme erlauben sowohl dem Menschen als auch der Maschine, zu verschiedenen Zeitpunkten die Führung zu übernehmen {cite}`allen1999mixed`. Ein rein maschinengesteuertes System stellt Fragen und der Mensch antwortet. Ein rein menschengesteuertes System überlässt dem Menschen alle Entscheidungen. Mixed-Initiative-Systeme balancieren beides.

In der Praxis kombinieren die besten IML-Systeme: - **Maschineninitiative:** „Ich bin bei diesen Beispielen unsicher — können Sie sie kennzeichnen?" - **Menscheninitiative:** „Ich bemerke, dass das Modell bei dieser Kategorie konsistent falsch liegt — lassen Sie mich weitere Beispiele bereitstellen" - **Bestätigung:** Das Modell zeigt sein aktuelles Verständnis; der Mensch bestätigt oder korrigiert

Gute IML-Interfaces machen das aktuelle Verständnis des Modells sichtbar und korrigierbar. Dies ist die **Verständlichkeits-Anforderung**: Menschen können ein Modell nur lenken, das sie zumindest annäherungsweise verstehen.

Menschliche Faktoren in IML

IML bringt menschliche Faktoren — kognitive Belastung, Ermüdung, Konsistenz und Vertrauen — direkt in die Lernschleife. Schlechtes IML-Design führt zu:

Annotationsmüdigkeit: Menschen treffen in längeren Sitzungen schnellere, unachtsame Entscheidungen. Fehler, die in die Trainingsdaten gelangen.

Ankerungsverzerrung: Menschen verlassen sich zu sehr auf die aktuellen Vorschläge des Modells. Wenn ein Interface die Vorhersage des Modells voranzeigt, sind Annotatoren weniger geneigt, sie zu korrigieren, selbst wenn sie falsch ist — eine systematische Quelle von Label-Rauschen, die sich über Annotationsrunden hinweg verstärkt {cite}`geva2019annotator`. Vorannotation kann den Durchsatz steigern {cite}`lingren2014evaluating`, während sie gleichzeitig die Rate verringert, mit der Annotatoren Modellfehler erkennen; diese beiden Effekte müssen beim IML-Interface-Design gegeneinander abgewogen werden.

Fehlkalibriertes Vertrauen: Menschen vertrauen dem Modell entweder zu sehr (akzeptieren falsche Modellausgaben) oder zu wenig (ignorieren korrekte Vorschläge). Beide Muster verringern den Wert der Mensch-Modell-Zusammenarbeit.

Sitzungskonsistenz: Menschen können bei demselben Beispiel zu verschiedenen Zeitpunkten, besonders nach langen Sitzungen, unterschiedliche Entscheidungen treffen. Konsistenzprüfungen (erneutes Präsentieren früherer Beispiele) können dies erkennen und korrigieren.

Gutes IML-Design mindert diese Probleme durch Interface-Entscheidungen: explizite Darstellung der Modellkonfidenz, Randomisierung der Anzeigereihenfolge, Begrenzung der Sitzungsdauer und Einbau von Konsistenzprüfungen.

Die IML-Feedbacktypen in der Praxis

Feedback auf Beispielebene

Der Mensch stellt ein Label oder eine Korrektur für ein bestimmtes Beispiel bereit. Dies ist die häufigste Form und direkt kompatibel mit überwachtem Lernen.

Feedback auf Merkmalsebene

Der Mensch gibt an, welche Merkmale relevant oder irrelevant sind. „Das Modell sollte auf die Wörter 'dringend' und 'Frist' für diese Kategorie achten." Dies ist ausdrucksvoller als Feedback auf Beispielebene und kann für bestimmte Aufgaben effizienter sein.

TFIDF Interactive und ähnliche Systeme erlauben Annotatoren, relevante Wörter in Textdokumenten hervorzuheben. Diese Hervorhebungen werden in Einschränkungen oder zusätzliche Aufsicht über die Aufmerksamkeit des Modells umgewandelt.

Feedback auf Modellebene

Der Mensch korrigiert das Verhalten des Modells für eine Klasse von Eingaben direkt: „Immer wenn die Eingabe [X] enthält, sollte die Ausgabe [Y] sein." Dies entspricht logischen Regeln oder Einschränkungen in Ansätzen wie Posterior Regularization {cite}`ganchev2010posterior` oder einschränkungsgesteuertem Lernen.

Fallstudie: Google Teachable Machine

Teachable Machine ist ein zugängliches webbasiertes IML-System, das nicht-technischen Nutzern ermöglicht, im Browser Bildklassifikatoren zu trainieren. Der Nutzer:

1. Nimmt Beispiele jeder Klasse mit der Webcam auf
2. Trainiert das Modell mit einem einzigen Klick (Fine-Tuning eines MobileNet im Browser)
3. Sieht sofort die Vorhersagen des Modells auf Live-Video
4. Fügt weitere Beispiele für Klassen hinzu, bei denen das Modell falsch liegt

Dies veranschaulicht die zentrale IML-Schleife: Beispiele bereitstellen Modell beobachten Fehler identifizieren gezieltere Beispiele bereitstellen. Das Echtzeit-Feedback (Modellausgaben aktualisieren sich in Echtzeit, typischerweise mit interaktiven Bildraten auf moderner Hardware) macht die ko-adaptive Schleife unmittelbar spürbar.

Implementierung einer einfachen IML-Schleife

```
import numpy as np
from sklearn.linear_model import SGDClassifier
from sklearn.preprocessing import StandardScaler
import warnings
warnings.filterwarnings('ignore')

class SimpleIMLSystem:
    """
    Minimal IML system that allows online feedback and displays
    model state after each annotation.
    """

    def __init__(self, n_features=10, n_classes=2):
        self.model = SGDClassifier(loss='log_loss', max_iter=1, warm_start=True,
                                   random_state=42)
        self.scaler = StandardScaler()
        self.X_seen = []
        self.y_seen = []
        self.n_classes = n_classes
        self.initialized = False

    def update(self, x, y_true):
        """Receive a single labeled example and update the model."""
        self.X_seen.append(x)
        self.y_seen.append(y_true)

        if len(self.X_seen) >= 2 * self.n_classes:
            X_arr = np.array(self.X_seen)
            y_arr = np.array(self.y_seen)
            X_scaled = self.scaler.fit_transform(X_arr)
            self.model.partial_fit(X_scaled[-1:], y_arr[-1:],
                                   classes=list(range(self.n_classes)))
            self.initialized = True

        return self

    def predict_with_confidence(self, x):
        """Predict label and return confidence."""
        if not self.initialized:
            return None, 0.0
        x_scaled = self.scaler.transform(x.reshape(1, -1))
        probs = self.model.predict_proba(x_scaled)[0]
        return self.model.predict(x_scaled)[0], probs.max()

    def current_accuracy(self, X_val, y_val):
        if not self.initialized:
            return None
```

```
        X_scaled = self.scaler.transform(X_val)
        return (self.model.predict(X_scaled) == y_val).mean()


# Simulate an IML session
rng = np.random.default_rng(42)
X_all, y_all = np.random.default_rng(0).random((500, 10)), np.random.default_rng(0).integers(
X_val, y_val = X_all[400:], y_all[400:]

system = SimpleIMLSystem(n_features=10, n_classes=2)
accs, confidences = [], []

print("Step | Labels | Accuracy | Example confidence")
print("-" * 50)

for step in range(100):
    x, y = X_all[step], y_all[step]
    pred, conf = system.predict_with_confidence(x)
    system.update(x, y)

    if (step + 1) % 10 == 0:
        acc = system.current_accuracy(X_val, y_val)
        if acc is not None:
            accs.append(acc)
            print(f"  {step+1:3d} |  {step+1:4d}  |  {acc:.3f}   | {conf:.3f}")
```

Der Großmutter-Test

Eine nützliche Heuristik zur Bewertung des IML-Interface-Designs — und des HITL-Systemdesigns im Allgemeinen — ist das, was wir den **Großmutter-Test** nennen werden (eine hier eingeführte Originalprägung als Designbeschränkung, nicht als Verweis auf frühere Arbeit):

> *Eine 1930 geborene Frau sollte dieses Gerät per Stimme bedienen können, und bei Frustration graceful auf eine Tastatur- oder Textoberfläche zurückfallen können.*

Der Test dreht sich nicht primär um Barrierefreiheit, auch wenn er das auch ist. Es geht um **Design für Reibung**. Wenn ein System ein mentales Modell neuronaler Netze, Trainingsschleifen oder Wahrscheinlichkeitsverteilungen erfordert, um es effektiv zu nutzen, hat es den Großmutter-Test nicht bestanden. Der Mensch im Loop sollte teilnehmen können, ohne die Maschinenseite der Schleife zu verstehen.

Die Implikationen für das IML-Design sind konkret:

Stimme-zuerst-Fallback: Die primäre Interaktionsmodalität sollte natürliche Sprache oder Geste sein — keine Parameter-Schieberegler oder Konfidenz-Schwellenwerte. Experten mögen Schieberegler wollen; jeder sollte „das ist falsch" sagen können.

Graceful Degradation: Wenn die bevorzugte Modalität des Nutzers versagt oder frustriert, sollte das System eine Alternative anbieten — keinen leeren Bildschirm oder eine Fehlermeldung. Das Interface ist Teil des Lernsystems; ein Nutzer, der nicht interagieren kann, kann nicht lehren.

Lesbarer Modellzustand: Das aktuelle Verständnis des Modells sollte in menschlichen Begriffen sichtbar sein. Nicht „Konfidenz: 0,73", sondern „Ich bin ziemlich sicher, dass dies [X] ist, aber ich habe Beispiele wie dieses in beide Richtungen gehen gesehen." Unsicherheit sollte in einer Sprache kommuniziert werden, die zur Korrektur einlädt.

Toleranz gegenüber Ambiguität: Ein 93-jähriger Nutzer und ein 23-jähriger ML-Ingenieur werden unterschiedlich mit demselben System interagieren. Der Großmutter-Test fragt, ob das System beide aufnehmen kann — nicht indem es das Alter des Nutzers erkennt, sondern indem es Interaktionen gestaltet, die über ein Spektrum von Expertise und Komfort funktionieren.

Der Test gewinnt besondere Bedeutung, da ML-Systeme von Forschungswerkzeugen zu alltäglicher Infrastruktur werden. Ein medizinisches Bildanalyse-Assistenzsystem für Radiologen, ein juristischer Dokumentenklassifikator für Justizkaufleute, ein Bildungsfeedbacksystem für Lehrkräfte — jedes umfasst Menschen-im-Loop, die sich nicht als KI-Trainer angemeldet haben. Für sie zu entwerfen ist kein Zugeständnis; es ist der Punkt.

:::{admonition} Designprinzip :class: tip Der Großmutter-Test ist eine Designbeschränkung, kein Zielpublikum. Systeme, die ihn bestehen, sind robuster gegenüber Nutzervielfalt, toleranter gegenüber Expertise-Lücken und ehrlicher darüber, was sie von Menschen-im-Loop erwarten. Wenn ein System vor der Nutzung eine Erklärung erfordert, bittet es den Menschen, zusätzliche Arbeit zu leisten. Diese Arbeit sollte durch proportionalen Nutzen gerechtfertigt sein. :::

IML und Basismodelle

Modernes IML nutzt zunehmend **vortrainierte Basismodelle** {cite}`bommasani2021opportunities` als Grundlage. Anstatt von Grund auf zu trainieren, verfeinern Nutzer ein großes vortrainiertes Modell mit einer kleinen Anzahl interaktiver Beispiele. Dies kann die Anzahl der benötigten Beispiele erheblich reduzieren — in günstigen Fällen nur 5–50 Beispiele statt Tausender, abhängig davon, wie gut die vortrainierten Repräsentationen zur Zielaufgabe passen {cite}`bommasani2021opportunities`.

Techniken, die dies ermöglichen: - **Few-Shot-Prompting:** Beispiele im Kontextfenster des LLM bereitstellen - **Adapter-Fine-Tuning:** Kleine Adaptermodule aktualisieren, während das Basismodell eingefroren bleibt - **Parameter-effizientes Fine-Tuning (PEFT):** LoRA, Prefix-Tuning und ähnliche Methoden, die schnelle, ressourcenarme Aktualisierungen ermöglichen

Basismodelle ändern die IML-Dynamik: Menschen lehren nicht mehr ein leeres Modell von Grund auf, sondern *steuern* ein Modell, das bereits sehr viel weiß. Die Herausforderung verlagert sich von „wie genug Beispiele bereitstellen" zu „wie genau spezifizieren, was wir anders wollen als das, was das Modell bereits tut."

> **See also:** Die Übersicht von {cite}`amershi2014power` bleibt die beste Einführung in IML-Prinzipien. Für Mixed-Initiative-Systeme im Besonderen, siehe {cite}`allen1999mixed`. Zu Ankerungseffekten bei der Annotation, siehe {cite}`geva2019annotator` und {cite}`lingren2014evaluating`.

Bestärkendes Lernen aus menschlichem Feedback

Keine Technik hat HITL ML mehr in den Mainstream gebracht als Reinforcement Learning from Human Feedback (RLHF). Es ist der Mechanismus hinter InstructGPT {cite}`ouyang2022training` und ein Kernbestandteil von Instruktionsfolge-Pipelines in vielen modernen großen Sprachmodellen {cite}`stiennon2020learning`. RLHF zu verstehen — nicht nur als zu befolgendes Rezept, sondern als prinzipiellen Ansatz zur Ausrichtung — ist für jeden, der in der modernen KI arbeitet, unerlässlich.

Das Ausrichtungsproblem

Große Sprachmodelle (LLMs), die rein auf Next-Token-Vorhersage trainiert wurden, optimieren für ein Proxy-Ziel: vorhersagen, welcher Text als nächstes in einem Korpus menschlich geschriebener Texte kommt. Dieses Ziel ist verwandt mit, aber verschieden von dem, was wir tatsächlich wollen: Antworten, die hilfreich, präzise, sicher und an menschliche Werte angepasst sind.

Die Diskrepanz zwischen dem Trainingsziel und dem gewünschten Verhalten wird als **Ausrichtungsproblem** {cite}`russell2019human` bezeichnet. Konkret lernt ein auf Internet-Text trainiertes Sprachmodell: - Plausibel klingende Fortsetzungen zu produzieren (die sachlich falsch sein können) - Die Verzerrungen und Schäden in Trainingsdaten widerzuspiegeln - Ausweichend oder manipulativ zu sein, wenn dies statistisch auf den Prompt folgt

RLHF adressiert die Ausrichtung, indem menschliche Präferenzen *Teil des Optimierungsziels* werden.

Die RLHF-Pipeline

RLHF verläuft in drei Phasen:

```
Phase 1: Supervised Fine-Tuning (SFT)
  --> Demonstrationsdaten sammeln (Mensch schreibt ideale Antworten)
  --> Basis-LLM auf Demonstrationen feinabstimmen

Phase 2: Training des Belohnungsmodells
  --> Paarweise Präferenzen sammeln (Mensch bewertet A vs. B)
  --> Belohnungsmodell R(x, y) trainieren, um menschliche Präferenzen vorherzusagen
```

```
Phase 3: RL-Feinabstimmung
  --> LLM mittels PPO/RL feinabstimmen, um R(x, y) zu maximieren
  --> KL-Strafe verhindert übermäßige Abweichung vom SFT-Modell
```

Phase 1: Supervised Fine-Tuning

Ausgehend von einem vortrainierten Basismodell π_0 sammeln wir einen Datensatz von (Prompt, ideale Antwort)-Paaren, geschrieben oder ausgewählt von menschlichen Auftragnehmern, die detaillierten Richtlinien folgen. Das Modell wird auf diesen Demonstrationen mit Standard-Kreuzentropie feinabgestimmt:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(x,y)\sim\mathcal{D}_{\text{demo}}}\left[\log \pi_\theta(y \mid x)\right]$$

Das SFT-Modell π_{SFT} ist ein viel besserer Ausgangspunkt für RLHF als das rohe vortrainierte Modell.

Phase 2: Training des Belohnungsmodells

Für eine Menge von Prompts $\{x_i\}$ generieren wir K Antworten pro Prompt mit π_{SFT} und präsentieren sie menschlichen Annotierenden als paarweise Vergleiche: „Welche Antwort ist besser, A oder B?"

Das Belohnungsmodell r_ϕ wird trainiert, um diese Präferenzen vorherzusagen. Unter dem **Bradley-Terry-Modell** (Kapitel 8) ist die Wahrscheinlichkeit, dass Antwort y_w gegenüber y_l bevorzugt wird:

$$P(y_w \succ y_l \mid x) = \sigma\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right)$$

Das Belohnungsmodell wird trainiert, den paarweisen Rankingverlust zu minimieren:

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}_{\text{pref}}}\left[\log \sigma\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right)\right]$$

Das Belohnungsmodell wird typischerweise vom SFT-Modell initialisiert, wobei der letzte Layer durch einen skalaren Kopf ersetzt wird.

Phase 3: RL-Feinabstimmung mit PPO

Mit einem trainierten Belohnungsmodell können wir Bestärkendes Lernen verwenden, um den LLM feinabzustimmen. Jeder Prompt x ist ein Zustand; jede Antwort y ist eine Trajektorie von Token-Entscheidungen; und die Belohnung ist $r_\phi(x, y)$.

Das Optimierungsziel enthält eine **KL-Divergenz-Strafe**, um zu verhindern, dass das Modell zu weit vom SFT-Ausgangspunkt abweicht (was zu Reward-Hacking führen würde {cite}`krakovna2020specification,gao2023scaling`):

$$\max_\theta \mathbb{E}_{x\sim\mathcal{D}, y\sim\pi_\theta(\cdot|x)}\left[r_\phi(x, y) - \beta \cdot \text{KL}\left[\pi_\theta(\cdot \mid x) \| \pi_{\text{SFT}}(\cdot \mid x)\right]\right]$$

Der Parameter β steuert die Stärke der KL-Strafe. Kleines β erlaubt mehr Optimierung, birgt aber das Risiko von Reward-Hacking; großes β hält das Modell nahe am SFT, begrenzt aber die Ausrichtungsgewinne.

Proximal Policy Optimization (PPO) {cite}`schulman2017proximal` ist der Standardalgorithmus für diese Phase, gewählt wegen seiner Stabilität gegenüber reinen Policy-Gradient-Methoden.

Eine vereinfachte RLHF-Demonstration

Die vollständige RLHF-Pipeline erfordert großskalige Infrastruktur. Das folgende Beispiel demonstriert die Kernideen im kleinen Maßstab.

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.nn import functional as F

torch.manual_seed(42)
rng = np.random.default_rng(42)

# ----------------------------------------------
# Toy setup: responses are 4-dimensional vectors
# "Quality" is known analytically (sum of positive values)
# We simulate a reward model learning this from pairwise feedback
# ----------------------------------------------

class RewardModel(nn.Module):
    def __init__(self, d=4, hidden=32):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(d, hidden), nn.ReLU(),
            nn.Linear(hidden, hidden), nn.ReLU(),
            nn.Linear(hidden, 1)
        )
    def forward(self, x):
        return self.net(x).squeeze(-1)

def true_quality(x):
    """The hidden ground-truth reward function."""
    return x.sum(dim=-1) + 0.5 * (x ** 2).mean(dim=-1)

# Generate pairwise preference data
N_PAIRS = 500
X1 = torch.randn(N_PAIRS, 4)
X2 = torch.randn(N_PAIRS, 4)
q1, q2 = true_quality(X1), true_quality(X2)
# Human prefers X1 when q1 > q2 (with some noise)
noise = torch.randn(N_PAIRS) * 0.5
preferred_1 = ((q1 - q2 + noise) > 0).float()

# Train reward model
rm = RewardModel(d=4, hidden=32)
optimizer = optim.Adam(rm.parameters(), lr=3e-3)
```

```
losses = []
for epoch in range(200):
    r1 = rm(X1)
    r2 = rm(X2)
    # Bradley-Terry loss
    logit = r1 - r2
    loss = F.binary_cross_entropy_with_logits(logit, preferred_1)
    optimizer.zero_grad(); loss.backward(); optimizer.step()
    losses.append(loss.item())

# Evaluate: does the reward model agree with true quality?
X_eval = torch.randn(1000, 4)
with torch.no_grad():
    r_pred = rm(X_eval).numpy()
    r_true = true_quality(X_eval).numpy()

corr = np.corrcoef(r_pred, r_true)[0, 1]
print(f"Reward model correlation with true quality: {corr:.4f}")
print(f"Final training loss: {losses[-1]:.4f}")

import matplotlib.pyplot as plt
plt.figure(figsize=(8, 3))
plt.subplot(1, 2, 1)
plt.plot(losses, color='#2b3a8f', linewidth=1.5)
plt.xlabel("Epoch"); plt.ylabel("Pairwise loss")
plt.title("Reward Model Training")

plt.subplot(1, 2, 2)
plt.scatter(r_true[:200], r_pred[:200], alpha=0.4, s=15, color='#2b3a8f')
plt.xlabel("True quality"); plt.ylabel("Predicted reward")
plt.title(f"Reward Model vs. Truth (r={corr:.3f})")
plt.tight_layout()
plt.savefig('reward_model.png', dpi=150)
plt.show()
```

Herausforderungen bei RLHF

Reward-Hacking

Ein zentraler Versagensmodus: Die Policy findet Wege, vom Belohnungsmodell hohe Belohnungen zu erhalten, die nicht echtem guten Verhalten entsprechen. Beispielsweise könnte ein LLM lernen, Antworten zu produzieren, die schmeichelnd oder selbstsicher klingen (was Annotatoren tendenziell hoch bewerten), anstatt präzise zu sein.

Reward-Hacking ist wahrscheinlicher, wenn: - Das Belohnungsmodell auf unzureichenden Präferenzdaten trainiert wurde - Der Policy erlaubt wird, weit vom SFT-Ausgangspunkt abzuweichen (kleines β) - Die Verteilung des Belohnungsmodells während des PPO-Trainings driftet

Minderungsstrategien: KL-Strafe, iteratives Belohnungsmodell-Training, diverse Evaluierung,

Constitutional-AI-Beschränkungen.

Evaluatorenverzerrung

Menschliche Annotatoren haben systematische Verzerrungen. Sie tendieren dazu, längere Antworten zu bevorzugen (Ausführlichkeitsverzerrung), selbstsicher klingende Texte (Konfidenzverzerrung) und Antworten, die mit ihren vorherigen Überzeugungen übereinstimmen. Diese Verzerrungen pflanzen sich in das Belohnungsmodell fort.

Das bekannte Kriechertum-Versagen von RLHF-Modellen — bei dem das Modell Nutzern sagt, was sie hören wollen, anstatt was wahr ist — ist teilweise ein Ergebnis der Evaluatorenpräferenz für gefällige Antworten.

Skalierbare Aufsicht

Für komplexe Aufgaben können Menschen nicht zuverlässig beurteilen, welche KI-Antwort korrekt ist. Ein Annotator, der zwei lange mathematische Beweise oder zwei Code-Implementierungen vergleicht, wählt möglicherweise einfach den lesbareren aus, unabhängig von der Korrektheit. **Skalierbare Aufsicht** ist das offene Forschungsproblem der Entwicklung von Evaluierungsverfahren, die zuverlässig bleiben, wenn die Aufgabenkomplexität wächst {cite}`bowman2022measuring`.

Constitutional AI (RLAIF)

Constitutional AI {cite}`bai2022constitutional`, entwickelt bei Anthropic, reduziert die Abhängigkeit von menschlichen Annotierenden, indem die KI selbst Präferenz-Labels generiert, geleitet von einer Menge von Prinzipien (einer „Verfassung"). Der Prozess:

1. Antworten auf potenziell schädliche Prompts generieren
2. Eine KI-Kritik verwenden, um Antworten gegen konstitutionelle Prinzipien zu bewerten
3. Antworten anhand von KI-Feedback überarbeiten (RLAIF — RL from AI Feedback)
4. Ein Belohnungsmodell auf KI-generierten Präferenzen trainieren
5. Mit RLHF unter Verwendung dieses Belohnungsmodells feinabstimmen

RLAIF kann Präferenzdaten in weit größerem Umfang als menschliche Annotation generieren und ermöglicht eine feingranulare Kontrolle über die im Belohnungsmodell kodierten Werte.

> **See also:** Das originale InstructGPT-Papier {cite}`ouyang2022training` beschreibt die erste großskalige Anwendung von RLHF auf LLMs. Die grundlegende RLHF-Arbeit für tiefes RL ist {cite}`christiano2017deep`. PPO wird in {cite}`schulman2017proximal` beschrieben. Constitutional AI stammt aus {cite}`bai2022constitutional`.

Lernen aus Demonstrationen

Wenn eine Aufgabe schwer zu spezifizieren, aber leicht zu demonstrieren ist, kann es effizienter sein, durch Beispiele zu lehren als durch Regeln zu definieren. Ein menschlicher Experte zeigt einem Roboterarm, wie er ein Objekt greift; die Interaktion eines Programmierers mit seiner IDE liefert eine Folge korrekter Bearbeitungen; ein Schachgroßmeister spielt eine Partie. **Lernen aus Demonstrationen** extrahiert eine Policy aus solchen Verhaltensdaten, ohne die Notwendigkeit handkodierter Belohnungsfunktionen oder expliziter Aufgabenspezifikationen.

Verhaltensklonen

Der einfachste Ansatz ist **Verhaltensklonen (Behavioral Cloning, BC)**: Die Demonstration als überwachte Daten behandeln und eine Abbildung von Zuständen auf Aktionen erlernen.

Gegeben einen Datensatz von Zustands-Aktions-Paaren $\mathcal{D} = \{(s_i, a_i)\}$ eines Experten, passen wir eine Policy $\pi_\theta(a \mid s)$ an, indem wir die negative Log-Likelihood minimieren:

$$\mathcal{L}_{\text{BC}}(\theta) = -\frac{1}{|\mathcal{D}|} \sum_{(s,a)\in\mathcal{D}} \log \pi_\theta(a \mid s)$$

Dies ist präzise Standard-Supervised-Learning auf sequenziellen Daten angewendet.

Das Kovariaten-Shift-Problem

BC hat eine fundamentale Schwäche: **Verteilungsverschiebung** zwischen Training und Einsatz. Die Demonstrationen des Experten decken die vom Experten besuchten Zustände ab. Aber während des Einsatzes kann die erlernte Policy leicht andere Entscheidungen treffen, was sie zu Zuständen führt, die der Experte nie besucht hat — Zustände, für die die Policy keine Aufsicht hat und schlecht versagen kann.

Entscheidend ist, dass sich Fehler **zusammensetzen**: Eine kleine Abweichung führt zu einem unbekannten Zustand, wo eine leicht falsche Aktion zu einem noch unbekannteren Zustand führt, und so weiter. Die Leistung verschlechtert sich als $O(T^2\epsilon)$, wobei T die Episodenlänge und ϵ die Fehlerrate bei jedem Schritt ist — viel schlechter als die $O(T\epsilon)$-Verschlechterung einer Orakel-Policy {cite}`ross2010efficient`.

> **Beispiel: Autonomes Fahren** :class: note
>
> Ein Verhaltenskloning-Modell für das Spurhalten, das auf menschlichen Fahrdaten trainiert wurde, funktioniert gut auf geraden Straßen (Zustände nahe der Trainingsverteilung). Aber in dem Moment, in dem es leicht abdriftet — ein

Zustand, in dem kein menschlicher Fahrer wäre, weil er bereits korrigiert hätte — hat es keine Daten, die es führen, und kann von der Straße abkommen.

```
Algorithmus DAgger:
  Initialisierung: D <- {} (leerer Datensatz)
  Anfangs-Policy pi_1 auf M Experten-Demonstrationen trainieren

  für Iteration i = 1, 2, ..., N:
    1. pi_i in der Umgebung ausführen, um Zustände {s_1, ..., s_t} zu sammeln
    2. Experten für Aktionen bei jedem besuchten Zustand befragen: a_t = pi*(s_t)
    3. Aggregieren: D <- D u {(s_1, a_1), ..., (s_t, a_t)}
    4. pi_{i+1} durch Supervised Learning auf D trainieren
```

DAgger erreicht $O(T\epsilon)$-Bedauern — dasselbe wie eine Orakel-Policy — weil die Trainingsverteilung konvergiert, um der Einsatzverteilung zu entsprechen.

Die Schlüsselanforderung ist, dass der Experte für jeden Zustand abgefragt werden kann, einschließlich Zustände, die der Experte natürlicherweise nie besuchen würde. Dies ist in der Simulation machbar (den Experten bitten, den Roboter aus einer ungewöhnlichen Konfiguration zu korrigieren), kann aber in realen physischen Systemen herausfordernd oder unsicher sein.

Inverses Bestärkendes Lernen

Manchmal wird das Verhalten des Experten besser nicht als Folge von nachzuahmenden Aktionen verstanden, sondern als Ergebnis der Optimierung einer unbekannten Belohnungsfunktion. **Inverses Bestärkendes Lernen (IRL)** {cite}`ng2000algorithms` erholt diese latente Belohnungsfunktion aus Demonstrationen.

Gegeben Demonstrationen $\tau = \{(s_1, a_1), \dots, (s_T, a_T)\}$ findet IRL eine Belohnungsfunktion $R(s, a)$, sodass die Policy des Experten unter R optimal ist.

Der Reiz von IRL gegenüber BC: Wenn wir die wahre Belohnungsfunktion erholen, können wir sie in neuen Umgebungen, mit unterschiedlicher Dynamik oder mit verbesserten Planern neu optimieren — weit über die demonstrierten Szenarien hinaus generalisierend.

Maximum-Entropie-IRL

MaxEnt IRL {cite}`ziebart2008maximum` löst das IRL-Ambiguitätsproblem (es gibt viele Belohnungsfunktionen, die mit einer Menge von Demonstrationen konsistent sind), indem die Belohnungsfunktion gewählt wird, die zwar mit dem demonstrierten Verhalten konsistent ist, aber zu einer Verteilung über Trajektorien mit *maximaler Entropie* führt. Trajektorien sind verteilt als:

$$P(\tau \mid R) \propto \exp\left(\sum_t R(s_t, a_t)\right)$$

Das Lernziel stimmt die beobachteten Merkmals-Erwartungen des Experten $\mu_E = \mathbb{E}_{\tau\sim\pi^*}[\phi(\tau)]$ mit den Merkmals-Erwartungen des Modells $\mu_\theta = \mathbb{E}_{\tau\sim\pi_\theta}[\phi(\tau)]$ ab.

GAIL: Generative Adversarial Imitation Learning

GAIL {cite}`ho2016generative` umgeht das Erlernen der Belohnungsfunktion vollständig, indem es eine GAN-ähnliche Formulierung verwendet, um die Zustands-Aktions-Verteilung des Experten direkt abzugleichen.

Ein Diskriminator D_ψ wird trainiert, um Experten-Zustands-Aktions-Paare $(s, a) \sim \pi^*$ von Policy-Zustands-Aktions-Paaren $(s, a) \sim \pi_\theta$ zu unterscheiden:

$$\mathcal{L}_D = -\mathbb{E}_{\pi^*}[\log D_\psi(s, a)] - \mathbb{E}_{\pi_\theta}[\log(1 - D_\psi(s, a))]$$

Der Generator (die Policy π_θ) wird trainiert, den Diskriminator zu täuschen — d. h. Zustands-Aktions-Paare zu produzieren, die wie die des Experten aussehen. Die Ausgabe des Diskriminators $\log D_\psi(s, a)$ dient als Belohnungssignal für die Policy.

GAIL erreicht Expert-Niveau auf Benchmarks für kontinuierliche Steuerung mit weit weniger Demonstrationen als BC und generalisiert besser als MaxEnt IRL in komplexen Umgebungen.

Verhaltensklonen in NLP: Ein praktisches Beispiel

```python
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

torch.manual_seed(42)

# ------------------------------------------------
# Toy NLP task: rewriting sentences to be more formal
# We simulate this as a simple sequence transformation
# In practice: fine-tuning a seq2seq model on expert rewrites
# ------------------------------------------------

class SimpleSeq2Seq(nn.Module):
    def __init__(self, vocab_size=100, embed_dim=32, hidden_dim=64):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.encoder = nn.GRU(embed_dim, hidden_dim, batch_first=True)
        self.decoder = nn.GRU(embed_dim, hidden_dim, batch_first=True)
        self.proj = nn.Linear(hidden_dim, vocab_size)
        self.hidden_dim = hidden_dim

    def forward(self, src, tgt):
        src_emb = self.embed(src)
        _, hidden = self.encoder(src_emb)
        tgt_emb = self.embed(tgt)
        out, _ = self.decoder(tgt_emb, hidden)
        return self.proj(out)

# Generate synthetic demonstration data
```

```
VOCAB = 100
rng = np.random.default_rng(42)
N, SEQ_LEN = 1000, 12

src_seqs = torch.tensor(rng.integers(1, VOCAB, (N, SEQ_LEN)), dtype=torch.long)
# "Expert" transformation: shift tokens by 1 (toy formalization)
tgt_seqs = torch.clamp(src_seqs + 1, 1, VOCAB - 1)
tgt_in  = torch.cat([torch.ones(N, 1, dtype=torch.long), tgt_seqs[:, :-1]], dim=1)

dataset = TensorDataset(src_seqs, tgt_in, tgt_seqs)
loader = DataLoader(dataset, batch_size=64, shuffle=True)

model = SimpleSeq2Seq(vocab_size=VOCAB)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.CrossEntropyLoss(ignore_index=0)

# Behavioral cloning training
train_losses = []
for epoch in range(20):
    epoch_loss = 0
    for src, tgt_i, tgt_o in loader:
        logits = model(src, tgt_i)
        loss = criterion(logits.reshape(-1, VOCAB), tgt_o.reshape(-1))
        optimizer.zero_grad(); loss.backward(); optimizer.step()
        epoch_loss += loss.item()
    train_losses.append(epoch_loss / len(loader))

print(f"Initial loss: {train_losses[0]:.3f}")
print(f"Final loss:   {train_losses[-1]:.3f}")

# Evaluate: check token accuracy on held-out examples
model.eval()
with torch.no_grad():
    src_test = src_seqs[-100:]
    tgt_test_in = tgt_in[-100:]
    tgt_test_out = tgt_seqs[-100:]
    logits = model(src_test, tgt_test_in)
    preds = logits.argmax(dim=-1)
    acc = (preds == tgt_test_out).float().mean().item()
    print(f"Token accuracy on held-out set: {acc:.3f}")
```

Vergleich der Imitationslernmethoden

Methode	Belohnung erforderlich?	Experte online abgefragt?	Generalisiert zu neuer Dynamik?	Komplexität
Verhaltensklonen	Nein	Nein	Schlecht (Verteilungsverschiebung)	Niedrig
DAgger	Nein	Ja	Mäßig	Mittel

Methode	Belohnung erforderlich?	Experte online abgefragt?	Generalisiert zu neuer Dynamik?	Komplexität
MaxEnt IRL	Erholt sie	Nein	Gut	Hoch
GAIL	Nein	Nein	Gut	Hoch

Anwendungen

Robotik. Roboter lehren, Objekte zu manipulieren, Umgebungen zu navigieren oder Haushaltsaufgaben durchzuführen. Physische Demonstrationen werden durch Teleoperation oder kinästhetisches Lehren gesammelt.

Autonomes Fahren. Frühe selbstfahrende Systeme wie ALVINN {cite}`pomerleau1989alvinn` und NVIDIAs DAVE basierten stark auf Verhaltensklonen aus menschlichen Fahrdaten.

Spiel-KI. Imitationslernen aus menschlichem Spielverhalten startet Agenten, bevor RL-Feinabstimmung erfolgt. AlphaStar trainierte auf menschlichen Replays vor RL; dieser Ansatz ist üblich, wenn Demonstrationen auf menschlichem Niveau verfügbar sind.

Code-Generierung. Sprachmodell-Feinabstimmung auf qualitativ hochwertigen Code-Demonstrationen (GitHub Copilot, Codex) ist eine Form des Verhaltensklonens.

Klinische Entscheidungsunterstützung. Lernen aus Entscheidungssequenzen erfahrener Ärzte für komplexe Protokolle.

> **See also:** Die grundlegende BC/DAgger-Analyse findet sich in {cite}`ross2011reduction`. MaxEnt IRL stammt aus {cite}`ziebart2008maximum`. GAIL stammt aus {cite}`ho2016generative`. Für eine umfassende Übersicht des Imitationslernens, siehe {cite}`osa2018algorithmic`.

Lernen aus Vergleichen und Rankings

Menschen um eine absolute Qualitätsbewertung einer Ausgabe zu bitten ist schwierig. Was ist die numerische Qualität dieses Aufsatzes auf einer Skala von 1 bis 10? Die Frage ist schlecht gestellt: Menschen fehlt eine stabile interne Skala, und ihre Bewertungen werden stark durch Ankerung, Kontext und Ermüdung beeinflusst.

Menschen zu bitten, zwei Ausgaben zu *vergleichen*, ist viel einfacher: Welcher Aufsatz ist besser, A oder B? Vergleichsurteile sind konsistenter, zuverlässiger und sprechen menschliche Präferenzen direkter an als absolute Bewertungen. Dieses Kapitel behandelt die mathematischen Grundlagen und praktischen Anwendungen des Lernens aus Vergleichen und Rankings.

Warum Vergleiche besser sind als Bewertungen

Psychologische Fundierung

Die Überlegenheit vergleichender Urteile hat eine lange Geschichte in der Psychometrie. Thurstons Gesetz des vergleichenden Urteils {cite}`thurstone1927law` (1927) zeigte, dass selbst wenn Menschen inkonsistente absolute Urteile haben, ihre relativen Urteile einem konsistenten probabilistischen Gesetz folgen. Vergleiche nutzen die Tatsache, dass Menschen viel besser in *relativer* Wahrnehmung als in absoluter Kalibrierung sind.

Statistische Effizienz

Jeder paarweise Vergleich liefert Information über die *relativen* Positionen zweier Elemente auf der Qualitätsskala. Bei K Elementen können $K - 1$ Vergleiche alle Elemente ordnen; nur $O(\log K)$ Vergleiche werden benötigt, um das beste Element zu finden. Absolute Bewertungen erfordern typischerweise mehr Urteile, um dieselbe Präzision zu erreichen.

Skalierbarkeit

Für generative Modelle ist die Anzahl unterschiedlicher Ausgaben effektiv unbegrenzt. Eine Ausgabe absolut zu bewerten erfordert die Festlegung einer gemeinsamen Skala über alle Ausgaben; Ausgaben zu vergleichen erfordert nur lokale, relative Urteile, die natürlich aneinander kalibriert sind.

Das Bradley-Terry-Modell

Das dominante probabilistische Modell für paarweise Vergleiche ist das **Bradley-Terry-Modell (BT)** {cite}`bradley1952rank`. Jedes Element i hat einen latenten Qualitätsscore $\alpha_i \in \mathbb{R}$. Die Wahrscheinlichkeit, dass Element i in einem direkten Vergleich gegenüber Element j bevorzugt wird, ist:

$$P(i \succ j) = \frac{e^{\alpha_i}}{e^{\alpha_i} + e^{\alpha_j}} = \sigma(\alpha_i - \alpha_j)$$

wobei σ die logistische Sigmoidfunktion ist. Dies entspricht der Annahme, dass die wahrgenommene Qualität von Element i gleich $\alpha_i + \epsilon$ ist, wobei ϵ ein standardlogistischer Rauschterm ist.

Identifizierbarkeit

Das BT-Modell ist bis auf eine Translation identifizierbar: Wenn α eine Lösung ist, ist es auch $\alpha + c$ für eine beliebige Konstante c. Eine Standardkonvention ist, einen Score festzusetzen (z. B. $\alpha_0 = 0$) oder $\sum_i \alpha_i = 0$ zu verlangen. Die Scores sind nur identifizierbar, wenn der **Vergleichsgraph** (Knoten = Elemente, Kanten = beobachtete Paare) **zusammenhängend** ist — wenn der Graph unverbundene Komponenten hat, sind die relativen Scores über Komponenten hinweg undefiniert.

Parameterschätzung

Gegeben einen Datensatz paarweiser Vergleiche $\mathcal{D} = \{(i, j, y_{ij})\}$, wobei $y_{ij} = 1$ wenn i gegenüber j bevorzugt wurde, ist die Log-Likelihood:

$$\mathcal{L}(\alpha) = \sum_{(i,j,y_{ij}) \in \mathcal{D}} \left[y_{ij} \log \sigma(\alpha_i - \alpha_j) + (1 - y_{ij}) \log \sigma(\alpha_j - \alpha_i) \right]$$

Dies ist eine konkave Funktion von α und kann durch Gradientenaufstieg oder Newtons Methode maximiert werden.

```python
import numpy as np
from scipy.optimize import minimize
from scipy.special import expit  # sigmoid

rng = np.random.default_rng(42)

# ------------------------------------------------
# Simulate Bradley-Terry: 8 items with true quality scores
# Generate pairwise comparisons and recover the scores
# ------------------------------------------------

N_ITEMS = 8
true_alpha = rng.normal(0, 1, N_ITEMS)
print(f"True quality ranking: {np.argsort(true_alpha)[::-1]}")

# Generate comparisons: every pair compared 5 times
comparisons = []
for i in range(N_ITEMS):
```

```
    for j in range(i + 1, N_ITEMS):
        for _ in range(5):
            p_i_wins = expit(true_alpha[i] - true_alpha[j])
            winner = i if rng.random() < p_i_wins else j
            loser  = j if winner == i else i
            comparisons.append((winner, loser))

print(f"Total comparisons: {len(comparisons)}")

def neg_log_likelihood(alpha, comparisons):
    """Bradley-Terry negative log-likelihood."""
    alpha = np.array(alpha)
    loss = 0.0
    for winner, loser in comparisons:
        log_prob = np.log(expit(alpha[winner] - alpha[loser]) + 1e-10)
        loss -= log_prob
    return loss

def neg_grad(alpha, comparisons):
    alpha = np.array(alpha)
    grad = np.zeros(len(alpha))
    for winner, loser in comparisons:
        p = expit(alpha[winner] - alpha[loser])
        grad[winner] -= (1 - p)
        grad[loser]  -= (-p)
    return grad

# Fix alpha[0] = 0 to resolve scale ambiguity
result = minimize(
    lambda a: neg_log_likelihood(np.concatenate([[0.0], a]), comparisons),
    x0=np.zeros(N_ITEMS - 1),
    jac=lambda a: neg_grad(np.concatenate([[0.0], a]), comparisons)[1:],
    method='L-BFGS-B'
)
est_alpha = np.concatenate([[0.0], result.x])

# Compare true vs estimated ranking
true_rank = np.argsort(true_alpha)[::-1]
est_rank  = np.argsort(est_alpha)[::-1]

print(f"\nTrue ranking (item indices): {true_rank}")
print(f"Estimated ranking:           {est_rank}")
rank_corr = np.corrcoef(true_alpha, est_alpha)[0, 1]
print(f"Correlation with true scores: {rank_corr:.4f}")
```

Thurstons Modell

Thurstons Modell {cite}`thurstone1927law` ist eng mit Bradley-Terry verwandt, verwendet aber ein Gauß'sches statt logistisches Rauschen:

$$P(i \succ j) = \Phi\left(\frac{\alpha_i - \alpha_j}{\sqrt{2}\sigma}\right)$$

wobei Φ die Standard-Normal-CDF ist. Für $\sigma = 1/\sqrt{2}$ wird dies äquivalent zu BT mit einem geringfügigen Skalierungsunterschied. In der Praxis liefern die beiden Modelle nahezu identische Ergebnisse.

Rang-Aggregation

Wenn jeder Annotator ein vollständiges Ranking von K Elementen liefert (statt paarweiser Vergleiche), besteht das Problem in der **Rang-Aggregation**: Mehrere gerankte Listen zu einem Konsensus-Ranking kombinieren.

Borda-Zählung: Jedes Element erhält einen Score gleich der Anzahl der Elemente, die im Ranking jedes Annotators unter ihm eingestuft sind. Scores werden über Annotatoren summiert. Einfach und robust.

Kemeny-Young: Das Ranking finden, das die Summe paarweiser Uneinigkeiten (den Kendall-Tau-Abstand) mit dem Ranking jedes Annotators minimiert. Dies ist NP-schwer für großes K, aber für kleine Mengen handhabbar.

RankNet / ListNet: Neuronale Ansätze, die eine Bewertungsfunktion aus gerankte Listen erlernen und Generalisierung auf ungesehene Elemente ermöglichen.

Duellier-Banditen

Beim **Online**-Präferenzlernen kommen Elemente in einem Strom an, und wir müssen entscheiden, welche Paare zu vergleichen sind, wobei Exploration (Lernen über unbekannte Elemente) und Exploitation (Präsentation hochwertiger Elemente) ausgewogen werden. Dies ist das **Duellier-Bandit**-Problem {cite}`yue2009interactively`.

Schlüsselalgorithmen: - **Doubler:** Pflegt ein Champion-Element; fordert es mit zufälligen Konkurrenten heraus - **RUCB (Relative Upper Confidence Bound):** Berechnet UCB-ähnliche Konfidenzintervalle für die Wahrscheinlichkeit jedes Elements, jedes andere zu schlagen - **MergeRank:** Kombiniert turnierartigen Vergleich mit UCB

Duellier-Banditen werden in Online-Empfehlungssystemen (welchen Artikel als nächstes anzeigen, bei implizitem Feedback) und bei der interaktiven Präferenzerkennung für die RLHF-Datensammlung verwendet.

Präferenzlernen für Sprachmodelle

Im Kontext von RLHF (Kapitel 6) wird das Bradley-Terry-Modell zum Training des Belohnungsmodells verwendet. Eine wichtige Variante ist **Direct Preference Optimization (DPO)** {cite}`rafailov2023direct`, die zeigt, dass das RLHF-Ziel direkt aus Präferenzdaten optimiert werden kann, ohne ein separates Belohnungsmodell zu trainieren:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta\log\frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right]$$

DPO ist einfacher als volles RLHF (keine PPO-Trainingsschleife, kein Belohnungsmodell), während es auf vielen Benchmarks vergleichbare oder bessere Ergebnisse erzielt {cite}`rafailov2023direct`. Es hat sich als weit verbreitete Alternative zu PPO-basiertem RLHF für Instruktionsfolge-Feinabstimmung etabliert, obwohl beide Ansätze aktiv genutzt werden und ihre relativen Stärken aufgabenabhängig sind.

Hochwertige Präferenzdaten sammeln

Die Qualität der Präferenzdaten bestimmt die Qualität des Belohnungsmodells. Wichtige Überlegungen:

Prompt-Vielfalt. Präferenzdaten sollten die vollständige Verteilung der Prompts abdecken, denen das Modell beim Einsatz begegnen wird. Lücken in der Abdeckung führen zu unzuverlässigem Belohnungsmodellverhalten in diesen Bereichen.

Antwort-Vielfalt. Der Vergleich zweier sehr ähnlicher Antworten liefert wenig Information. Die verglichenen Antworten sollten sich genug unterscheiden, damit Annotatoren eine klare Präferenz haben.

Annotatorenübereinstimmung. Niedrige Annotatorenübereinstimmung deutet darauf hin, dass die Vergleichskriterien unklar sind. Übereinstimmung messen (Cohens κ) und Richtlinien überarbeiten, wenn sie unter akzeptable Schwellenwerte fällt.

Kalibrierung. Annotatoren sollten verstehen, *warum* eine Antwort besser ist: Nützlichkeit, Genauigkeit, Sicherheit, Stil? Aufgaben, die mehrere Kriterien bündeln, neigen dazu, inkonsistente Präferenzen zu erzeugen. Es ist oft besser, Präferenzen für jedes Kriterium separat zu sammeln.

> **See also:** Bradley-Terry-Modell: {cite}`bradley1952rank`. Thurstone: {cite}`thurstone1927law`. Duellier-Banditen: {cite}`yue2009interactively`. Direct Preference Optimization (DPO): {cite}`rafailov2023direct`.

HITL in der natürlichen Sprachverarbeitung

Natürliche Sprache ist die Domäne, in der HITL ML wohl die größte Wirkung hatte — und wo seine tiefsten konzeptuellen Schwierigkeiten zutage treten. Sprache ist von Natur aus sozial: Ihre Bedeutung wird von menschlichen Gemeinschaften konstruiert, ihre Pragmatik hängt von Kontext und Absicht ab, und ihre Qualität kann nur von menschlichen Lesern beurteilt werden. Aber dies bedeutet auch, dass NLP-Annotation nicht nur ein Prozess der Beobachtungssammlung ist. Es ist ein Prozess der *Konstruktion* von Kategorien.

Das konstitutive Problem bei der NLP-Annotation

In der medizinischen Bildgebung gibt es eine Grundwahrheit: Ein Tumor ist vorhanden oder nicht. Das Label des Radiologen mag unsicher sein, aber es versucht, etwas Reales zu erfassen. NLP-Annotation ist oft grundlegend anders. Wenn ein Annotator einen Tweet als „toxisch" markiert, gibt es kein toxisches Molekül im Tweet, das wir zu erkennen versuchen. **Das Label konstituiert die Kategorie.**

Dies hat tiefgreifende Konsequenzen, die häufig unterschätzt werden:

Die Annotationsbelegschaft definiert das Phänomen. Ein Label-Schema für „anstößige Sprache" kodiert die Sensibilitäten derjenigen, die es entworfen und angewendet haben. Ein Team englischsprachiger Annotatoren mit einheitlichem demografischem Hintergrund, das unter Richtlinien einer Unternehmensrichtlinien-Abteilung arbeitet, produziert einen Datensatz, der das Verständnis dieses Teams von Anstößigkeit widerspiegelt — keinen universellen menschlichen Standard. Auf solchen Daten trainierte Modelle werden dieselben impliziten Grenzen aufweisen.

Richtlinien sind Theorie, ob sie es eingestehen oder nicht. Jedes Annotationsschema macht ontologische Behauptungen. Zu entscheiden, dass „Ironie" und „Sarkasmus" dieselbe Klasse sind, ist eine theoretische Behauptung, keine neutrale Bequemlichkeit. Zu entscheiden, „Wut" als eine einzelne Klasse zu kennzeichnen, anstatt „rechtschaffene Wut" von „feindseliger Wut" zu unterscheiden, kollidiert eine Unterscheidung, die für die nachgelagerte Aufgabe wichtig sein kann. Diese Entscheidungen werden unter Produktionsdruck getroffen und selten überprüft.

Label-Instabilität über die Zeit. Soziale Sprache entwickelt sich. Ein auf 2018-Daten trainiertes Toxizitätsmodell wird Inhalte aus 2024 falsch klassifizieren, nicht weil es statistisch untertrainiert ist, sondern weil sich die soziale Bedeutung bestimmter Begriffe geändert hat. NLP-Annotation ist kein Sampling aus einer statischen Verteilung — es ist Sampling aus einem sich bewegenden Ziel, zu dessen Konstruktion der Annotationsakt teilweise beiträgt.

:::{admonition} Das Annotationsartefakt-Problem :class: important

Geva et al. {cite}`geva2019annotator` demonstrierten, dass NLI-Datensätze (Natural Language Inference) systematische Artefakte enthalten, die durch den Annotationsprozess selbst eingeführt wurden. Annotatoren, die aufgefordert werden, „Schlussfolgerungs"-Hypothesen für eine gegebene Prämisse zu schreiben, neigen dazu, bestimmte syntaktische Muster zu verwenden; Annotatoren, die „Widerspruchs"-Hypothesen schreiben, verwenden andere. Modelle lernen, basierend auf diesen Artefakten zu klassifizieren, anstatt die beabsichtigte semantische Beziehung — sie lösen die Annotationsaufgabe, nicht die zugrunde liegende NLP-Aufgabe.

Dies ist keine Nachlässigkeit. Es ist eine inhärente Konsequenz davon, dass Menschen Beispiele konstruieren, um ein Label zu passen. Der HITL-Prozess fügt ein systematisches Signal ein, das nie in den Daten sein sollte. :::

Textklassifikations-Annotation

Die einfachste NLP-Annotationsaufgabe ist die Zuweisung einer Kategorie zu einem Textdokument. Stimmungsanalyse, Themenklassifikation, Absichtserkennung und Spam-Filterung sind alles Klassifikationsaufgaben.

Herausforderungen spezifisch für Textklassifikation:

Subjektivität. Kategorien wie „toxisch" oder „anstößig" sind von Natur aus subjektiv und variieren über kulturelle Kontexte, Annotatorenhintergründe und Plattformkontext. Zwischen Annotatoren unterscheiden sich Wahrnehmungen von Anstößigkeit erheblich nach Demografie — eine Tatsache mit direkten Implikationen für Fairness {cite}`blodgett2020language`.

Label-Ambiguität. Viele Dokumente gehören zu mehreren Kategorien oder liegen auf einer Grenze. Eine Rezension, die zu 60 % positiv und zu 40 % negativ ist, ist genuinen ambig, nicht falsch annotiert. Das Erzwingen eines einzigen Labels verwirft echte Information (siehe Kapitel 13 über Soft-Labels und Annotatorenuneinigkeit).

Label-Granularität. Ein 2-Klassen-Stimmungsmodell kann für die Social-Media-Überwachung ausreichen; eine 7-Punkte-Skala kann für die Produktfeedback-Analyse benötigt werden. Die richtige Granularität hängt von der nachgelagerten Aufgabe ab, wird aber in der Regel vor der Annotation festgelegt — eine folgenreiche Designentscheidung, die mit unzureichenden Daten getroffen wird.

Erkennung benannter Entitäten

NER-Annotation erfordert das Identifizieren von Textspannen und die Zuweisung eines Entitätstyps (PERSON, ORGANISATION, ORT usw.). Die Annotation ist aus mehreren Gründen komplexer als Dokumentklassifikation:

Spannen-Grenzen sind ambig. In „Apple CEO Tim Cook kündigte an…" umfasst die PERSON-Entität „Tim Cook" oder „Apple CEO Tim Cook"? Richtlinien müssen diese Fälle explizit behandeln, und die Annotatorenübereinstimmung bei Spannen ist konsistent niedriger als bei Typen.

Typzuweisung erfordert Weltwissen. „Apple" ist in einem Kontext ORG, in einem anderen PRODUKT, und in „Apfelkuchen" keines von beidem. Annotatoren benötigen ausreichendes Domänenwissen für korrekte Typzuweisungen.

Verschachtelte Entitäten. „Die Universität Bonn" enthält ORGANISATION innerhalb von ORT. Standard-BIO-Tagging kann keine verschachtelten Entitäten darstellen; komplexere Schemata (z. B. BIOES oder graphbasierte Formate) sind erforderlich.

Annotationseffizienz. Vorannotation mit einem Basis-NER-Modell beschleunigt die Annotation erheblich, indem Annotatoren Vorhersagen korrigieren anstatt von Grund auf zu annotieren. In einer Studie über klinisches NER wurden Durchsatzsteigerungen von 30–60 % beobachtet {cite}`lingren2014evaluating`; das Ausmaß solcher Gewinne hängt stark von der Qualität des Basismodells und der Domäne ab.

Relationsextraktion und semantische Annotation

Über die Entitätserkennung hinaus erfordert die Relationsextraktion die Annotation von *Beziehungen* zwischen Entitäten:

- Annotatoren müssen sowohl Entitäten als auch das sie verbindende Prädikat verstehen
- Relationstypen haben komplexe semantische Unterscheidungen (ARBEITET_BEI vs. ANGESTELLT_VON vs. GRÜNDETE)
- Negative Beispiele (Entitätspaare ohne Relation) sind viel häufiger als positive

Annotatorenübereinstimmung bei der Relationsextraktion ist tendenziell niedriger als bei Klassifikationsaufgaben. Für gut definierte Schemata werden κ-Werte im Bereich von 0,65–0,80 häufig berichtet {cite}`pustejovsky2012natural`; für komplexe semantische Relationen (Ereigniskausalität, temporale Relationen) kann die Übereinstimmung je nach Schema-Design und Annotatorentraining erheblich niedriger ausfallen.

Maschinelle Übersetzungspost-Editierung (MTPE)

Maschinelle Übersetzungspost-Editierung ist eine ausgereifte Form von HITL-NLP. Ein menschlicher Übersetzer korrigiert die Ausgabe eines MT-Systems, anstatt von Grund auf zu übersetzen:

Leichte Post-Editierung (LPE): Nur kritische Fehler werden korrigiert. Geeignet, wenn die Übersetzungsqualitätsanforderungen moderat sind.

Vollständige Post-Editierung (FPE): Die Ausgabe wird auf Publikationsqualität korrigiert. Die bearbeitete Ausgabe wird typischerweise zu Trainingsdaten für weitere MT-Verbesserung — ein echter Human-in-the-Loop-Verfeinerungszyklus.

HTER (Human-targeted Translation Edit Rate): Eine Metrik, die den Editierabstand zwischen der MT-Ausgabe und der post-editierten Übersetzung misst, normalisiert nach Satzlänge {cite}`graham2015accurate`. Als grobe Faustformel gilt HTER unter etwa 0,3 oft als gute MT-Ausgabe; über 0,5 kann Übersetzung von Grund auf schneller sein — obwohl diese Schwellenwerte nach Domäne und Sprachpaar variieren.

Konversations-KI und Dialog-Annotation

Die Annotation von Dialog führt zeitliche Struktur ein:

- **Turn-Ebenen-Annotation:** Jeden Turn kennzeichnen (Absicht, Stimmung, Qualität)
- **Dialog-Ebenen-Annotation:** Gesamtkohärenz, Aufgabenerfolg, Nutzerzufriedenheit bewerten
- **Interaktionstrace-Annotation:** Spezifische Fehlerpunkte in einer Konversation identifizieren

HITL ist bei Dialog besonders wichtig, weil Modellversagen oft subtil und kumulativ ist: Ein sachlicher Fehler in Turn 3 mag erst in Turn 7 offensichtlich werden. Menschliche Annotatoren, die Konversationen verfolgen, können diese Langstrecken-Fehlermuster identifizieren, die automatisierte Metriken vollständig übersehen.

Programmatische Kennzeichnung und schwache Überwachung

Wenn annotierte Daten knapp sind, ermöglicht **schwache Überwachung** die Verwendung mehrerer verrauschter, überlappender Kennzeichnungsfunktionen, um im großen Maßstab probabilistische Labels zu generieren. **Snorkel** {cite}`ratner2017snorkel` ist das kanonische Rahmenwerk:

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score

rng = np.random.default_rng(42)

# ---------------------------------------------------------
# Toy weak supervision: sentiment classification
# 5 labeling functions (LFs) with different coverage/accuracy
# Label: +1 (positive), -1 (negative), 0 (abstain)
# ---------------------------------------------------------

N = 1000
true_labels = rng.choice([-1, 1], size=N)
X_features = np.column_stack([
    true_labels * 0.8 + rng.normal(0, 0.5, N),
    rng.normal(0, 1, N),
    rng.normal(0, 1, N),
])

def make_lf(accuracy, coverage, seed):
    rng_lf = np.random.default_rng(seed)
    votes = np.zeros(N, dtype=int)
    active = rng_lf.random(N) < coverage
    correct = rng_lf.random(N) < accuracy
    votes[active & correct]  = true_labels[active & correct]
    votes[active & ~correct] = -true_labels[active & ~correct]
    return votes
```

```
LFs = np.column_stack([
    make_lf(accuracy=0.85, coverage=0.60, seed=1),
    make_lf(accuracy=0.78, coverage=0.45, seed=2),
    make_lf(accuracy=0.70, coverage=0.80, seed=3),
    make_lf(accuracy=0.90, coverage=0.30, seed=4),
    make_lf(accuracy=0.60, coverage=0.90, seed=5),
])

def majority_vote(LF_matrix):
    labels = []
    for i in range(LF_matrix.shape[0]):
        votes = LF_matrix[i][LF_matrix[i] != 0]
        labels.append(0 if len(votes) == 0 else int(np.sign(votes.sum())))
    return np.array(labels)

mv_labels = majority_vote(LFs)
covered = mv_labels != 0

print(f"Coverage:                    {covered.mean():.1%}")
print(f"MV accuracy (covered):       {(mv_labels[covered] == true_labels[covered]).mean():.3f

X_train, y_train = X_features[covered], mv_labels[covered]
X_test,  y_test  = X_features[~covered], true_labels[~covered]

if len(X_train) > 10 and len(X_test) > 10:
    clf = LogisticRegression().fit(X_train, y_train)
    preds = clf.predict(X_test)
    print(f"F1 on uncovered test set:    {f1_score(y_test, preds):.3f}")
```

RLHF für Sprachmodelle: Die Annotationsrealität

Kapitel 6 hat RLHF technisch behandelt. Aus einer NLP-Perspektive ist die Annotationsaufgabe schwieriger als sie von außen erscheint.

Was Annotatoren tatsächlich gefragt werden — zwei Modellausgaben zu vergleichen und anzugeben, welche „besser" ist — klingt einfach. In der Praxis ist „besser" ein unterspezifiziertes Konstrukt, das Annotatoren mit persönlichen Heuristiken auflösen. Manche gewichten Flüssigkeit stark; andere Sachgenauigkeit; andere bestrafen Ausführlichkeit. Ohne strenge Richtlinien spiegelt der resultierende Präferenzdatensatz nicht abstrakte menschliche Werte wider, sondern die spezifischen Auflösungsstrategien der eingesetzten Annotationsbelegschaft.

Die wichtigsten Annotationsdimensionen sind:

- *Nützlichkeit:* Beantwortet die Antwort die Frage gut? Ist sie informativ, klar und angemessen detailliert?
- *Faktentreue:* Ist die Antwort sachlich korrekt? Dies erfordert, dass Evaluierende Domänenwissen haben — eine Anforderung, die im großen Maßstab ernsthafte Qualitätsprobleme schafft, da generalistische Annotatoren keine Spezialbehauptungen verifizieren können.
- *Harmlosigkeit:* Vermeidet die Antwort toxische, voreingenommene, schädliche oder

unangemessene Inhalte? Diese Urteile erfordern detaillierte Richtlinien, weil „schädlich" stark vom Kontext abhängt und sich über Kulturen, Zeit und Plattformkontext verschiebt.
- *Kalibrierung:* Drückt die Antwort angemessene Unsicherheit aus? Annotatoren bevorzugen systematisch Antworten, die sicher klingen, was ein Belohnungssignal gegen angemessene epistemische Bescheidenheit erzeugt.

Die Wechselwirkung zwischen Kriterien ist komplex: Eine maximal hilfreiche Antwort auf eine gefährliche Frage kann maximal schädlich sein. Richtlinien müssen spezifizieren, wie konkurrierende Kriterien abgewogen werden — und diese Abwägungen sind effektiv politische Entscheidungen, keine Annotationsentscheidungen. Die Annotationsbelegschaft trifft Politik.

Skalierung konzentriert demografischen Einfluss. RLHF für große Modelle umfasst relativ kleine Annotationsbelegschaften (Hunderte bis niedrige Tausende), die Milliarden von nachgelagerten Entscheidungen treffen. Die demografischen und kulturellen Vorurteile dieser Belegschaft pflanzen sich im großen Maßstab in das Modellverhalten fort — auf eine Weise, die nicht geschehen würde, wenn die Annotation stärker verteilt wäre. Dies ist eines der am wenigsten diskutierten strukturellen Probleme in der aktuellen RLHF-Pipeline.

Die Annotation-Modell-Rückkopplungsschleife

In NLP mehr als in jeder anderen Domäne werden Annotations- und Modellentwicklungsprozesse über die Zeit verflochten:

1. Annotatoren werden mit den bestehenden Ausgaben eines Modells als Referenz kalibriert (oft implizit).
2. Das Belohnungsmodell lernt, was Annotationen tendieren zu bevorzugen.
3. Der Generator wird feinabgestimmt, um Ausgaben zu produzieren, die hohe Belohnung erhalten.
4. Diese Ausgaben beeinflussen, wie „gut" in nachfolgenden Annotationsrunden aussieht.

Diese Rückkopplungsschleife ist nicht von Natur aus pathologisch — sie ermöglicht RLHF zu konvergieren — aber es bedeutet, dass das Verhalten des Modells von einem sich bewegenden Ziel geformt wird, zu dessen Bewegung der Annotationsprozess selbst beiträgt. Empirisch ist es schwierig zu unterscheiden, was das Modell gelernt hat, weil es menschliche Präferenzen widerspiegelt, von dem, was es gelernt hat, weil es gelernt hat, Annotatorenheuristiken zu mustergleichen.

Es gibt keine saubere Lösung. Die beste aktuelle Praxis ist, Drift zu überwachen mittels gehaltener Präferenzurteile, die in regelmäßigen Abständen gesammelt werden, und die Annotationsrichtlinien-Version als experimentelle Variable zu behandeln.

Evaluierung generativer NLP-Modelle

Im Gegensatz zu Klassifikationsmodellen mit einer klaren Genauigkeitsmetrik erfordert die Bewertung der Generierungsqualität menschliches Urteilsvermögen:

Evaluierungsmethode	Beschreibung	Kosten
Direkte Bewertung (DA)	Annotatoren bewerten Qualität auf einer Skala	Mittel
Vergleichendes Urteil	Annotatoren vergleichen zwei Ausgaben	Niedrig
MQM (Multidimensionale Qualitätsmetriken)	Strukturierte Fehlertaxonomie	Hoch
RLHF-Präferenz	Präferenz-Labels für Training verwendet	Mittel
LLM als Richter	LLM bewertet Ausgaben (korreliert mäßig mit Mensch)	Sehr niedrig
BERTScore, BLEU	Automatische Metriken (unvollkommene Korrelation mit menschlichem Urteil)	Sehr niedrig

Automatische Metriken (BLEU für MT, ROUGE für Zusammenfassung) sind billig, aber unvollkommen mit menschlichen Qualitätsurteilen korreliert {cite}`reiter2018structured`. LLM-als-Richter-Ansätze zeigen moderate Übereinstimmung mit menschlichen Annotatoren {cite}`zheng2023judging` und werden zunehmend für schnelle Iteration verwendet, sollten aber keine menschliche Evaluierung für abschließende Bewertungen ersetzen. Für Entscheidungen mit echten Einsätzen bleibt menschliche Beurteilung notwendig.

> **See also:** Snorkel schwache Überwachung: {cite}`ratner2017snorkel`. NLP-Annotationsrichtlinien: {cite}`pustejovsky2012natural`. Annotationsartefakte in NLI: {cite}`geva2019annotator`. Annotatorenverzerrung und NLP-Datensätze: {cite}`blodgett2020language`. Zur Evaluierung generativer Modelle: {cite}`reiter2018structured`.

HITL in der Computer-Vision

Computer-Vision bietet einige der visuell intuitivsten Beispiele für HITL ML. Die ImageNet-Challenge, aufgebaut auf 14 Millionen von Menschen annotierten Bildern, leitete die Deep-Learning-Ära ein. Medizinische Bildannotation durch Radiologen treibt diagnostische KI an. Autonome Fahrzeuge sind auf Millionen von Menschen annotierten Frames angewiesen, um die Welt wahrzunehmen.

Was leicht zu übersehen ist: Dies sind nicht einfach Fälle, in denen Menschen Grundwahrheit liefern. Es sind Fälle, in denen Menschen Datensätze konstruieren, die spezifische wahrnehmungsbezogene, kulturelle und operative Entscheidungen einbetten — Entscheidungen, die erst später sichtbar werden, wenn Modelle auf vorhersehbare Weise versagen.

Wie Annotationsentscheidungen zu Modellverzerrungen werden

Das Standardrahmen behandelt Annotation als Datensammlung: Menschen beobachten die Welt und zeichnen auf, was sie sehen. Das genauere Rahmen ist, dass Annotation *Datensatz-Design* ist: Menschen entscheiden, welche Kategorien zu verwenden sind, wo Grenzen zu ziehen sind, welche Randfälle einzuschließen sind und wie Ambiguität aufzulösen ist — und all diese Entscheidungen formen, was das trainierte Modell wahrnehmen wird.

Der ImageNet-Fall

ImageNet {cite}`russakovsky2015imagenet` ist der folgenreichste Datensatz in der Geschichte der Computer-Vision. Sein Label-Set leitet sich von WordNet-Synsets ab, primär ausgewählt weil sie zahlreich und semantisch distinct sind. Mehrere Konsequenzen dieser Designentscheidung traten Jahre später zutage:

- **Personenkategorien kodierten demografische Assoziationen.** Frühe Versionen der Synset-Labels von ImageNet für Menschen enthielten viele, die heute als abwertend oder voreingenommen angesehen würden, und spiegelten sowohl die historische WordNet-Quelle als auch die impliziten Entscheidungen der Annotationsbelegschaft wider, welche Labels auf welche Bilder anzuwenden sind {cite}`yang2020towards`. Auf Personenbilder angewendete Labels kodierten Rassen-, Geschlechts- und Klassenassoziationen, die sich direkt in Modell-Einbettungen fortpflanzten.

- **Feinkörnige Artenklassifikation, grobe von fast allem anderen.** ImageNet kann 120 Hunderassen unterscheiden, fasst aber enorme Variation bei Werkzeugen, Fahrzeugen, Lebensmitteln und Möbeln zu einzelnen Kategorien zusammen. Dies war eine Konsequenz des Folgens der WordNet-Struktur, keine bewusste Entscheidung darüber, was zählt. Auf ImageNet trainierte Modelle zeigen dieselbe asymmetrische Präzision.

- **Westliche, englischsprachige visuelle Standards.** Die Bilder wurden hauptsächlich von Flickr und Internet-Suchen mit englischsprachigen Abfragen gesammelt. Die resultierende Verteilung ist stark auf die visuelle Umgebung und kulturellen Objekte wohlhabender, englischsprachiger Länder ausgerichtet.

Nichts davon waren Fehler. Es waren Annotationsdesign-Entscheidungen, die schnell im großen Maßstab getroffen wurden, oft von Personen, die nicht antizipierten, wie der Datensatz verwendet werden würde. Die Lektion ist nicht, dass ImageNet anders hätte aufgebaut werden sollen (obwohl dies der Fall hätte sein sollen), sondern dass **Annotationsdesign Modelldesign ist**, und es sollte mit derselben Sorgfalt behandelt werden.

:::{admonition} Annotationsschema ist eine Theorie der Welt :class: note

Jede Label-Taxonomie macht Behauptungen darüber, welche Unterscheidungen wichtig sind. „Auto" von „LKW" zu trennen, während alle Limousinen in eine Klasse zusammengefasst werden, ist eine theoretische Behauptung darüber, welche Unterscheidungen semantisch relevant sind. „Person" als eine einzelne Klasse unabhängig von Körperhaltung, Kleidung oder Aktivität zu annotieren ist eine andere theoretische Behauptung. Auf diesen Schemata trainierte Modelle werden dieselben Unterscheidungen machen und nicht mehr — sie werden nicht über die Kategorien verallgemeinern, die sie zu unterscheiden trainiert wurden. :::

Bildklassifikations-Annotation

Die einfachste CV-Annotationsaufgabe ist die Zuweisung eines oder mehrerer Labels an ein ganzes Bild.

Label-Hierarchie. Das Label „Hund" ist in einer semantischen Hierarchie ein Kind von „Tier". Auf flachen Taxonomien trainierte Modelle verallgemeinern möglicherweise nicht gut über Abstraktionsebenen hinweg. ImageNet verwendet eine Synset-basierte Hierarchie, die Evaluierung auf mehreren Spezifizitätsstufen ermöglicht.

Mehrlabel-Ambiguität. Eine Straßenszene kann gleichzeitig ein Auto, eine Person, ein Fahrrad und eine Ampel enthalten. Zu entscheiden, welche Labels einzuschließen sind, erfordert klare Richtlinien zu Relevanzsschwellen.

Long-Tail-Verteilungen. Natürliche Bilddatensätze folgen einem Potenzgesetz: Wenige Kategorien sind extrem häufig; die meisten sind selten. Aktives Lernen ist besonders wertvoll für Long-Tail-Kategorien, bei denen zufälliges Sampling nur eine Handvoll Beispiele liefert.

Objekterkennung: Begrenzungsrahmen-Annotation

Die Objekterkennung erfordert, dass Annotatoren achsenausgerichtete Begrenzungsrahmen um Instanzen jeder Objektklasse zeichnen. Dies führt geometrische Präzisionsanforderungen und erhebliche Randfälle ein.

Annotationsqualitätsmetriken:

IoU (Intersection over Union) misst die Überlappung zwischen einem annotierten Rahmen und einem Referenzrahmen:

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

IoU $\geq 0,5$ ist der Standardschwellenwert für eine „korrekte" Erkennung in PASCAL VOC; COCO {cite}`lin2014microsoft` verwendet einen Bereich von Schwellenwerten von 0,5 bis 0,95, was erheblich anspruchsvoller und informativer ist.

Annotationsrandfälle, die in Richtlinien gelöst werden müssen: - Verdeckte Objekte: sichtbaren Anteil annotieren oder vollständigen Umfang extrapolieren? - Abgeschnittene Objekte (teilweise außerhalb des Rahmens): einschließen oder ausschließen? - Menschenmassen-Regionen: eine spezielle „Menge"-Annotation verwenden oder individuelle Instanzen annotieren?

Jede dieser Entscheidungen ändert, was „korrekte Erkennung" bedeutet — und ändert daher, was das Modell trainiert wird zu tun.

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as patches

def compute_iou(boxA, boxB):
    """Compute IoU between two boxes [x1, y1, x2, y2]."""
    xA = max(boxA[0], boxB[0])
    yA = max(boxA[1], boxB[1])
    xB = min(boxA[2], boxB[2])
    yB = min(boxA[3], boxB[3])
    inter_area = max(0, xB - xA) * max(0, yB - yA)
    boxA_area = (boxA[2] - boxA[0]) * (boxA[3] - boxA[1])
    boxB_area = (boxB[2] - boxB[0]) * (boxB[3] - boxB[1])
    union_area = boxA_area + boxB_area - inter_area
    return inter_area / (union_area + 1e-6)

ref_box  = [1.0, 1.0, 4.0, 4.0]
ann1_box = [1.1, 0.9, 4.1, 4.2]   # close
ann2_box = [0.5, 0.5, 3.5, 3.8]   # less precise

print(f"IoU (ann1 vs ref):  {compute_iou(ann1_box, ref_box):.3f}")
print(f"IoU (ann2 vs ref):  {compute_iou(ann2_box, ref_box):.3f}")
print(f"IoU (ann1 vs ann2): {compute_iou(ann1_box, ann2_box):.3f}")

fig, ax = plt.subplots(figsize=(5, 5))
ax.set_xlim(0, 5); ax.set_ylim(0, 5); ax.set_aspect('equal')
ax.add_patch(patches.Rectangle(
    (ref_box[0], ref_box[1]), ref_box[2]-ref_box[0], ref_box[3]-ref_box[1],
    linewidth=2.5, edgecolor='#2b3a8f', facecolor='none', label='Reference'))
ax.add_patch(patches.Rectangle(
    (ann1_box[0], ann1_box[1]), ann1_box[2]-ann1_box[0], ann1_box[3]-ann1_box[1],
    linewidth=2, edgecolor='#0d9e8e', facecolor='none', linestyle='--',
    label=f'Ann1 (IoU={compute_iou(ann1_box, ref_box):.2f})'))
ax.add_patch(patches.Rectangle(
    (ann2_box[0], ann2_box[1]), ann2_box[2]-ann2_box[0], ann2_box[3]-ann2_box[1],
    linewidth=2, edgecolor='#e05c5c', facecolor='none', linestyle=':',
```

```
    label=f'Ann2 (IoU={compute_iou(ann2_box, ref_box):.2f})'))
ax.legend(fontsize=10)
ax.set_title("Bounding Box Agreement (IoU)", fontsize=12)
plt.tight_layout()
plt.savefig('bbox_iou.png', dpi=150)
plt.show()
```

Semantische und Instanzsegmentierung

Segmentierungsannotation erfordert die Zuweisung eines Klassen-Labels zu jedem Pixel in einem Bild — unter den teuersten Annotationstypen.

Semantische Segmentierung: Jedes Pixel gehört zu einer semantischen Klasse (Straße, Himmel, Person, Auto). Alle Pixel derselben Klasse teilen dasselbe Label, unabhängig davon, zu welchem individuellen Objekt sie gehören.

Instanzsegmentierung: Jede einzelne Objektinstanz erhält ein eindeutiges Label. Eine Menschenmenge von 20 Personen wird zu 20 unterschiedlichen Masken.

Panoptische Segmentierung: Kombiniert beides: „Ding"-Klassen (zählbare Objekte) haben Instanzmasken; „Stoff"-Klassen (Straße, Himmel) haben semantische Masken.

SAM-unterstützte Annotation: Metas Segment Anything Model {cite}`kirillov2023segment` generiert aus einem einzelnen Punkt-Klick hochwertige Segmentierungsmasken. Annotatoren klicken auf die Mitte eines Objekts; SAM schlägt eine Maske vor; der Annotator akzeptiert oder korrigiert. Die SAM-Autoren berichten Beschleunigungen der Annotationseffizienz von etwa dem 6,5-fachen gegenüber polygonbasierter Kennzeichnung; die Gewinne variieren je nach Szenentyp und Annotationswerkzeugen.

SAM repräsentiert einen umfassenderen Wandel: **Die Rolle des Annotators ändert sich vom Zeichnen zum Überprüfen**. Dies hat über die Geschwindigkeit hinaus Implikationen für die Annotationsqualität. Wenn Annotatoren zeichnen, ist ihre Aufmerksamkeit während des gesamten Prozesses engagiert. Wenn Annotatoren überprüfen und „Akzeptieren" klicken, gibt es Belege dafür, dass sie Fehler leichter übersehen — eine Version des Automatisierungsbiases spezifisch für den Annotationskontext.

Aktives Lernen für Computer-Vision

Aktives Lernen ist besonders wertvoll in CV, weil: 1. Bilder hochdimensional und merkmalreich sind — Einbettungen von vortrainierten Modellen tragen starke Signale für die Unsicherheitsschätzung 2. Große unannotierte Pools billig sind (Fotos, Video-Frames) 3. Annotation (insbesondere Segmentierung) teuer ist und für spezialisierte Domänen nicht einfach per Crowdsourcing durchgeführt werden kann

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
```

```python
rng = np.random.default_rng(42)

X, y = make_classification(n_samples=3000, n_features=50, n_informative=25,
                           n_classes=5, n_clusters_per_class=2, random_state=42)
X_train, y_train = X[:2500], y[:2500]
X_test,  y_test  = X[2500:], y[2500:]

def margin_uncertainty(model, X_pool):
    probs = model.predict_proba(X_pool)
    sorted_p = np.sort(probs, axis=1)
    return sorted_p[:, -2] - sorted_p[:, -1]  # most negative = most uncertain

n_init = 50
results = {'active': [], 'random': []}
label_counts = list(range(50, 401, 30))

for strategy in ['active', 'random']:
    labeled = list(rng.choice(len(X_train), n_init, replace=False))
    unlabeled = [i for i in range(len(X_train)) if i not in labeled]

    for target in label_counts:
        while len(labeled) < target and unlabeled:
            if strategy == 'active' and len(labeled) >= 10:
                model_temp = LogisticRegression(max_iter=300).fit(
                    X_train[labeled], y_train[labeled])
                margins = margin_uncertainty(model_temp, X_train[unlabeled])
                idx = int(np.argmin(margins))
            else:
                idx = rng.integers(0, len(unlabeled))
            labeled.append(unlabeled.pop(idx))

        clf = LogisticRegression(max_iter=300).fit(X_train[labeled], y_train[labeled])
        results[strategy].append(accuracy_score(y_test, clf.predict(X_test)))

plt.figure(figsize=(7, 4))
plt.plot(label_counts, results['active'], 'o-', color='#2b3a8f',
         linewidth=2, label='Active (margin sampling)')
plt.plot(label_counts, results['random'], 's--', color='#e05c5c',
         linewidth=2, label='Random baseline')
plt.xlabel("Labeled training images", fontsize=12)
plt.ylabel("Test accuracy", fontsize=12)
plt.title("Active Learning for 5-Class Image Classification", fontsize=13)
plt.legend(fontsize=11); plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('cv_active_learning.png', dpi=150)
plt.show()
```

Semi-Supervised Learning mit menschlicher Lenkung

Die große Menge verfügbarer unannotierter visueller Daten macht semi-supervised learning besonders attraktiv für CV.

Self-Training / Pseudo-Kennzeichnung: Ein Modell auf annotierten Daten trainieren; hochkonfidente Vorhersagen auf unannotierten Daten als Pseudo-Labels verwenden; neu trainieren. Die kritische Designfrage ist der Konfidenz-Schwellenwert. Ein niedrigerer Schwellenwert bringt mehr Beispiele ein, aber führt Rauschen ein; ein hoher Schwellenwert lässt den Großteil des unannotierten Pools ungenutzt. Menschliche Beteiligung kann diesen Schwellenwert leiten — Annotatoren überprüfen stichprobenartig pseudo-annotierte Beispiele, um ihn zu kalibrieren.

FixMatch und Konsistenz-Regularisierung: Diese Methoden trainieren Modelle, um konsistente Vorhersagen unter Augmentierung zu produzieren. Die wichtigste HITL-Erkenntnis: Menschen werden nicht nur für Labels konsultiert, sondern für **Augmentierungsdesign** — welche Invarianzen soll das Modell lernen? Ein Modell für medizinische Bildgebung sollte gegenüber Rotation und Helligkeit invariant sein, nicht aber gegenüber Skalierung; ein Modell für Texterkennung sollte nicht gegenüber perspektivischer Verzerrung invariant gemacht werden. Diese domänenspezifischen Entscheidungen erfordern menschliche Expertise, und ihr Fehler beeinträchtigt das semi-supervised learning erheblich.

Aktives semi-supervised learning: Die effizienteste Kombination: Aktives Lernen konzentriert menschliche Labels dort, wo die Modellunsicherheit am höchsten ist; Self-Training annotiert automatisch den Hochkonfidenz-Schwanz. Der menschliche Aufwand konzentriert sich dort, wo er am wertvollsten ist, und das Modell bootstrapt sich auf den Rest. In jedem Zyklus liefert ein menschliches Audit einer Zufallsstichprobe von Pseudo-Labels eine Qualitätsprüfung, ohne vollständige Überprüfung zu erfordern.

Videoannotation

Videoannotation multipliziert die Herausforderungen der Bildannotation um die Zeit:

- **Tracking:** Objekte müssen über Frames hinweg identifiziert werden. Annotatoren beschriften Schlüsselframes; Tracking-Algorithmen interpolieren dazwischen. Tracking-Versagen — Verdeckung, Wiedereintritt, schnelle Bewegung — erfordern höhere Raten menschlicher Neubeschriftung als stabiles Tracking.
- **Zeitliche Konsistenz:** Im Frame t gezeichnete Grenzen sollten räumlich konsistent mit Frame $t + 1$ sein. Inkonsistente Annotationen sind ein Trainingssignal, das dem Modell sagt, dass Objekte diskontinuierlich springen — eine Form von Annotationsrauschen, die besonders schädlich für Erkennungsmodelle ist.
- **Skalierbarkeit:** Ein 1-stündiges Video bei 30fps umfasst 108.000 Frames. Vollständige Annotation ist nicht praktikabel; Stichprobenstrategien müssen sorgfältig entworfen werden, um sicherzustellen, dass seltene Ereignisse (Randfälle, Beinahe-Fehler, Fehlerszenarien) nicht systematisch ausgeschlossen werden.

Moderne Videoannotationswerkzeuge unterstützen **intelligentes Tracking**, das Annotationen über Frames propagiert und Frames markiert, bei denen die Tracking-Konfidenz unter einen Schwellenwert fällt, und den Annotator auffordert, neu zu prüfen. Dies ist eine direkte Anwendung der aktiven Lernidee auf die Annotationspipeline selbst: Das Werkzeug fragt den Annotator genau dort, wo seine Interpolation unsicher ist.

Das Seltene-Ereignisse-Problem in autonomen Systemen. Bei Anwendungen, bei denen die Konsequenzen seltener Ereignisse katastrophal sind — autonomes Fahren, UAV-Navigation — ist die Verteilung der im normalen Betrieb gesehenen Frames schlecht mit der Verteilung der tatsächlich wichtigsten Frames abgestimmt. Ein Datensatz, der durch gleichmäßiges Sampling von Fahrvideo erstellt wird, enthält Millionen von „nichts Interessantes passiert"-Frames und eine Handvoll der Beinahe-Unfall-, Ungewöhnlich-Beleuchtungs- und Sensor-Degradierungs-Frames, die tatsächlich für die Sicherheit wichtig sind. HITL-Aktives Lernen, das solche Frames identifiziert und priorisiert, ist kein Effizienz-Hack — es ist eine Sicherheitsanforderung.

> **See also:** ImageNet-Datensatz: {cite}`russakovsky2015imagenet`. Label-Verzerrung in ImageNet: {cite}`yang2020towards`. COCO-Benchmark: {cite}`lin2014microsoft`. SAM (Segment Anything): {cite}`kirillov2023segment`. Core-Set aktives Lernen für CV: {cite}`sener2018active`.

HITL im Gesundheitswesen und in der Wissenschaft

Gesundheitswesen und Wissenschaft repräsentieren zwei der Domänen, in denen HITL ML am folgenreichsten und am meisten diskutiert wird. Die Einsätze sind hoch: Eine verfehlte Krebsdiagnose oder ein fehlerhaftes Arzneimittelziel hat echte menschliche Kosten. Annotation erfordert seltene und teure Expertise. Regulatorische Anforderungen schränken ein, was Modelle tun können und wie sie validiert werden müssen. Und im Gegensatz zu NLP, wo das Annotationsproblem teilweise ein soziales Konstruktionsproblem ist, gibt es hier oft echte Grundwahrheit — ein Tumor ist vorhanden oder nicht —, auch wenn kein individueller Beobachter sie zuverlässig bestimmen kann.

Das dominierende Rahmen in populären Berichten ist „KI vs. Mensch": Wird KI Radiologen ersetzen? Dieses Rahmen ist falsch auf eine Art, die zählt. Die eigentliche Frage ist, welche Form der Mensch-KI-Zusammenarbeit bessere Ergebnisse produziert als jedes allein, und wie man Systeme aufbaut, die diese Zusammenarbeit ermöglichen, anstatt sie zu stören.

Medizinische Bildanalyse

Medizinische Bildgebung — Radiologie (Röntgen, CT, MRT), Pathologie (Gewebeschnitte), Dermatologie, Ophthalmologie — ist die Domäne, in der medizinische KI am schnellsten vorangeschritten ist.

Anforderungen an Experten-Annotatoren

Medizinische Bildannotation erfordert typischerweise Ärzte mit spezifischer Facharztausbildung. Dies macht Annotation:

- **Langsam:** Spezialisten haben begrenzte Zeit; Annotation konkurriert mit klinischen Pflichten
- **Teuer:** Kosten reichen von Dutzenden bis Hunderten von Dollar pro annotiertem Fall, abhängig von Subspezialität, Modalität und Aufgabenkomplexität
- **Variabel:** Selbst Spezialisten sind uneinig, besonders bei Grenzfällen — eine Tatsache, die oft als Problem behandelt wird, aber tatsächlich informativ ist

Inter-Radiologen-Variabilität

Leservariabilität ist in der Radiologie gut dokumentiert. Bei der Röntgenthorax-Interpretation ist die Leseruneinigkeit substanziell — in der CheXNet-Studie kennzeichneten vier Radiologen dasselbe Pneumonie-Erkennungstestset mit F1-Scores, die etwa 12 Prozentpunkte umspannten {cite}`rajpurkar2017chexnet`, was echte diagnostische Unsicherheit bei

Grenzfällen widerspiegelt. Für die Nodulenerkennung auf Lungen-CT kann die Intra-Leservariabilität (selber Leser, selber Fall, anderer Tag) so groß sein wie die Inter-Leservariabilität.

Diese Variabilität ist nicht nur Rauschen — sie spiegelt echte diagnostische Unsicherheit wider. Auf den Annotationen eines einzelnen Radiologen trainierte Modelle können die spezifischen Verzerrungen dieses Arztes erlernen anstatt der zugrunde liegenden Pathologie.

:::{admonition} Die CheXNet-Kontroverse als HITL-Lektion :class: note

Als Rajpurkar et al. behaupteten, ihr CheXNet-Modell habe bei der Pneumonieerkennung „die Radiologenleistung übertroffen", wurde die Behauptung sofort von der Radiologiegemeinschaft bestritten {cite}`yu2022assessing`. Ein Teil der Kontroverse betraf das spezifische Testset und den Radiologievergleich. Aber ein tieferes Problem war methodologisch: Die verwendete „Radiologenleistung"-Baseline verwendete Einzelleser unter Zeitdruck, während klinische Radiologie typischerweise Konsultation, Vergleich mit früherer Bildgebung und Zugang zu klinischem Kontext umfasst — nichts davon hatte das Modell.

Die Lektion ist nicht, dass das Modell gut oder schlecht war, sondern dass **Leistungsvergleiche die HITL-Konfiguration spezifizieren müssen**. Ein Modell, das einen einzelnen Radiologen bei kaltem Lesen übertrifft, kann immer noch weniger genau sein als ein Radiologe, der die Ausgabe des Modells als zweite Meinung verwendet. Dies sind verschiedene Systeme mit verschiedenen Fehlermodi, und ihre aggregierte Wirkung unterscheidet sich. :::

:::{admonition} Soft-Labels in der Medizin :class: important

Mehrere medizinische KI-Projekte sind zu **Soft-Labels** übergegangen, die die Verteilung von Expertenmeinungen widerspiegeln, anstatt eines einzelnen „Goldstandard"-Labels. Ein Röntgenthorax, der von einem Radiologenausschuss als 60 % Pneumonie / 40 % Atelektase gekennzeichnet wird, trägt mehr Information als eine erzwungene binäre Wahl. Auf solchen Verteilungen trainierte Modelle zeigen bessere Kalibrierung und angemessenere Unsicherheitsquantifizierung — und diese Unsicherheit ist klinisch bedeutsam, da sie dem Kliniker sagt, wann er konsultieren soll, nicht nur, was das Modell denkt. :::

Aktives Lernen für seltene Erkrankungen

Aktives Lernen ist besonders wertvoll für seltene Erkrankungen und seltene Pathologien, bei denen selbst ein großer unannotierter Pool nur wenige positive Fälle enthält. Standard-Zufalls-Sampling würde Expertenzeit verschwenden, indem meistens negative Fälle annotiert werden.

Unsicherheitsbasiertes aktives Lernen wählt natürlich die Grenzfälle, bei denen das Modell unsicher ist — die bei seltenen Erkrankungen tendenziell die positiven Fälle und Grenz-Negative sind. Dies konzentriert Spezialisten-Zeit dort, wo sie am wertvollsten ist. Die Kombination aus klassenunausgeglichenem Training (mit `class_weight='balanced'` oder ähnlichem) und unsicherheitsbasierter Auswahl ist eine Standardpraxis bei Selten-Pathologie-Erkennungsaufgaben.

Klinische NLP-Annotation

Elektronische Krankenakten (EHRs) enthalten einen enormen Reichtum klinischer Erzähltexte: Arztnotizen, Entlassungszusammenfassungen, Radiologieberichte, Pathologieberichte. Die Extraktion strukturierter Information aus diesem Text erfordert NLP — und hochwertiges NLP erfordert annotierte Trainingsdaten.

Häufige klinische NLP-Annotationsaufgaben: - **Klinisches NER:** Identifizierung von Medikamenten, Dosierungen, Diagnosen, Verfahren und Symptomen in Text - **Negationserkennung:** „Kein Anzeichen einer Pneumonie" vs. „Pneumonie bestätigt" — eine kritische Unterscheidung, die überraschend schwierig ist - **Temporales Denken:** Unterscheidung aktueller Erkrankungen von der Vorgeschichte („Vorgeschichte eines MI, vorstellig mit Brustschmerzen") - **De-Identifizierung:** Entfernung von geschützten Gesundheitsinformationen (PHI) zur Ermöglichung von Datenaustausch

PHI-De-Identifizierung ist sowohl eine Annotationsaufgabe als auch eine Datenverwaltungsanforderung. Unter HIPAA (USA) und DSGVO (EU) können Gesundheitsdaten nicht ohne Entfernung oder Anonymisierung von Patientenidentifikatoren geteilt werden. Automatisierte De-Identifizierungstools existieren, sind aber unvollkommen; menschliche Überprüfung automatisierter Ausgaben ist Standardpraxis, und das Risikoprofil ist asymmetrisch: Falsch-Negative (übersehene PHI) schaffen rechtliche Exposition, was konservative Schwellenwerte notwendig macht.

i2b2 / n2c2 als Vorlage

Die i2b2 (Informatics for Integrating Biology and the Bedside) und die Nachfolge-n2c2 (National NLP Clinical Challenges) Shared-Task-Initiativen haben eine Reihe von experten-annotierten klinischen NLP-Datensätzen veröffentlicht. Diese illustrieren sowohl das Potenzial als auch die Kosten: Annotationsanstrengungen umfassen typischerweise Teams klinischer Fachexperten, die mehrere Monate lang an Hunderten von Dokumenten pro Challenge arbeiten. Die n2c2-Datensätze haben schnellen Fortschritt genau deshalb katalysiert, weil sie das Datenaustausch-Governance-Problem (De-Identifizierung + institutionelle Vereinbarungen) gelöst haben, nicht nur das Annotationsproblem.

Regulatorische Überlegungen

Medizinische KI unterliegt in den meisten Rechtsordnungen regulatorischer Aufsicht.

FDA (Vereinigte Staaten): KI/ML-basierte Software als medizinisches Gerät (SaMD) erfordert Vormarktzulassung oder -genehmigung. Der KI/ML-Aktionsplan der FDA von 2021 betont **vorher festgelegte Änderungskontrollpläne** — die Dokumentierung, wie das Modell aktualisiert wird und wie diese Aktualisierungen vor dem Einsatz validiert werden. Ein Modell, das kontinuierlich aus klinischem Feedback lernt, ist unter diesem Rahmen nach jeder Aktualisierung ein anderes Gerät und kann eine Neuvalidierung erfordern.

CE-Kennzeichnung (Europa): Medizinische Geräte einschließlich KI-Systeme müssen der Medizinprodukteverordnung (MDR) entsprechen. Die MDR erfordert klinische Bewertung, Post-Market-Surveillance und Dokumentation der für Training und Validierung verwendeten Daten.

Wichtige HITL-Implikation: Regulatorische Rahmen erfordern eine klare Dokumentation von Annotationsprozessen, Annotatorenqualifikationen, Interrater-Reliabilität und etwaigen Änderungen der Trainingsdaten. Dies ist kein bürokratischer Overhead — es ist der Prüfpfad, der es einem Kliniker ermöglicht zu verstehen, welche Trainingsdaten das aktuelle Verhalten des Modells produziert haben, und er ist gesetzlich vorgeschrieben. HITL-Pipelines, die Annotation als informellen Unterprozess behandeln, schaffen regulatorisches Risiko, das typischerweise nur im schlimmsten Moment sichtbar wird.

Wissenschaftliche Datenannotation

Über das Gesundheitswesen hinaus spielt HITL ML eine wachsende und unterschätzte Rolle in der wissenschaftlichen Forschung, wo die Annotationsherausforderung oft Domänenexpertise mit Skalierung verbindet.

Astronomie: Galaxy Zoo

Galaxy Zoo {cite}`lintott2008galaxy` hat die morphologische Klassifikation von Galaxien aus dem Sloan Digital Sky Survey an Bürgerwissenschaftler vergeben. Das ursprüngliche Projekt sammelte über 40 Millionen Klassifikationen von mehr als 100.000 Freiwilligen und demonstrierte, dass Crowdsourcing wissenschaftlicher Bildklassifikation im großen Maßstab machbar ist, wenn die Aufgabe in einfache Fragen zerlegt werden kann, die ohne Spezialausbildung beantwortet werden können („Ist diese Galaxie glatt oder strukturiert?“).

Das Galaxy-Zoo-Erlebnis produzierte zwei wichtige methodologische Erkenntnisse. Erstens war die Übereinstimmung zwischen Bürgerwissenschaftlern und professionellen Astronomen bei klaren Fällen hoch und wich systematisch bei Grenzfällen ab — genau den Fällen, bei denen die Unterscheidung wissenschaftlich wichtig ist. Die Lösung bestand nicht darin, Bürgerwissenschaftsdaten bei Grenzfällen zu verwerfen, sondern die Verteilung der Freiwilligenantworten als Soft-Label zu behandeln, das echte morphologische Ambiguität kodiert. Zweitens übertraf der auf Galaxy-Zoo-Labels trainierte Klassifikator Modelle, die trainiert wurden, die Annotationen eines einzelnen Experten zu reproduzieren, weil die Menge-Verteilung echte visuelle Unsicherheit erfasste, die eine erzwungene Wahl eines einzelnen Experten kollabierte.

Genomik: Pathogenitätsklassifikation

Die Annotation genomischer Varianten — die Entscheidung, ob eine Variante pathogen, gutartig oder von unsicherer Bedeutung ist — ist ein hochriskantes NLP- und Expertenjudgment-Problem. Klinische Variantendatenbanken wie ClinVar aggregieren Experteninterpretationen von mehreren einsendenden Labors, und Uneinigkeiten zwischen Labs sind häufig. Aktives Lernen wird verwendet, um zu priorisieren, welche Varianten eine vollständige Expertenüberprüfung (Literatursuche, Bewertung funktionaler Evidenz) erfordern, gegenüber denen, die durch bestehende Evidenz automatisch klassifiziert werden können. Das Ergebnis ist eine Hybrid-Pipeline, bei der die meisten Varianten durch automatisierte Logik behandelt werden, eine Teilmenge eine Expertenüberprüfung erfordert und die schwierigsten Fälle für den Multi-Lab-Konsensus markiert werden.

Klima- und Geowissenschaften

Die Kennzeichnung von Satellitenbildern für Landnutzungsänderungen, Entwaldung, Gletscherausdehnung und Sturmbahnen umfasst Fernerkundungsexperten und zunehmend Bürgerwissenschaftsplattformen. Die primäre HITL-Herausforderung in dieser Domäne ist zeitlich: Heute erstellte Labels können veralten, wenn sich die Welt ändert, und die Ground-Truth-Verifikation (Felderhebungen) ist teuer und logistisch eingeschränkt. Aktives Lernen, das Bilder priorisiert, bei denen die Modellvorhersage mit physikalischen Vorannahmen nicht übereinstimmt (z. B. Vorhersage von Entwaldung in einer bekanntermaßen geschützten Region), ist eine praktische Möglichkeit, knappe Feldverifikationsressourcen zu lenken.

Neurowissenschaften: Konnektomik

Die Rekonstruktion neuronaler Schaltkreise aus Elektronenmikroskopie-Bildern — Konnektomik — erfordert pixelgenaue Annotation von Neuronenmembranen über riesige Bildstapel. Das Eyewire-Projekt hat diese Aufgabe gamifiziert und Zehntausende von Spielern in die Verfolgung von Neuronen durch 3D-Bildvolumina einbezogen. Das Gamification-Design löste ein spezifisches HITL-Problem: Die Aufgabe erfordert anhaltende Aufmerksamkeit und räumliches Denken über lange Sitzungen, was zu Qualitätsverschlechterung bei der traditionellen Annotation führt. Das Aufteilen der Aufgabe in Spielsegmente mit sozialen Mechaniken hielt das Annotator-Engagement und die Qualität in Maßstäben aufrecht, die professionelle Annotation nicht erreichen kann.

Management von Experten-Annotatoren

Wenn Annotation seltene Expertise erfordert, gelten die üblichen Crowdsourcing-Ansätze (Kapitel 13) nicht.

Die fundamentale Spannung ist, dass die Menschen, die Annotationen höchster Qualität produzieren können, auch die Menschen sind, deren Zeit am wertvollsten und am stärksten beschränkt ist. Jede Designentscheidung in einer Experten-Annotationspipeline sollte gegen die Frage bewertet werden: Macht dies den besten Einsatz knapper Expertenzeit?

Was dies in der Praxis bedeutet:

- **Aggressiv vorannotieren.** Niedrig-qualifizierte Annotatoren, automatisierte Modelle oder regelbasierte Systeme verwenden, um Kandidaten zu generieren, die der Spezialist überprüft und korrigiert, anstatt von Grund auf zu erstellen. Das Urteil des Spezialisten ist der Engpass; ihm eine Vorannotation zur Korrektur zu geben ist schneller als ihn von einem leeren Bildschirm aus zu fragen, vorausgesetzt die Vorannotationsqualität ist ausreichend, dass Korrektur nicht langsamer ist als neu zu beginnen.
- **Auf Expertenaufmerksamkeit, nicht Durchsatz ausrichten.** Für hohen Durchsatz optimierte Annotationsinterfaces (schnelle binäre Entscheidungen, Tastaturkürzel, minimale Anzeige) sind für Crowdsourcing geeignet. Experten-Annotation profitiert oft von reichhaltigeren Interfaces: Seitenvergleich mit früheren Fällen, einfacher Zugang zu Referenzmaterialien, Annotationsvertrauensfelder und die Möglichkeit, einen Fall zur Diskussion zu markieren. Diese verlangsamen einzelne Annotationen, verbessern aber die Qualität und reduzieren den Bedarf an Neuannotation.
- **Individuelle Annotatorenmuster explizit verfolgen.** Mit einem kleinen Pool von Spezialisten ist es machbar und wichtig, die Übereinstimmungsrate jedes Annotators mit dem Panel zu verfolgen, Fälle zu markieren, die mit ihrer eigenen Geschichte inkonsistent erscheinen, und sie in regelmäßigen Kalibrierungssitzungen zu besprechen. Dies ist keine Überwachung — es ist derselbe Qualitätsprozess, den die klinische Medizin für die Leistungsüberprüfung verwendet, und Spezialisten reagieren im Allgemeinen gut, wenn er als gemeinsame Qualitätsverbesserung und nicht als Bewertung gerahmt wird.
- **Sitzungsdesign ist wichtig.** Medizinische Annotation ist kognitiv anspruchsvoll. Belege aus Radiologie und Pathologie deuten darauf hin, dass Fehlerraten nach etwa 90 Minuten kontinuierlichen Lesens messbar steigen, und dass Pausen von wenigen Minuten die Aufmerksamkeit teilweise wiederherstellen können. Annotationsinterfaces, die Pausenaufforderungen durchsetzen (ohne dass sie abgebrochen werden können), sind eine einfache HITL-Designentscheidung mit echtem Qualitätseinfluss.

Eine HITL-Aktive-Lern-Pipeline für medizinische Bildgebung

```python
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
from sklearn.datasets import make_classification
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)

# Simulate a rare-pathology detection task
# 8% positive class (e.g., rare pathology)
X, y = make_classification(
    n_samples=5000, n_features=100,
    n_informative=20, n_redundant=10,
    weights=[0.92, 0.08],
    random_state=42
)
X_train, y_train = X[:4000], y[:4000]
X_test,  y_test  = X[4000:], y[4000:]

print(f"Training set positive prevalence: {y_train.mean():.1%}")

def run_medical_al(strategy, n_initial=50, budget=300):
    labeled = list(rng.choice(len(X_train), n_initial, replace=False))
    unlabeled = [i for i in range(len(X_train)) if i not in labeled]
    aucs = []

    while len(labeled) < n_initial + budget:
        model = LogisticRegression(max_iter=500, class_weight='balanced')
        model.fit(X_train[labeled], y_train[labeled])

        if len(labeled) % 30 == 0:
            preds = model.predict_proba(X_test)[:, 1]
            aucs.append(roc_auc_score(y_test, preds))

        X_pool = X_train[unlabeled]
        if strategy == 'uncertainty' and len(labeled) >= 10:
            probs = model.predict_proba(X_pool)
            entropy = -np.sum(probs * np.log(probs + 1e-9), axis=1)
            q = int(np.argmax(entropy))
        else:
            q = rng.integers(0, len(unlabeled))

        labeled.append(unlabeled.pop(q))

    return np.array(aucs)

aucs_al  = run_medical_al('uncertainty')
```

```
aucs_rnd = run_medical_al('random')
label_counts = np.arange(len(aucs_al)) * 30 + 50

plt.figure(figsize=(7, 4))
plt.plot(label_counts, aucs_al,  'o-',  color='#2b3a8f', linewidth=2, label='Uncertainty AL')
plt.plot(label_counts, aucs_rnd, 's--', color='#e05c5c', linewidth=2, label='Random baseline'
plt.xlabel("Expert labels obtained", fontsize=12)
plt.ylabel("AUROC", fontsize=12)
plt.title("Active Learning for Rare Pathology Detection", fontsize=13)
plt.legend(); plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('medical_al.png', dpi=150)
plt.show()

# Show how AL preferentially selects positive examples
# by checking which class the queried examples belong to
n_init = 50
labeled_al  = list(rng.choice(len(X_train), n_init, replace=False))
labeled_rnd = labeled_al.copy()
unlabeled_al  = [i for i in range(len(X_train)) if i not in labeled_al]
unlabeled_rnd = unlabeled_al.copy()

model = LogisticRegression(max_iter=500, class_weight='balanced')
model.fit(X_train[labeled_al], y_train[labeled_al])
probs = model.predict_proba(X_train[unlabeled_al])
entropy = -np.sum(probs * np.log(probs + 1e-9), axis=1)
top50_al  = [unlabeled_al[i] for i in np.argsort(entropy)[-50:]]
top50_rnd = list(rng.choice(unlabeled_rnd, 50, replace=False))

pos_rate_al  = y_train[top50_al].mean()
pos_rate_rnd = y_train[top50_rnd].mean()
print(f"\nPositive rate in next 50 queries:")
print(f"  Uncertainty AL: {pos_rate_al:.1%}  (base rate: {y_train.mean():.1%})")
print(f"  Random:         {pos_rate_rnd:.1%}")
print(f"  AL queries {pos_rate_al/y_train.mean():.1f}x more positives than random")
```

See also: Galaxy Zoo Crowdsourcing: {cite}`lintott2008galaxy`. CheXNet Radiologieleistung: {cite}`rajpurkar2017chexnet`. Röntgenqualität und KI-gestützte Diagnose: {cite}`yu2022assessing`. Klinische NLP-Annotationsmethodik: {cite}`pustejovsky2012natural`. Für FDA KI/ML-Aktionsplan-Leitlinien, siehe die veröffentlichte FDA-Dokumentation (2021).

Annotationsplattformen und Werkzeuge

Die Annotationsplattform ist die Umgebung, in der menschliches Feedback zu Daten wird. Eine gute Plattform erhöht den Durchsatz, reduziert Fehler, erhält Qualitätskontrollen aufrecht und macht die Annotationspipeline im großen Maßstab handhabbar. Die richtige Plattform zu wählen — und zu wissen, wann man baut vs. kauft — ist eine folgenreiche Entscheidung in jedem HITL-Projekt.

Die Annotationsplattform-Landschaft

Der Markt für Annotationswerkzeuge ist in den letzten Jahren erheblich gewachsen und gereift, angetrieben durch die Unternehmensnachfrage nach ML-Trainingsdaten. Werkzeuge reichen von vollständig verwalteten Diensten bis zu Open-Source-Self-Hosted-Rahmenwerken.

Open-Source-Plattformen

Label Studio ist die beliebteste Open-Source-Annotationsplattform und unterstützt Text-, Bild-, Audio-, Video- und Zeitreihendaten durch eine einheitliche XML-basierte Aufgabenkonfiguration. Es kann selbst gehostet werden und integriert sich mit ML-Backends für aktives Lernen. Hauptstärken: Flexibilität, Gemeinschaftsunterstützung und die Möglichkeit, benutzerdefinierte ML-Vorhersagen für Vorannotation einzubetten.

Prodigy (von den Machern von spaCy) ist ein hochgradig workflow-orientiertes Annotationswerkzeug für NLP-Aufgaben. Seine Streaming-Architektur sendet Beispiele einzeln und unterstützt aktives Lernen von Haus aus. Das Annotationsinterface ist minimal, aber schnell — konzipiert um den Annotationsdurchsatz zu maximieren. Prodigy ist bezahlte Software, aber in der NLP-Forschung weit verbreitet.

CVAT (Computer Vision Annotation Tool) ist das führende Open-Source-Werkzeug für CV-Annotation mit starker Unterstützung für Erkennung, Segmentierung und Videoannotation. Ursprünglich bei Intel entwickelt, unterstützt CVAT Tracking-Algorithmen, Skelett-Annotation und Drittanbieter-Algorithmus-Integrationen.

Doccano zielt auf Sequenzkennzeichnungsaufgaben (NER, Relationsextraktion, Textklassifikation) ab. Sein einfaches Web-Interface macht es für Teams ohne dedizierte Datentechnik-Ressourcen zugänglich.

Kommerzielle Plattformen

Scale AI bietet End-to-End-verwaltete Annotationsdienste: menschliche Belegschaft, Qualitätsmanagement und API-Integration. Besonders stark bei autonomem Fahren, Robotik und komplexer 3D-Annotation. Die Preisgestaltung basiert auf Aufgabenkomplexität und Volumen.

Labelbox ist eine vollständige Plattform für Datenkuration, Annotation und ML-gestützte Kennzeichnung. Starke Enterprise-Funktionen: Projektmanagement, Qualitäts-Workflows, Modell-Feedback-Schleifen und Integrationen mit großen ML-Plattformen (SageMaker, Vertex AI, Azure ML).

Appen (ehemals Figure Eight / CrowdFlower) betreibt eine große globale Annotationsbelegschaft neben Werkzeugen. Eine gute Wahl, wenn Volumen und Belegschaftsmanagement primäre Anliegen sind.

Surge AI konzentriert sich auf Experten-Annotatoren und ist stark für Aufgaben, die Domänenwissen oder nuanciertes Urteil erfordern.

Humanloop ist spezialisiert auf LLM-Feedback-Sammlung — Präferenz-Annotation, RLHF-Datensammlung und Modellevaluierung.

Vergleich der Annotationsplattform-Funktionen

Funktion	Label Studio	Prodigy	CVAT	Labelbox	Scale AI
Lizenz	Open Source	Kommerziell	Open Source	Kommerziell	Kommerziell
Hosting	Selbst / Cloud	Selbst	Selbst / Cloud	Cloud	Verwaltet
Textannotation	□	□	—	□	□
Bildannotation	□	Begrenzt	□	□	□
Videoannotation	□	—	□	□	□
Integration aktives Lernen	□	□	Begrenzt	□	□
ML-gestützte Vorannotation	□	□	□	□	□
Qualitätskontrolle Workflows	Grundlegend	Grundlegend	Grundlegend	Erweitert	Erweitert
API / Programmatischer Zugang	□	□	□	□	□
Belegschaftsmanagement	—	—	—	Begrenzt	□

Annotation als Code

Ein kritischer, aber oft übersehener Aspekt der Annotationsinfrastruktur ist **Versionskontrolle für Annotationen und Annotationsschemata**. Annotation als Code zu behandeln bedeutet:

Schema-first-Design. Die Label-Taxonomie und Annotationsregeln werden in einer versionierten Konfigurationsdatei (YAML oder JSON) definiert, bevor die Annotation beginnt. Änderungen am Schema erzeugen eine neue Version.

Annotationsversionierung. Annotationen werden mit einem Link zur Schema-Version gespeichert, unter der sie erstellt wurden. Dies ermöglicht Prüfung, Rollback und Vergleich von Annotationen über Schema-Versionen hinweg.

Reproduzierbare Pipelines. Die Annotationspipeline — von Rohdaten zu trainingsfertigen Labels — sollte aus Code reproduzierbar sein. Vorannotationsmodelle, Qualitätsfilter, Aggregationslogik und Datenteilungen sollten alle aufgezeichnet werden.

```yaml
# Example: Label Studio annotation schema (text classification)
label_config: |
  <View>
    <Text name="text" value="$text"/>
    <Choices name="sentiment" toName="text" choice="single">
      <Choice value="positive"/>
      <Choice value="negative"/>
      <Choice value="neutral"/>
      <Choice value="mixed"/>
    </Choices>
  </View>
schema_version: "2.1.0"
task_type: text_classification
guidelines_version: "guidelines_v3.pdf"
```
```text
Datenquelle
    |
    v
Unannotierter Pool -->  Annotationsplattform --> Annotierter Datensatz
    ^                    |                              |
    |                    | (ML-gestützt)                v
Aktives Lernen <---------+                        Trainingslauf
Anfragestrategie                                      |
    ^                                                 v
    +------------ Trainiertes Modell <---------- Evaluierung
                                                      |
                                              Einsatz &
                                              Überwachung
```

Wichtige Integrationspunkte: 1. **Datenerfassung:** Unannotierte Daten fließen automatisch aus dem Data Warehouse in die Annotationsplattform 2. **Modell-Serving:** Das aktuelle beste Modell wird in der Annotationsplattform für Vorannotation und aktives Lern-Scoring bereitgestellt 3. **Export:** Abgeschlossene Annotationen werden in einem Format exportiert, das mit dem Trainings-Rahmenwerk kompatibel ist (COCO JSON, Hugging Face Datasets usw.) 4. **Feedback-Schleife:** Produktionsmodellfehler werden zur Korrektur an die Annotationsplattform zurückgeleitet

> **See also:** Zur Label Studio-Dokumentation und aktiven Lern-Integration: https://labelstud.io. Zu Prodigy: https://prodi.gy. CVAT: https://cvat.ai. Für einen umfassenden Vergleich von Annotationswerkzeugen, siehe {cite}`monarch2021human`, Kapitel 7.

Crowdsourcing und Qualitätssicherung

Wenn Annotationsaufgaben einfach genug sind, um von Nicht-Experten durchgeführt zu werden, bieten Crowdsourcing-Plattformen Zugang zu großen, on-demand Annotationsbelegschaften zu niedrigen Stückkosten. Der Aufbau hochwertiger annotierter Datensätze aus Menschenmassen erfordert sorgfältiges Aufgabendesign, strategische Redundanz und rigorose Qualitätskontrolle.

Crowdsourcing-Plattformen

Amazon Mechanical Turk (MTurk) ist der originale Crowdsourcing-Marktplatz, der 2005 gestartet wurde. Arbeiter („Turker") erledigen Mikro-Aufgaben (HITs), die von Auftraggebern eingestellt werden. Eine Studie aus 2018 fand einen medianen effektiven Stundenlohn für Turker von ca. 2 $/Std. — weit unter dem Mindestlohn in vielen Hocheinkommensländern {cite}`hara2018data` — ein ethisches Anliegen, das in Kapitel 15 behandelt wird. MTurk eignet sich am besten für einfache Aufgaben mit klaren, verifizierbaren Kriterien.

Prolific ist eine akademische Crowdsourcing-Plattform, die einen Mindestlohnstandard durchsetzt (derzeit ca. 9 £/Std., ca. 11 $/Std., wie in den veröffentlichten Richtlinien von Prolific angegeben), Teilnehmer nach demografischen Merkmalen auswählt und ein Panel von Arbeitern pflegt, die sich für die Forschungsteilnahme angemeldet haben. Bevorzugt für sozialwissenschaftliche Forschung und Aufgaben, die Repräsentativität erfordern.

Appen (und ähnliche: Telus International, iMerit) bietet verwaltete Annotationsbelegschaften mit Qualitätsmanagement für komplexere Aufgaben und Unternehmensprojekte.

Spezialisierte Gemeinschaften. Für domänenspezifische Aufgaben können Gemeinschaften von Fachbegeisterten hochwertige Annotationen liefern: Galaxy Zoo für Astronomie, eBird für Vogelarten, Chess Tempo für Schachpositions-Annotation.

Aufgabendesign für Crowdsourcing

Komplexe Aufgaben zerlegen

Komplexe Aufgaben sollten in einfache, klar definierte Mikro-Aufgaben zerlegt werden. Anstatt Arbeiter zu bitten, ein Dokument umfassend zu annotieren, sollte man ihnen eine fokussierte Frage auf einmal stellen: „Enthält dieser Satz einen Personennamen?" oder „Bewerten Sie die Klarheit dieser Übersetzung auf einer Skala von 1–5."

Vorteile der Zerlegung: - Geringere kognitive Anforderung pro Aufgabe weniger Ermüdung, höhere Qualität - Jede Mikro-Aufgabe kann separat qualitätskontrolliert werden - Einfacher zu prüfen und zu debuggen

Die Bedeutung von Anweisungen

Der einzige wichtigste Prädiktor für Crowdsourcing-Qualität ist die Qualität der Anweisungen. Gute Aufgabenanweisungen: - Erklären den *Zweck* der Aufgabe in einem Satz - Geben eine klare, eindeutige Definition jeder Kategorie - Bieten 3–5 ausgearbeitete Beispiele (besonders Randfälle) - Sind nicht länger, als Arbeiter tatsächlich lesen werden (< 300 Wörter für einfache Aufgaben)

Führen Sie eine **Pilotstudie** durch (10–50 Arbeiter, 20–100 Aufgaben), bevor Sie skalieren. Analysieren Sie Pilot-Uneinigkeiten; die meisten zeigen auf Anweisungs-Ambiguitäten, die behoben werden können.

Goldstandard-Fragen

Goldstandard-Fragen — Aufgaben mit bekannten richtigen Antworten — in den Aufgabenstapel einbetten. Arbeiter, die Goldfragen unter einem Schwellenwert nicht bestehen, werden aus dem Projekt entfernt.

```
import numpy as np
from scipy.stats import binom

rng = np.random.default_rng(42)

def simulate_gold_screening(n_workers=100, gold_per_batch=5,
                            p_good_worker=0.7, good_acc=0.92,
                            bad_acc=0.55, threshold=0.70):
    """
    Simulate quality screening via gold questions.
    Returns: precision and recall of identifying bad workers.
    """
    worker_quality = rng.choice(['good', 'bad'], size=n_workers,
                                p=[p_good_worker, 1 - p_good_worker])
    results = {'tp': 0, 'fp': 0, 'tn': 0, 'fn': 0}

    for q in worker_quality:
        acc = good_acc if q == 'good' else bad_acc
        n_correct = rng.binomial(gold_per_batch, acc)
        passed = (n_correct / gold_per_batch) >= threshold
        if q == 'bad' and not passed:  results['tp'] += 1
        if q == 'good' and not passed: results['fp'] += 1
        if q == 'bad' and passed:      results['fn'] += 1
        if q == 'good' and passed:     results['tn'] += 1

    tp, fp, fn = results['tp'], results['fp'], results['fn']
    precision = tp / (tp + fp + 1e-6)
    recall    = tp / (tp + fn + 1e-6)
    return precision, recall

# Vary gold question count
gold_counts = [3, 5, 8, 10, 15, 20]
precisions, recalls = [], []
for g in gold_counts:
    p_list, r_list = [], []
```

```
    for _ in range(50):
        p, r = simulate_gold_screening(gold_per_batch=g)
        p_list.append(p); r_list.append(r)
    precisions.append(np.mean(p_list))
    recalls.append(np.mean(r_list))

import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(gold_counts, precisions, 'o-', color='#2b3a8f', label='Precision', linewidth=2)
ax.plot(gold_counts, recalls,    's--', color='#0d9e8e', label='Recall',    linewidth=2)
ax.set_xlabel("Gold questions per batch", fontsize=12)
ax.set_ylabel("Screening performance", fontsize=12)
ax.set_title("Worker Screening via Gold Standard Questions", fontsize=13)
ax.legend(fontsize=11); ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('gold_screening.png', dpi=150)
plt.show()

p, r = simulate_gold_screening(gold_per_batch=5)
print(f"5 gold questions: Precision={p:.3f}, Recall={r:.3f}")
p, r = simulate_gold_screening(gold_per_batch=10)
print(f"10 gold questions: Precision={p:.3f}, Recall={r:.3f}")
```

Statistische Modelle für Label-Aggregation

Mehrheitsvoting ist eine natürliche Ausgangslinie, ignoriert aber Unterschiede in der Annotatorengenauigkeit. Statistische Modelle können besser abschneiden.

Das Dawid-Skene-Modell

Das **Dawid-Skene-Modell (DS)** {cite}`dawid1979maximum` schätzt gemeinsam: - Das **wahre Label** z_i für jeden Eintrag i - Die **Konfusionsmatrix** $\pi_j^{(k,l)}$ für jeden Annotator j: die Wahrscheinlichkeit, dass Annotator j einen Eintrag mit wahrer Klasse k als Klasse l kennzeichnet

Der EM-Algorithmus iteriert: - **E-Schritt:** Gegeben Annotator-Konfusionsmatrizen, berechne die posteriore Wahrscheinlichkeit jedes wahren Labels - **M-Schritt:** Gegeben Item-Label-Schätzungen, aktualisiere Annotator-Konfusionsmatrizen

```
import numpy as np
from scipy.special import softmax

def dawid_skene_em(annotations, n_classes, n_iter=20):
    """
    Simplified Dawid-Skene EM for binary classification.
    annotations: dict {item_idx: [(annotator_idx, label), ...]}
    Returns: estimated true labels and annotator accuracies.
    """
    items = sorted(annotations.keys())
    n_items = len(items)
```

```
    annotators = sorted({a for anns in annotations.values() for a, _ in anns})
    n_annotators = len(annotators)
    ann_idx = {a: i for i, a in enumerate(annotators)}

    # Initialize: majority vote
    T = np.zeros((n_items, n_classes))  # soft label estimates
    for i, item in enumerate(items):
        for _, label in annotations[item]:
            T[i, label] += 1
        T[i] /= T[i].sum()

    # Confusion matrices: shape (n_annotators, n_classes, n_classes)
    PI = np.ones((n_annotators, n_classes, n_classes)) * 0.5

    for _ in range(n_iter):
        # M-step: update confusion matrices
        PI = np.zeros((n_annotators, n_classes, n_classes)) + 1e-6
        for i, item in enumerate(items):
            for ann, label in annotations[item]:
                a = ann_idx[ann]
                for k in range(n_classes):
                    PI[a, k, label] += T[i, k]
        PI /= PI.sum(axis=2, keepdims=True)

        # E-step: update soft label estimates
        T = np.zeros((n_items, n_classes))
        for i, item in enumerate(items):
            log_p = np.zeros(n_classes)
            for ann, label in annotations[item]:
                a = ann_idx[ann]
                log_p += np.log(PI[a, :, label] + 1e-10)
            T[i] = softmax(log_p)

    return {item: T[i] for i, item in enumerate(items)}, PI

# Simulate crowdsourcing scenario
rng = np.random.default_rng(42)
N_ITEMS, N_ANNOTATORS, ANNS_PER_ITEM = 200, 10, 3
true_labels = rng.integers(0, 2, N_ITEMS)
# Annotator accuracies: 3 good (0.9), 4 medium (0.75), 3 noisy (0.6)
accuracies = [0.90]*3 + [0.75]*4 + [0.60]*3

annotations = {}
for i in range(N_ITEMS):
    anns_for_item = []
    chosen = rng.choice(N_ANNOTATORS, ANNS_PER_ITEM, replace=False)
    for a in chosen:
        acc = accuracies[a]
        label = true_labels[i] if rng.random() < acc else 1 - true_labels[i]
        anns_for_item.append((a, int(label)))
    annotations[i] = anns_for_item
```

```
# Run Dawid-Skene
soft_labels, confusion = dawid_skene_em(annotations, n_classes=2, n_iter=30)
ds_preds = {i: int(np.argmax(soft_labels[i])) for i in range(N_ITEMS)}

# Compare with majority vote
mv_preds = {}
for i in range(N_ITEMS):
    votes = [l for _, l in annotations[i]]
    mv_preds[i] = int(np.round(np.mean(votes)))

ds_acc = np.mean([ds_preds[i] == true_labels[i] for i in range(N_ITEMS)])
mv_acc = np.mean([mv_preds[i] == true_labels[i] for i in range(N_ITEMS)])

print(f"Majority vote accuracy:  {mv_acc:.3f}")
print(f"Dawid-Skene accuracy:    {ds_acc:.3f}")
print(f"\nEstimated annotator accuracies (diagonal of confusion matrix):")
for a in range(N_ANNOTATORS):
    est_acc = np.mean([confusion[a, k, k] for k in range(2)])
    print(f"  Annotator {a}: estimated={est_acc:.3f}, true={accuracies[a]:.2f}")
```

MACE

MACE (Multi-Annotator Competence Estimation) {cite}`hovy2013learning` ist ein alternatives probabilistisches Modell, das explizit Annotator-Spamming (zufällige Kennzeichnung) vs. kompetente Annotation darstellt. Ein Annotator gibt entweder ein bedeutungsvolles Label (mit Wahrscheinlichkeit $1 - \text{spam}_j$) oder ein zufälliges Label (mit Wahrscheinlichkeit spam_j). Dieses Zwei-Komponenten-Modell ist oft besser kalibriert als Dawid-Skene für Crowdsourcing-Szenarien, bei denen einige Annotatoren reine Spam-Absender sind.

Redundanz- und Aggregationsstrategie

Die optimale Anzahl von Annotatoren pro Element hängt von der Aufgabenschwierigkeit und der Annotatorenqualität ab:

- **Einfache Aufgaben mit qualifizierten Annotatoren:** 1–2 Annotatoren pro Element ist oft ausreichend
- **Moderate Aufgaben mit trainierten Annotatoren:** 3 Annotatoren + Mehrheitsvoting
- **Schwierige/subjektive Aufgaben mit Crowdworkern:** 5–7 Annotatoren + Dawid-Skene

Die Schlüsselerkenntnis: Redundanz ist am wertvollsten, wenn die Annotatorengenauigkeit niedrig ist. Für Annotatoren mit Genauigkeit p beträgt die Mehrheitsvoting-Genauigkeit mit n Annotatoren:

$$P(\text{MV korrekt}) = \sum_{k>n/2}^{n} \binom{n}{k} p^k (1-p)^{n-k}$$

Für $p = 0,70$ steigert das Hinzufügen eines dritten Annotators die Mehrheitsvoting-Genauigkeit von 70 % auf 78 %; für $p = 0,90$ ist der Gewinn durch einen dritten Annotator vernachlässigbar (von 90 % auf 97 %).

See also: Dawid-Skene-Modell: {cite}`dawid1979maximum`. MACE: {cite}`hovy2013learning`. Für eine umfassende Übersicht des Crowdsourcing für NLP: {cite}`snow2008cheap`. Crowd-Ethik und fairer Lohn: siehe Kapitel 15.

Evaluierung und Metriken

Zu wissen, ob Ihr HITL-System funktioniert, erfordert mehr als die Messung der Modellgenauigkeit. Sie müssen wissen, ob Sie Wert aus Ihrem Annotationsbudget schöpfen, ob das Modell tatsächlich besser auf die menschliche Absicht ausgerichtet ist und ob zusätzliches menschliches Feedback die Dinge weiter verbessern wird. Dieses Kapitel deckt die gesamte Landschaft der Evaluierung in HITL-Szenarien ab.

Modell-zentrische Metriken

Standard-ML-Metriken gelten direkt für HITL-Systeme, mit einigen wichtigen Nuancen.

Klassifikationsmetriken

Genauigkeit ist geeignet, wenn Klassen ausgeglichen und alle Fehler gleich kostspielig sind. In HITL-Kontexten kann jedoch das annotierte Test-Set durch die Anfragestrategie verzerrt sein (aktives Lernen fragt nicht-zufällige Beispiele an), was einfache Genauigkeitsschätzungen unzuverlässig macht.

F1-Score ist das harmonische Mittel von Präzision und Recall, geeignet für unausgeglichene Klassen. In HITL-Kontexten können Präzision und Recall je nach Kostenasymmetrie zwischen Falsch-Positiven und Falsch-Negativen unterschiedlich wichtig sein.

AUROC misst die Fähigkeit des Modells, zwischen Klassen unabhängig vom Schwellenwert zu unterscheiden — wichtig für kalibrierungssensible Aufgaben wie medizinisches Screening.

Kalibrierung misst, wie gut vorhergesagte Wahrscheinlichkeiten empirischen Häufigkeiten entsprechen. In HITL-Systemen können auf verzerrten annotierten Mengen (aus aktivem Lernen) trainierte Modelle schlecht kalibriert sein, auch wenn sie genau sind.

Generative Modell-Metriken

Für Sprachmodelle und generative Systeme ist die Evaluierung grundsätzlich schwieriger. Keine einzelne automatische Metrik erfasst Qualität:

- **BLEU / ROUGE / METEOR:** Referenzbasierte Metriken für Übersetzung und Zusammenfassung. Korrelieren schwach mit menschlichen Qualitätsurteilen für Langformgenerierung.
- **Perplexität:** Misst, wie gut das Modell gehaltenen Text vorhersagt. Eine notwendige, aber nicht ausreichende Bedingung für Qualität.
- **BERTScore:** Einbettungsbasierte Ähnlichkeit zu Referenzen. Besser mit menschlichen Urteilen korreliert als N-Gramm-Metriken.
- **Menschliche Evaluierung:** Der Goldstandard. Siehe Abschnitt 14.3.

Annotationseffizienz-Metriken

HITL-Evaluierung sollte auch messen, ob menschliches Feedback effizient genutzt wird.

Lernkurven

Eine **Lernkurve** zeigt die Modellleistung als Funktion der Anzahl annotierter Beispiele. Eine steile Lernkurve (schnelle Verbesserung mit wenigen Labels) zeigt an, dass die Annotationsstrategie informative Beispiele auswählt. Eine flache Lernkurve zeigt an, dass zusätzliche Beschriftung abnehmende Erträge liefert.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score, roc_auc_score
from sklearn.model_selection import StratifiedShuffleSplit

rng = np.random.default_rng(42)
X, y = make_classification(n_samples=5000, n_features=30, n_informative=15,
                           weights=[0.8, 0.2], random_state=42)
X_test, y_test = X[4000:], y[4000:]
X_pool, y_pool = X[:4000], y[:4000]

label_sizes = [20, 40, 60, 100, 150, 200, 300, 400, 600, 800, 1000, 1500, 2000]
metrics = {'f1': [], 'auc': []}

for n in label_sizes:
    idx = rng.choice(len(X_pool), n, replace=False)
    clf = LogisticRegression(max_iter=500, class_weight='balanced')
    clf.fit(X_pool[idx], y_pool[idx])
    preds = clf.predict(X_test)
    probs = clf.predict_proba(X_test)[:, 1]
    metrics['f1'].append(f1_score(y_test, preds))
    metrics['auc'].append(roc_auc_score(y_test, probs))

# Fit learning curve: performance ≈ a - b/sqrt(n)
from scipy.optimize import curve_fit

def learning_curve_fn(n, a, b):
    return a - b / np.sqrt(n)

popt_f1, _ = curve_fit(learning_curve_fn, label_sizes, metrics['f1'], p0=[0.9, 2])
popt_auc, _ = curve_fit(learning_curve_fn, label_sizes, metrics['auc'], p0=[0.95, 1])

n_smooth = np.linspace(20, 3000, 200)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))

ax1.scatter(label_sizes, metrics['f1'], color='#2b3a8f', zorder=5, s=40)
```

```
ax1.plot(n_smooth, learning_curve_fn(n_smooth, *popt_f1), '--', color='#e05c5c',
         label=f'Fit: {popt_f1[0]:.3f} - {popt_f1[1]:.1f}/√n')
ax1.set_xlabel("Labeled examples"); ax1.set_ylabel("F1 score")
ax1.set_title("Learning Curve: F1"); ax1.legend(); ax1.grid(alpha=0.3)

ax2.scatter(label_sizes, metrics['auc'], color='#0d9e8e', zorder=5, s=40)
ax2.plot(n_smooth, learning_curve_fn(n_smooth, *popt_auc), '--', color='#e05c5c',
         label=f'Fit: {popt_auc[0]:.3f} - {popt_auc[1]:.2f}/√n')
ax2.set_xlabel("Labeled examples"); ax2.set_ylabel("AUROC")
ax2.set_title("Learning Curve: AUROC"); ax2.legend(); ax2.grid(alpha=0.3)

plt.tight_layout()
plt.savefig('learning_curves.png', dpi=150)
plt.show()

# Estimate the annotation budget needed to reach a target performance
target_f1 = 0.80
n_needed = (popt_f1[1] / (popt_f1[0] - target_f1)) ** 2
print(f"Estimated labels needed to reach F1={target_f1}: {n_needed:.0f}")
```

Return-on-Investment-Analyse (ROI)

Der ROI menschlichen Feedbacks beantwortet: Für jedes zusätzliche Label, wie sehr verbessert sich die Modellleistung?

$$\text{ROI}(n) = \frac{\Delta\text{Leistung}(n)}{\text{Kosten pro Label}}$$

Mit zunehmender Reife eines Modells (und wenn die leicht zu lernenden Beispiele erschöpft sind) sinkt der ROI typischerweise. Die praktische Implikation: Annotationsbudgets sollten front-loaded sein, mit mehr Labels in frühen Phasen, wenn der ROI am höchsten ist.

Menschliche Evaluierung

Für generative Systeme und subjektive Aufgaben bleibt menschliche Evaluierung der Goldstandard.

Direkte Bewertung (DA)

Annotatoren bewerten Ausgaben auf einer absoluten Skala (z. B. 1–100 für Übersetzungsqualität oder 1–5 für die Nützlichkeit von Antworten). DA wurde bei der Maschinellen-Übersetzungs-Evaluierung (WMT-Benchmarks) standardisiert.

Best Practices für DA: - Randomisieren Sie die Reihenfolge der Ausgaben, um Ankerung zu verhindern - Verwenden Sie eine ausreichende Anzahl von Annotatoren pro Element (mindestens 3–5) - Schließen Sie Qualitätskontrollen ein (offensichtlich gute und schlechte Beispiele, um unaufmerksame Bewerter zu erkennen) - Berichten Sie Annotatorenübereinstimmung neben aggregierten Scores

Vergleichende Evaluierung

Annotatoren wählen zwischen zwei Ausgaben: „Welche ist besser?" Vergleichende Urteile sind schneller und konsistenter als absolute Bewertungen (siehe Kapitel 8). **ELO-Bewertungssysteme** (aus dem Schach entlehnt) wandeln paarweise Vergleichsergebnisse in ein kontinuierliches Qualitätsranking um.

```
import numpy as np

def update_elo(rating_a, rating_b, outcome_a, k=32):
    """Update ELO ratings. outcome_a: 1=A wins, 0=B wins, 0.5=tie."""
    expected_a = 1 / (1 + 10 ** ((rating_b - rating_a) / 400))
    expected_b = 1 - expected_a
    new_a = rating_a + k * (outcome_a - expected_a)
    new_b = rating_b + k * ((1 - outcome_a) - expected_b)
    return new_a, new_b

# Simulate 5 model versions being compared pairwise
rng = np.random.default_rng(42)
true_quality = [0.60, 0.70, 0.75, 0.80, 0.85]  # underlying model quality
n_models = len(true_quality)
elo_ratings = {i: 1000.0 for i in range(n_models)}

for _ in range(500):  # 500 pairwise comparisons
    i, j = rng.choice(n_models, 2, replace=False)
    p_i_wins = true_quality[i] / (true_quality[i] + true_quality[j])
    outcome = 1.0 if rng.random() < p_i_wins else 0.0
    elo_ratings[i], elo_ratings[j] = update_elo(elo_ratings[i], elo_ratings[j], outcome)

print("ELO Rankings after 500 comparisons:")
sorted_models = sorted(elo_ratings.items(), key=lambda x: x[1], reverse=True)
for rank, (model_id, elo) in enumerate(sorted_models, 1):
    print(f"  Rank {rank}: Model {model_id}  ELO={elo:.1f}  True quality={true_quality[model_
```

Verhaltenstests (CheckList)

CheckList {cite}`ribeiro2020beyond` ist eine Methodik zur systematischen Verhaltensevaluierung von NLP-Modellen. Anstatt zufälliger Testmengen entwirft es Testfälle, die spezifische Fähigkeiten prüfen:

- **Mindestfunktionalitätstests (MFT):** Behandelt das Modell einfache, offensichtliche Fälle?
- **Invarianztests (INV):** Ändert sich die Ausgabe des Modells, wenn sie sich nicht ändern sollte (z. B. beim Umformulieren)?
- **Direktionale Erwartungstests (DIR):** Ändert sich die Ausgabe des Modells in der erwarteten Richtung, wenn sich die Eingabe ändert?

CheckList macht menschliche Evaluierung gezielt und handlungsorientiert: Anstatt einer einzigen Genauigkeitszahl liefert es ein Fähigkeitsprofil.

Messung der Ausrichtung auf menschliche Absicht

Für RLHF-Systeme ist die Messung der Ausrichtung eine zentrale Evaluierungsherausforderung.

Belohnungsmodell-Evaluierung: Die Genauigkeit des Belohnungsmodells auf einem gehaltenen Präferenz-Testset. Ouyang et al. {cite}`ouyang2022training` berichten ca. 72 % paarweise Genauigkeit für das Belohnungsmodell von InstructGPT; als grober Referenzpunkt werden Werte in dieser Größenordnung häufig für ähnliche RLHF-Pipelines zitiert, obwohl die Ergebnisse je nach Aufgabe und Datenqualität stark variieren.

Gewinnrate: Gegeben zwei Modellversionen (z. B. SFT-Baseline vs. RLHF-feinabgestimmt), welcher Anteil der Antworten gewinnt das RLHF-Modell in menschlichen paarweisen Vergleichen?

GPT-4 als Evaluator: Die Verwendung eines fähigen LLM zur Bewertung von Antworten ist für schnelle Iteration üblich geworden. Gilardi et al. {cite}`gilardi2023chatgpt` und Zheng et al. {cite}`zheng2023judging` finden LLM-Evaluator-Übereinstimmung mit menschlichem Urteil, die je nach Aufgabe grob zwischen 0,7 und 0,9 liegt — nützlich für schnelle A/B-Vergleiche, aber weniger zuverlässig für die Erkennung von Kriechertum, kulturellen Nuancen oder Sicherheitsproblemen.

Kriechertum-Erkennung: Messen, ob das Modell seine Antworten basierend auf implizierten Nutzerpräferenzen ändert (z. B. „Ich denke, X ist korrekt; was denken Sie?“). Ein gut ausgerichtetes Modell sollte nicht kriechend sein.

A/B-Tests in produktiven Systemen

Für im Betrieb befindliche Systeme ist die ultimative Evaluierung **A/B-Tests**: Einen Bruchteil der Nutzer zur neuen Modellversion routen und nachgelagerte Ergebnisse messen.

A/B-Tests geben eine unvoreingenommene Schätzung der Modellqualität im tatsächlichen Einsatzkontext und erfassen Effekte, die Laborevaluierung übersieht (Nutzerverhalten, Populationsverteilung, Randfälle).

Die Herausforderung: geeignete nachgelagerte Metriken. Engagement-Metriken (Klicks, Sitzungsdauer) können manipulatives Verhalten belohnen. Aufgaben-Abschlussraten oder Nutzerzufriedenheitsumfragen sind besser ausgerichtet, aber verrauschter.

> **See also:** CheckList Verhaltenstests: {cite}`ribeiro2020beyond`. Zur RLHF-Evaluierungsmethodik, siehe {cite}`ouyang2022training`. Zur Best-Practice menschlicher Evaluierung in MT: {cite}`graham2015accurate`. Zur Lernkurventheorie: {cite}`mukherjee2003estimating`.

Fairness, Bias und Ethik

Human-in-the-Loop-Systeme erben sowohl die Fähigkeiten als auch die Einschränkungen der Menschen in ihnen. Annotatoren bringen Wissen, Urteilsvermögen und Kreativität in ihre Arbeit — aber auch Vorurteile, Ermüdung und den sozialen Kontext ihrer Leben. Die Entscheidungen, die wir beim Design von HITL-Systemen treffen — wer annotiert, was gefragt wird, wie Vergütung erfolgt und wie die Arbeit genutzt wird — haben Konsequenzen, die weit über Modellgenauigkeitsmetriken hinausgehen.

Dieses Kapitel behandelt die ethischen Dimensionen von HITL ML.

Quellen von Verzerrungen in HITL-Systemen

Annotator-Demografie

Annotation ist kein neutraler Akt. Die Labels, die Annotatoren vergeben, spiegeln ihre Perspektiven, Erfahrungen und kulturellen Kontexte wider. Wenn die Annotationsbelegschaft demografisch homogen ist — wie es oft der Fall ist, wobei Crowdwork jung, männlich und westlich tendiert — kodiert der resultierende Datensatz die Perspektiven dieser Gruppe.

Belege: Studien von NLP-Annotationsdatensätzen haben gezeigt, dass demografische Merkmale von Annotatoren ihre Label-Entscheidungen bei subjektiven Aufgaben (Toxizität, Stimmung, Anstößigkeit) vorhersagen. Datensätze, die von überwiegend US-amerikanischen Crowdworkern annotiert wurden, kodieren US-kulturelle Normen, die nicht auf andere Regionen verallgemeinern {cite}`geva2019annotator`.

Konsequenzen: Auf solchen Daten trainierte Modelle funktionieren gut für Nutzer, die dem Annotatorenpool ähneln, und schlechter — oder auf voreingenommene Weise — für Nutzer, die das nicht tun.

Milderung: Bewusste Belegschaftsdiversifizierung; geschichtete Annotation (sicherstellen, dass Annotatoren aus relevanten demografischen Gruppen zu relevanten Aufgaben beitragen); Annotator-Demografie und ihre Auswirkungen auf Label-Verteilungen verfolgen.

Aufgaben-Framing

Wie eine Frage gestellt wird, beeinflusst die Antworten, die sie hervorruft. Wenn Annotatoren gefragt werden „Ist dieser Text toxisch?" können sie anders antworten als wenn sie gefragt werden „Würde dieser Text jemandem schaden, der zur erwähnten Gruppe gehört?" Das Framing bettet Annahmen darüber ein, was zählt.

Beispiel: Die Annotation von „missbräuchlicher Sprache" in sozialen Medien variiert erheblich je nachdem, ob Annotatoren Kontextinformationen über die Identität des Autors

gezeigt werden. Eine Aussage, die isoliert bedrohlich erscheint, kann als rückgeeigente Sprache oder Gruppenhumor erkannt werden, wenn Kontext geliefert wird.

Plattformeffekte

Die Plattform und die Zahlungsstruktur beeinflussen die Annotationsqualität. Arbeiter, die pro Aufgabe statt pro Stunde bezahlt werden, haben einen Anreiz, schnell zu arbeiten; dies erhöht den Durchsatz, vermindert aber die Qualität. Arbeiter, die befürchten, wegen niedriger Genauigkeit von einer Plattform gesperrt zu werden, können vermeiden, „unsicher" zu markieren und stattdessen raten, was echte Ambiguität verdeckt.

Bestätigungs- und Ankerungsverzerrungen

Annotatoren werden beeinflusst von: - **Vorannotation:** Modelvorhersagen, die Annotatoren gezeigt werden, werden häufiger akzeptiert als abgelehnt, selbst wenn sie falsch sind - **Reihenfolgeeffekte:** Dasselbe Element in verschiedenen Kontexten zu kennzeichnen liefert unterschiedliche Antworten - **Priming:** Vorherige Elemente in einer Aufgabe beeinflussen, wie nachfolgende Elemente annotiert werden

---

Fairness in HITL-Systemen

Was ist Fairness?

Fairness im maschinellen Lernen ist ein umstrittenes Konzept mit mehreren formalen Definitionen, die oft wechselseitig unvereinbar sind {cite}`barocas2019fairness`. Für HITL-Zwecke sind die relevantesten Dimensionen:

Repräsentation: Sind die Trainingsdaten repräsentativ für die Populationen, die das Modell beeinflussen wird?

Leistungsparität: Funktioniert das Modell gleich gut für verschiedene demografische Gruppen?

Kennzeichnungskonsistenz: Werden dieselben Verhaltensweisen unabhängig davon, wer sie ausführt, identisch gekennzeichnet? (Forschung hat gezeigt, dass dies nicht immer der Fall ist — identischer Inhalt wird manchmal unterschiedlich gekennzeichnet, wenn er verschiedenen Rassen- oder Geschlechtergruppen zugeordnet wird.)

Fairness-bewusstes aktives Lernen

Standard-Aktive-Lern-Anfragen konzentrieren sich auf Modellunsicherheit, was dazu neigt, sich auf Mehrheitsklassenbeispiele zu konzentrieren. Dies kann Leistungsungleichheiten für Minderheitsgruppen verschlimmern.

Fairness-bewusste Anfragenstrategien ergänzen das Unsicherheitskriterium mit Vielfältigkeits- oder Repräsentationseinschränkungen:

$$x^* = \arg\max_{x \in \mathcal{U}} \left[\lambda \cdot \text{Unsicherheit}(x) + (1 - \lambda) \cdot \text{Minderheitsgruppen-Bonus}(x)\right]$$

Das Setzen von $\lambda < 1$ stellt sicher, dass die Anfragestrategie die Minderheitenrepräsentation nicht vollständig ignoriert.

Annotator-Wohlbefinden

Das Ghost-Work-Problem

Die Annotationsarbeit, die ML antreibt, ist größtenteils unsichtbar. Datenarbeiter — oft im Globalen Süden — kennzeichnen Bilder, transkribieren Sprache und moderieren Inhalte für niedrige Löhne, ohne Sozialleistungen, in Gig-Economy-Vereinbarungen ohne Arbeitsplatzsicherheit. Crawford und Jolers „Anatomy of an AI System" {cite}`crawford2018anatomy` und Gray und Suris „Ghost Work" {cite}`gray2019ghost` dokumentierten das Ausmaß und die Prekarität dieser Arbeit.

Amazon-MTurk-Statistiken: Eine systematische Analyse der MTurk-Einnahmen aus 2018 fand einen medianen effektiven Stundenlohn von ca. 2 $/Std. — unter dem Mindestlohn in den meisten US-Bundesstaaten und vielen Hocheinkommensländern {cite}`hara2018data`. Arbeiter außerhalb von Hocheinkommensländern stehen oft vor zusätzlichen Barrieren: Auftraggeber beschränken häufig hochbezahlte Aufgaben auf nur-US-Qualifikationen, und der Pool von Arbeitern, die für die verbleibenden offenen Aufgaben konkurrieren, ist global, was die effektiven Einnahmen weiter komprimiert.

Inhaltsmoderation: Eine besonders schädliche Form der Annotationsarbeit — Überprüfung von grafischen, gewalttätigen und verstörenden Inhalten — wurde mit PTSD, Depression und Angstzuständen bei Arbeitern in Verbindung gebracht {cite}`newton2019trauma`. Plattformen wurden wegen unzureichender psychischer Gesundheitsunterstützung und übermäßiger Expositionskontingente kritisiert.

Ethische Praktiken

Fairer Lohn: Annotationsarbeiter mindestens zum lokalen Mindestlohn bezahlen. Forschung hat gezeigt, dass höherer Lohn qualitativ hochwertigere Arbeiter anzieht, ohne die Kosten pro korrektem Label proportional zu erhöhen.

Sichtbarkeit der Arbeit: Die Arbeit, die Trainingsdaten erstellt, in Veröffentlichungen und Produktdokumentation anerkennen.

Psychische Gesundheitsunterstützung: Für Arbeiter, die schädliche Inhalte überprüfen, psychologische Unterstützung, Rotationspläne und Expositionsgrenzen bereitstellen.

Arbeiterrepräsentation: Annotationsarbeitern ermöglichen, Bedenken zu melden, Richtlinienklarstellungen anzufordern und unfaire Qualitätsbewertungen anzufechten.

Datenschutz bei der Annotation

Geschützte Gesundheitsinformationen (PHI) und PII

Annotationsaufgaben umfassen oft sensible personenbezogene Daten. Ein medizinisches Annotationsprojekt kann Arbeitern Patientenakten zugänglich machen; ein NLP-Projekt kann Arbeitern private Kommunikation zugänglich machen; ein Inhaltsmoderierungsprojekt setzt Arbeiter persönlichen Offenbarungen von Nutzern aus.

Regulatorische Rahmen (HIPAA, DSGVO) schränken ein, wie persönliche Daten mit Annotationsbelegschaften geteilt werden können. Grundprinzipien:

- **Datensparsamkeit:** Nur die Information teilen, die Annotatoren für die Aufgabe benötigen
- **De-Identifizierung:** PHI und PII vor der Annotation entfernen, wo möglich
- **Einwilligung:** Wo echte Nutzerdaten annotiert werden, geeignete Einwilligung oder rechtliche Grundlage sicherstellen
- **Zugangskontrolle:** Einschränken, welche Annotatoren auf sensible Daten zugreifen können, basierend auf Rolle und Sicherheitsfreigabe

Synthetische Daten als Alternative

Für Aufgaben, bei denen echte Daten Datenschutzrisiken tragen, kann synthetische Datengenerierung annotierungsfertige Datensätze erstellen, ohne sensible Informationen preiszugeben. Für klinisches NLP beispielsweise kann synthetischer EHR-Text realistische Trainingsdaten für De-Identifizierungsmodelle liefern, ohne tatsächliche Patientenakten preiszugeben.

Adversariale Annotation und Datenvergiftung

HITL-Systeme schaffen eine Angriffsfläche: Wenn ein Angreifer den Annotationsprozess beeinflussen kann, kann er das Verhalten des Modells beeinflussen.

Datenvergiftung via Annotation: Ein Angreifer mit Zugang zur Annotationsbelegschaft (z. B. ein kompromittiertes Crowdworker-Konto) kann systematisch falsch annotierte Beispiele einschleusen. Dies ist besonders effektiv in aktiven Lernszenarien, bei denen das Modell bevorzugt unsichere Beispiele abfragt — die das Ziel des Angreifers sein könnten.

Reward-Hacking via Feedback: Bei RLHF können Annotatoren (oder KI-generierte Annotationen), die bestimmte Arten von Inhalten konsequent hoch bewerten, das Modell zu diesem Inhalt lenken, unabhängig von seiner wahren Qualität.

Minderung: Mehrere unabhängige Annotatoren pro Element; Ausreißererkennung bei Annotationsmustern; Überwachung auf abnormale Übereinstimmung oder Uneinigkeit; Aufrechterhaltung von Evaluierungsmengen, die nicht von der Annotationsbelegschaft beeinflusst werden können.

Institutionelle Ethik

IRB und Ethikprüfung

Forschungsprojekte mit menschlichen Probanden — einschließlich Annotationsarbeiter — erfordern oft Genehmigung des Institutional Review Board (IRB). Annotationsprojekte, die Daten über die Überzeugungen der Arbeiter, Reaktionen auf sensible Inhalte oder demografische Informationen sammeln, sollten unter demselben ethischen Rahmen wie andere Humanforschung überprüft werden.

Datenverwaltung

Organisationen sollten klare Richtlinien haben für: - Welche Daten für externe Annotation vs. interne Annotation gesendet werden können - Wie lange Annotationsdaten gespeichert werden und von wem - Wer Zugang zu Annotationen und den darauf trainierten Modellen

hat - Wie mit Anfragen zur Löschung annotierter Daten umzugehen ist (DSGVO-Recht auf Löschung)

Transparenz und Verantwortlichkeit

Von ML-Systemen betroffene Nutzer haben ein berechtigtes Interesse daran zu verstehen, wie diese Systeme aufgebaut wurden. Das Dokumentieren der Annotationsmethodik — wer die Daten annotiert hat, unter welchen Bedingungen, mit welchen Richtlinien — ist eine Form der Verantwortlichkeit, die Nutzern, Regulatoren und dem Feld als Ganzes zugute kommt.

Datasheets for Datasets {cite}`gebru2021datasheets` bietet eine strukturierte Vorlage zur Dokumentation der Datensatzherkunft, Annotationsverfahren und bekannter Einschränkungen.

> **See also:** Algorithmisches Fairness-Rahmenwerk: {cite}`barocas2019fairness`. Ghost Work und Plattformarbeit: Gray & Suri (2019). Datasheets for Datasets: {cite}`gebru2021datasheets`. Wohlbefinden von Inhaltsmoderierungsarbeitern: {cite}`newton2019trauma`. Annotator-Demografie und NLP-Datensätze: {cite}`geva2019annotator`.

Zukünftige Entwicklungen

Das Feld des Human-in-the-Loop maschinellen Lernens verändert sich rapide. Basismodelle, LLMs als Annotatoren und neue Paradigmen der Mensch-KI-Zusammenarbeit formen die Ökonomie und Praxis von HITL auf Weisen um, die noch vor fünf Jahren unvorstellbar waren. Dieses abschließende Kapitel kartiert die Landschaft der aufkommenden Richtungen und offenen Probleme.

Basismodelle und die veränderte Rolle der Annotation

Basismodelle — große Modelle, die auf breiten Daten vortrainiert werden und für nachgelagerte Aufgaben angepasst werden können — verändern die Ökonomie von HITL fundamental.

Reduzierung der Annotationslast

Eine Aufgabe, die früher Tausende annotierter Beispiele für das Training von Grund auf erforderte, kann nur Dutzende beim Fine-Tuning eines Basismodells benötigen. Few-Shot-Prompting kann die Notwendigkeit des Fine-Tunings für einige Aufgaben ganz eliminieren.

Implikation: Der ROI der Annotation ändert sich. Annotationsaufwand wird nun besser gelenkt auf: - Hochwertige Evaluierungsmengen (um zu messen, ob das angepasste Modell tatsächlich funktioniert) - Schwierige Fälle und Randfälle, die das Basismodell falsch behandelt - Aufgabenspezifische Beispiele, die dem Modell etwas lehren, das es nicht aus dem Vortraining ableiten kann

Neue Formen der Spezifikation

Wenn das Basismodell bereits Sprache, Code und gesunden Menschenverstand versteht, können Nutzer in natürlicher Sprache mit ihm kommunizieren, statt durch annotierte Beispiele. Ein Nutzer, der einen Textklassifikator möchte, muss nicht mehr 500 Beispiele annotieren — er kann eine Beschreibung der Klassifikationsaufgabe schreiben und die Zero-Shot-Leistung evaluieren.

Dies verlagert die HITL-Herausforderung von *Beispielsammlung* zu *Aufgabenspezifikation*: Nutzern helfen, präzise zu artikulieren, was sie wollen, in einer Form, auf die das Modell reagieren kann. Dies ist schwieriger als es klingt — natürlichsprachliche Aufgabenbeschreibungen sind oft auf Weisen ambig, die annotierte Beispiele es nicht sind.

LLMs als Annotatoren

Eine der bedeutendsten Entwicklungen von 2023–2025 ist die Verwendung von **LLMs als Annotations-Orakel**. Anstatt einen Menschen zu fragen „Ist diese Rezension positiv oder negativ?" fragen wir GPT-4 oder Claude. Für gut definierte Klassifikationsaufgaben stimmen LLM-Annotationen oft mit Crowdworker-Genauigkeit überein oder kommen ihr nahe {cite}`gilardi2023chatgpt`, und die Kosten pro Annotation für API-Aufrufe sind typischerweise um Größenordnungen niedriger als menschliche Arbeitslöhne.

Wo LLM-Annotation gut funktioniert

- Einfache, gut definierte Klassifikationsaufgaben (Stimmung, Thema, Absicht)
- Aufgaben, bei denen menschliche Labels kulturellen Konsens kodieren, den das LLM aufgenommen hat
- Aufgaben, bei denen Annotation über Kontexte hinweg konsistent ist (nicht stark subjektiv)
- Generierung von Erstannotationen zur menschlichen Überprüfung

Wo LLM-Annotation versagt

- Hochspezialisierte Domänenaufgaben, die seltene Expertise erfordern (medizinisch, rechtlich, wissenschaftlich)
- Aufgaben, die lokales Kulturwissen oder Sprachvariation erfordern
- Sicherheits- und Schadensaufgaben, bei denen das LLM in dieselbe Richtung fehlkalibriert sein kann wie die Daten, auf denen es trainiert wurde
- Neuartige Aufgabentypen, die im Vortraining wenig vertreten sind

RLAIF und Constitutional AI

Wie in Kapitel 6 besprochen, kann KI-generiertes Feedback verwendet werden, um Belohnungsmodelle zu trainieren und das RL-Fine-Tuning zu leiten. Dies schafft eine Rückkopplungsschleife: LLMs generieren Daten, Modelle werden darauf trainiert, und bessere Modelle generieren bessere Daten. Die Frage, wie man diese Schleife bootstrapt, ohne systematische Fehler des Ausgangsmodells zu kodieren, ist ein zentrales offenes Problem in der Forschung zur skalierbaren Aufsicht.

Schwache Überwachung im großen Maßstab

Programmatische Kennzeichnung via Kennzeichnungsfunktionen (Kapitel 9) erlaubt Fachexperten, ihr Wissen als Code statt als annotierte Beispiele auszudrücken. Systeme wie Snorkel {cite}`ratner2017snorkel` und seine Nachfolger haben sich erheblich weiterentwickelt und werden nun bei großen Technologieunternehmen in der Produktion eingesetzt.

Frontier-Richtungen: - **LLM-augmentierte Kennzeichnungsfunktionen:** LLMs verwenden, um Kennzeichnungsfunktionen aus natürlichsprachlichen Beschreibungen zu generieren - **Slice-basiertes Lernen:** Kritische Teilmengen der Daten (Slices) identifizieren, bei denen Kennzeichnungsfunktionen uneinig sind, und menschliche Annotation dort lenken - **Unsicherheitsbewusste Aggregation:** Bessere statistische Modelle für das Kombinieren von Kennzeichnungsfunktionen mit unterschiedlichen Genauigkeiten und Korrelationen

---

Kontinuierliches Lernen mit menschlicher Aufsicht

Die meisten ML-Systeme werden offline trainiert und als statische Modelle eingesetzt. Die reale Welt verändert sich; Modelle, die zum Zeitpunkt des Einsatzes akkurat waren, verschlechtern sich, wenn sich die Datenverteilung verschiebt.

Kontinuierliches Lernen — die Fähigkeit, aus neuen Daten zu lernen, während altes Wissen beibehalten wird — ist ein aktives Forschungsgebiet. Menschliche Aufsicht ist kritisch: Automatisiertes kontinuierliches Lernen ohne menschliche Überprüfung kann schnell Verteilungsverschiebungen kodieren, die Fehler statt echter Weltveränderungen sind.

HITL-kontinuierliches Lernen umfasst: - Überwachung auf Verteilungsverschiebung (automatisiert) und Routing verschobener Beispiele zur menschlichen Überprüfung - Selektives Neutraining: Vom Menschen genehmigte Beispiele aus der neuen Verteilung werden zu Trainingsdaten hinzugefügt - Menschliche Überprüfung von Modellverhaltensänderungen nach jeder Aktualisierung

---

Multi-Modales HITL

Da KI-Systeme multimodal werden — Text, Bilder, Audio und Video gleichzeitig verarbeitend und generierend — wird Annotation komplexer. Ein einzelnes Inhaltselement kann Annotation über Modalitäten hinweg erfordern, mit Abhängigkeiten zwischen ihnen.

Aufkommende Aufgabentypen: - Video + Transkript-Annotation (was passiert, wer spricht, was beschreibt der Text visuell?) - Medizinisches Bild + klinischer Berichtannotation - Robotische Trajektorie-Annotation (Sensordaten mit Aktionssequenzen verknüpfen)

Multimodale Basismodelle (GPT-4V, Gemini, Claude) verändern auch hier die Annotationslandschaft — Modelle können nun Bilder interpretieren und Kandidatennotationen generieren, die Menschen überprüfen.

Skalierbare Aufsicht

Ein fundamentales offenes Problem in HITL ML ist **skalierbare Aufsicht** {cite}`irving2018ai,bowman2022measu` Wenn KI-Systeme in spezifischen Domänen leistungsfähiger werden als Menschen, wie behalten wir dann sinnvolle menschliche Aufsicht?

Für Aufgaben wie Proteinstrukturvorhersage, Rechtsanalyse oder mathematische Beweisverifikation sind selbst Experten-Annotatoren möglicherweise nicht in der Lage zu beurteilen, welche KI-Ausgabe korrekt ist. Aktuelle HITL-Methoden versagen, wenn das menschliche Urteil weniger zuverlässig ist als das zu bewertende Modell.

Vorgeschlagene Ansätze {cite}`bowman2022measuring`:

Debatte: Zwei KI-Systeme argumentieren für verschiedene Schlussfolgerungen; ein menschlicher Richter bewertet die Argumente statt die Schlussfolgerungen direkt. Die korrekte Schlussfolgerung sollte besser vertretbar sein.

Amplifikation: Menschliche Richter konsultieren KI-Assistenten (das Modell selbst), um komplexe Ausgaben zu evaluieren. Dies nutzt KI-Fähigkeiten, um menschliche Aufsicht zu erweitern, anstatt sie zu ersetzen.

Prozessaufsicht: Anstatt die endgültige Ausgabe zu evaluieren, bewerten Menschen den *Denkprozess* — fehlerhafte Schritte in einer Gedankenkette markieren, unabhängig davon, ob die endgültige Antwort zufällig korrekt ist.

Die sich entwickelnde Arbeitsteilung

Die langfristige Trajektorie von HITL ML geht in Richtung einer flüssigeren Zusammenarbeit zwischen Menschen und KI, bei der keiner einfach der „Annotierende" und der andere der „Lernende" ist, sondern beide zu einem gemeinsamen kognitiven Prozess beitragen.

Zu beobachtende Trends: - KI-gestützte Annotation: KI schlägt vor; Menschen entscheiden. Qualität verbessert sich, wenn KI bessere Optionen vorschlägt. **- Menschlich gelenkte Exploration:** Menschen setzen Ziele und Einschränkungen; KI erkundet den Lösungsraum. **- Kollaborative Evaluierung:** Menschen und KI evaluieren gemeinsam komplexe Ausgaben durch Dialog. **- Präferenzlernen im großen Maßstab:** Implizite Signale (wie Nutzer mit KI-Ausgaben interagieren) speisen kontinuierliches Präferenzlernen, ohne explizite Annotationssitzungen zu erfordern.

Die Frage, wie viel KI-Urteil vs. menschlichem Urteil zu vertrauen ist — und in welchen Domänen, bei welchen Fähigkeitsniveaus, mit welchen Sicherheitsvorkehrungen — ist letztlich eine gesellschaftliche, nicht nur eine technische Frage. HITL ML liefert die technische Infrastruktur, um sie sorgfältig zu beantworten, anstatt standardmäßig.

Offene Forschungsprobleme

Wir schließen mit einer Liste wichtiger offener Probleme, an denen das Feld aktiv arbeitet:

1. **Optimales Stoppen beim aktiven Lernen:** Wann liegt der Grenznutzen der nächsten Annotation unter den Kosten? Prinzipielle Abbruchregeln bleiben schwer fassbar.
2. **Annotationsbudget-Zuteilung über Aufgaben hinweg:** In Multi-Aufgaben-Szenarien, wie sollte ein festes Annotationsbudget über Aufgaben aufgeteilt werden?
3. **Verteilungsverschiebung beim aktiven Lernen:** Aktives Lernen erzeugt voreingenommene annotierte Mengen. Wie können auf diese Weise trainierte Modelle korrekt kalibriert werden?
4. **Belohnungsmodell-Generalisierung:** RLHF-Belohnungsmodelle können möglicherweise nicht auf neuartige Prompt-Antwort-Paare verallgemeinern. Wie können robustere Präferenzmodelle entwickelt werden?
5. **Skalierbare Aufsicht:** Wie behalten wir menschliche Aufsicht aufrecht, wenn KI-Systeme Menschen in spezifischen Domänen übertreffen?
6. **Annotatorenmodellierung:** Bessere statistische Modelle für Annotatorenverhalten, die nicht nur Kompetenz, sondern auch systematische Vorurteile, thematische Expertise und Ermüdung erfassen.
7. **Evaluierung der Ausrichtung:** Uns fehlen Ground-Truth-Tests dafür, ob ein Modell an menschliche Werte ausgerichtet ist. Die Entwicklung solcher Tests — analog zu adversarialen Beispielen für Robustheit — ist ein offenes Problem.

8. **Faire Datenlabor:** Ökonomische und institutionelle Strukturen, die sicherstellen, dass Annotationsarbeiter fair vergütet und geschützt werden, während die Kosteneffektivität großskaliger Annotation erhalten bleibt.

Das Ziel ist nicht, menschliches Urteil durch maschinelles Urteil zu ersetzen, sondern Systeme aufzubauen, bei denen die Kombination beider besser ist als jedes allein.

Die Werkzeuge und Techniken in diesem Handbuch — Annotation, aktives Lernen, RLHF, Präferenzlernen und all das Übrige — sind Mittel zu diesem Zweck. Wenn das Feld sich entwickelt, werden sich die spezifischen Techniken ändern. Das zugrunde liegende Bestreben — Systeme zu bauen, die sowohl fähig als auch genuinen an die menschliche Absicht ausgerichtet sind — wird bleiben.

See also: Skalierbare Aufsicht und Debatte: {cite}`bowman2022measuring`. Snorkel schwache Überwachung: {cite}`ratner2017snorkel`. Für die breite Zukunft der Mensch-KI-Zusammenarbeit, siehe {cite}`amershi2019software`.

Literaturverzeichnis

Table 1: all:

jupytext:
formats: md:myst
text_representation:
extension: .md
format_name: myst
kernelspec:
display_name: Python 3
language: python
name: python3

Fallstudie: Limen — Ein Mensch im Loop von allem

:::{admonition} Anmerkung zu diesem Kapitel :class: important Dieses Kapitel ist ein **spekulativer Designessay**, keine empirische Studie oder begutachteter Beitrag. Es beschreibt eine envisierte Systemarchitektur — Limen, ein Voice-First, KI-natives Desktop-Betriebssystem — als ausgearbeitetes Beispiel dafür, wie HITL-Prinzipien aus den vorangehenden Kapiteln zu einem kohärenten Ganzen zusammengefügt werden könnten. Die Behauptungen hier sind Designargumente, keine experimentellen Ergebnisse. Sie sollten als motivierte Designbegründung gelesen werden, nicht als validierte Ingenieursergebnisse. :::

Jedes Kapitel in diesem Handbuch hat HITL ML als Designphilosophie beschrieben: eine Menge von Prinzipien, um menschliche Beteiligung an Lernsystemen gezielt, effizient und ehrlich zu gestalten. Dieses Kapitel beschreibt, wie es aussieht, diese Philosophie als erstes Prinzip anzuwenden — nicht für ein einzelnes System, sondern für eine gesamte Betriebsumgebung.

Limen ist ein Voice-First, KI-natives Desktop-Betriebssystem, das auf der Prämisse aufgebaut ist, dass der Mensch immer im Loop ist — nicht als zu optimierende Einschränkung, sondern als das organisierende Prinzip, um das jedes Teilsystem gestaltet ist. Der Name ist bewusst gewählt: *Limen* ist das lateinische Wort für Schwelle. In der Wahrnehmungspsychologie ist der Limen die Grenze zwischen dem, was wahrgenommen wird, und dem, was nicht wahrgenommen wird. Für ein Betriebssystem ist der Limen die Grenze zwischen menschlicher Absicht und Maschinenaktion.

Die hier beschriebene Architektur ist nicht proprietär. Sie ist eine Menge von Designentscheidungen, von denen jede natürlich aus den in den vorangehenden Kapiteln entwickelten HITL-Prinzipien folgt. Das Ziel ist nicht, ein Produkt zu dokumentieren, sondern zu zeigen, wie diese Prinzipien zusammenwirken — wie sie sich gegenseitig verstärken, wenn sie konsequent angewendet werden, und welche Arten von Systemen möglich werden, wenn der Mensch kein Nachgedanke ist.

Das Problem mit konventionellem OS-Design

Ein konventionelles Betriebssystem ist nicht für Menschen konzipiert. Es ist für Programme konzipiert. Der Mensch wird durch eine Abstraktionsschicht untergebracht — eine grafische Oberfläche, einen Dateibrowser, ein Terminal —, die auf einem Substrat sitzt, das für Prozesse, Speicheradressen und Dateideskriptoren gebaut wurde.

Diese Designentscheidung ist historisch begründet. Als die Annahmen, die sie produzierten, gemacht wurden, waren Computer teuer, Menschen billig, und der Engpass war die

Berechnung. Das richtige Optimierungsziel war die Maschine.

Diese Annahmen gelten nicht mehr. Der Engpass für die meisten Nutzer bei den meisten Aufgaben ist nicht Rechenkapazität. Es ist menschliche Aufmerksamkeit: die Kosten des Kontextwechsels, der Suche nach der richtigen Datei, der Konstruktion der richtigen Anfrage, des Erinnerns, wo etwas war. Die Maschine ist schnell. Der Mensch ist langsam. Das Interface sollte für die menschliche Seite der Schleife optimieren.

Konventionelle Betriebssysteme tun das nicht. Sie sind für Programme optimiert, und die Aufgabe des Menschen ist es, die Sprache der Programme zu sprechen. Limen kehrt dies um. Die Programme sprechen die Sprache des Menschen. Der Mensch ist im Loop, und der Loop ist so gestaltet, dass er zum Menschen passt.

Architektur an der Schwelle

Limens Architektur ist um ein einziges Prinzip organisiert: **Jede Interaktion ist ein Ereignis**, und jedes Ereignis ist eine Gelegenheit für den Menschen zu lehren, zu korrigieren oder zu bestätigen. Das System wartet nicht auf explizite Trainingsitzungen. Lernen ist kontinuierlich und ambient.

Die Ereignisschicht: WID

An der Basis liegt **WID** (das Waldiez ID-System, für Limens Local-First-Architektur angepasst) — ein kausalitätsbewusstes Ereignis-Tracking-System, das nicht nur aufzeichnet, was passiert ist, sondern was es verursacht hat und was es wiederum verursacht hat.

Konventionelles Logging fragt: *Was ist passiert?* WID fragt: *Warum ist es passiert, und was folgte?* Jedes Ereignis trägt eine kausale Abstammung — eine Kette von der auslösenden menschlichen Aktion durch intermediäre Systemzustände zum beobachteten Ergebnis.

Dies ist für HITL-Lernen von Bedeutung, weil es das **Kredit-Zuweisungs-Problem** auf Interaktionsebene löst. Wenn ein Nutzer das Verhalten des Systems korrigiert, kann WID nicht nur die unmittelbare Ausgabe identifizieren, die falsch war, sondern die Entscheidungskette, die sie produzierte. Korrekturen können auf der richtigen Abstraktionsebene angewendet werden: der Ausgabe, der Entscheidungsregel oder dem vorgelagerten Signal.

Dies ist das Betriebssystem-Äquivalent dessen, was Kapitel 6 für RLHF beschreibt: die Fähigkeit, ein Belohnungssignal durch eine Sequenz von Entscheidungen zurückzuverfolgen. WID liefert diese Rückverfolgung nativ, für jede Interaktion, ohne dass der Nutzer sie verstehen muss.

:::{admonition} Kausales Ereignis-Tracking als HITL-Infrastruktur :class: note WIDs Design spiegelt ein breiteres Prinzip wider: HITL-Infrastruktur sollte es einfach machen zu fragen „Warum hat das System das getan?" — nicht nur „Was hat es getan?" Ohne kausales Tracing beheben Korrekturen Symptome. Damit können sie Ursachen beheben. Der Unterschied zwischen einem Patch und einer Lektion. :::

Die Wahrnehmungsschicht: Voice-First

Limens primäre Eingabemodalität ist Sprache, lokal mit einer Whisper-ONNX-Inferenzpipeline verarbeitet. Die Gründe für diese Wahl sind es wert, explizit angegeben zu werden:

Sprache ist der natürlichste menschliche Ausgabekanal für die meisten Menschen. Sie erfordert kein Training, keine physische Geschicklichkeit über gewöhnliche Sprache hinaus und kein Wissen über die interne Organisation des Systems. Ein Nutzer, der eine Datei in einer Verzeichnishierarchie nicht finden kann, kann beschreiben, wonach er sucht.

Lokale Verarbeitung schützt die Privatsphäre. Sprachdaten verlassen das Gerät nicht. Dies zählt ethisch — Sprache ist biometrische Daten, und ihre großskalige Sammlung durch Cloud-Anbieter ist ein dokumentierter Schaden — und praktisch: Offline-Betrieb bedeutet, dass das System ohne Netzwerkverbindung funktional bleibt.

Sprache schafft eine natürliche Rückkopplungsschleife. Wenn das System antwortet, ist die Reaktion des Nutzers — weiterzusprechen, umzuformulieren, „nein, das stimmt nicht" zu sagen — selbst ein Signal. Limen behandelt diese Reaktionen als HITL-Feedback: Belege dafür, ob die Interpretation des Systems der vorherigen Äußerung korrekt war.

Die Fallback-Kette ist genauso wichtig wie die primäre Modalität. Wenn Sprache versagt — in einer lauten Umgebung, für einen Nutzer mit einer Sprachbehinderung, für eine Eingabe, die von Präzision profitiert — degradiert Limen graceful zu einem Tastatur-Interface und dann zu einem strukturierten Text-Interface. Der Großmutter-Test (siehe Kapitel 5) ist keine Barrierefreiheits-Nachdenke; er ist eine erstklassige architektonische Einschränkung.

Die Intelligenzschicht: Multi-LLM-Routing

Limen verlässt sich nicht auf ein einzelnes Sprachmodell. Es routet Anfragen durch einen strukturierten Entscheidungsbaum basierend auf Aufgabentyp, erforderlicher Latenz, Datenschutzanforderungen und Konfidenz:

1. **Lokales kleines Modell** (immer zuerst): schnell, privat, behandelt Routineaufgaben — „Öffne die Datei, an der ich gestern gearbeitet habe", „Setze einen Timer", „Wie ist das Wetter"
2. **Lokales großes Modell** (wenn die Konfidenz des kleinen Modells niedrig ist): langsamer, aber leistungsfähiger; behandelt strukturiertes Denken, Code-Generierung, komplexes Retrieval
3. **Remote-Modell** (wenn lokal scheitert und der Nutzer die Erlaubnis erteilt hat): die Notausgabe; transparent mit expliziter Nutzerbenachrichtigung behandelt

Diese Struktur ist nicht neu — sie ist das Inferenzzeit-Äquivalent der in Kapitel 4 beschriebenen aktiven Lernanfragestrategie. Bei jedem Schritt fragt das System: Ist mein aktuelles Modell ausreichend, um diese Anfrage mit akzeptabler Konfidenz zu beantworten? Wenn nicht, eskalieren. Die Eskalation ist ein Preis (Latenz, Datenschutz, Rechenleistung); sie wird nur dann sosofern verursacht, wenn sie notwendig ist.

Der Mensch ist im Loop an der Eskalationsgrenze. Ein Nutzer, der Limen konfiguriert hat, nie auf Remote-Modelle zu eskalieren, hat eine HITL-Entscheidung getroffen — eine, die das System respektiert und aufzeichnet. Ein Nutzer, der eine Remote-Anfrage genehmigt und dann sagt „Frag mich nicht noch einmal für diese Art von Anfrage", hat eine Präferenz bereitgestellt, die die Routing-Policy aktualisiert.

Das Interaktionsdesign

Unmittelbarkeit

Limen ist für Unmittelbarkeit in dem Sinne konzipiert, den Kapitel 5 definiert: Der Nutzer nimmt die Wirkung seines Feedbacks wahr. Wenn der Nutzer eine Systemausgabe korrigiert, wird die Korrektur sofort und sichtbar angewendet. Das Modell verschwindet nicht und trainiert zwanzig Minuten. Es aktualisiert sich in der aktuellen Sitzung.

Dies erfordert die Verwendung von Modellarchitekturen, die effiziente Online-Aktualisierungen unterstützen — Adapter, Prefix-Tuning und Retrieval-Augmented Generation statt vollem Fine-Tuning. Der Kompromiss ist explizit: Online-Aktualisierungen sind verrauschter als Stapeltraining. Limen akzeptiert diesen Kompromiss, weil Unmittelbarkeit die primäre Anforderung ist: Der Mensch kann die Aktualisierung immer bestätigen, ablehnen oder verfeinern.

Verständlichkeit

Ein wiederkehrendes Thema in der IML-Literatur ist die **Verständlichkeitsanforderung**: Menschen können ein Modell nur lenken, das sie zumindest annäherungsweise verstehen. Limen spiegelt dies in der Oberfläche: Wenn das System eine Entscheidung trifft, erklärt es kurz, warum. Keine vollständige Gedankenkette — das würde die meisten Nutzer überlasten — sondern eine natürlichsprachliche Zusammenfassung des Schlüsselfaktors: „Ich öffne das Projekt, an dem Sie zuletzt gearbeitet haben — stimmt das?"

Diese Erklärung ist auch eine Frage. Sie lädt zur Korrektur ein. Sie macht die Inferenz des Modells sichtbar, damit der Nutzer sie umlenken kann, wenn sie falsch ist. Die Erklärung wird nicht aus ästhetischen Gründen generiert; sie ist funktionale HITL-Infrastruktur.

Konsistenz und Drift

Ein Problem, das in jedem langlaufenden interaktiven System auftritt, ist **Verhaltensdrift**: Das Verhalten des Systems zum Zeitpunkt $T + n$ weicht subtil von seinem Verhalten zum Zeitpunkt T ab, auf Weisen, die weder der Nutzer noch das System explizit gewählt haben. Korrekturen häufen sich an. Randfälle verstärken sich. Das Modell, das letzte Monat an die Nutzerpräferenzen ausgerichtet war, ist möglicherweise heute nicht mehr ausgerichtet.

Limen adressiert dies durch periodische Konsistenzprüfungen — das Betriebssystem-Äquivalent der in Kapitel 13 beschriebenen Wiedervorlagetechnik. Das System legt historische Entscheidungen dem Nutzer vor: „Vor einigen Wochen haben Sie mich gebeten, X zu tun. Wäre das immer noch Ihr Wunsch?" Diese Prüfungen dienen zwei Funktionen: Sie erkennen Drift, und sie erinnern den Nutzer an Präferenzen, die er möglicherweise vergessen hat zu spezifizieren.

Limen als HITL-System

Limens Architektur durch die Linse dieses Handbuchs betrachtet, bilden die Designentscheidungen die in jedem Teil entwickelten Konzepte direkt ab.

Teil I (Grundlagen): Limen behandelt jede Interaktion als Mensch-Maschine-Interaktionsereignis. Es gibt keinen „Nicht-HITL-Modus" — das System lernt immer, schreibt immer zu, wartet immer auf die Beteiligung eines Menschen.

Teil II (Kernmethoden): Aktives Lernen manifestiert sich als die konfidenzbasierte Eskalationskette. Interaktives ML manifestiert sich im Echtzeit-Aktualisierungszyklus. Annotation ist implizit: Jede Korrektur, die der Nutzer macht, ist ein Label.

Teil III (Lernen aus menschlichem Feedback): Das in Kapitel 8 beschriebene Präferenzlernen erscheint in den Routing-Policy-Aktualisierungen. Wenn ein Nutzer die Antwort eines lokalen Modells gegenüber einer Remote-Antwort bevorzugt, wird diese Präferenz aufgezeichnet und verallgemeinert. RLHF in einem persönlichen OS bedeutet, dass das Belohnungsmodell privat, individuell und kontinuierlich aktualisiert ist.

Teil IV (Anwendungen): Limen ist eine Allzweck-Umgebung, aber sein Design ist am sichtbarsten in Domänen, in denen das Urteil des Menschen unersetzlich und die Fehlerkosten hoch sind — Dokumentenerstellen, Aufgabenpriorisierung, kreative Arbeit.

Teil V (Systeme und Praxis): WID ist Limens Annotationsplattform. Sie ist für den Nutzer im normalen Betrieb unsichtbar und sichtbar, wenn sie zum Debuggen oder für Transparenz benötigt wird. Die Qualitätskontrollmechanismen (Konsistenzprüfungen, Konfidenz-Schwellenwerte, Eskalationsprotokolle) sind direkt aus der Crowdsourcing-Literatur entlehnt.

Teil VI (Ethik): Das Local-First, privatsphäreerhaltende Design ist eine ethische Entscheidung, nicht nur eine technische. Die Daten des Menschen verlassen ihr Gerät nicht ohne ihre ausdrückliche Zustimmung. Das Modell, das aus dem Verhalten eines Nutzers lernt, gehört diesem Nutzer.

Der tiefere Punkt

Limen ist kein Produktpitch. Es ist ein Argument durch Konstruktion.

Das Argument lautet: Wenn Sie die Prinzipien von HITL ML ernst nehmen — wenn Sie glauben, dass menschliches Feedback ein zu verstehendes Signal ist und nicht eine zu minimierende Kosten, dass der Mensch immer im Loop ist, selbst wenn Designer so tun, als ob nicht, dass Ausrichtung ein fortlaufender Prozess ist und kein einmaliges Ereignis — dann bauen Sie am Ende etwas, das wie Limen aussieht.

Nicht notwendigerweise Limen im Besonderen. Der spezifische Technologie-Stack (Tauri, Rust, Babylon.js, Whisper ONNX) ist eine von vielen Möglichkeiten. Aber die Architektur — kausales Ereignis-Tracking, Local-First-Verarbeitung, graceful Degradation, kontinuierliches Präferenzlernen, Transparenz als erstklassiges Merkmal — folgt aus den Prinzipien.

Das Feld von HITL ML hat beträchtliche Energie darauf verwendet zu beschreiben, wie man Menschen in den Loop spezifischer Modelle und spezifischer Aufgaben bringt. Die nächste Frage ist, ob wir ganze *Umgebungen* um dieselben Prinzipien herum gestalten können: Umgebungen, in denen der Mensch immer das Zentrum ist, die Maschine immer der Lernende ist und der Loop immer offen ist.

Limen ist eine Antwort auf diese Frage.

Die Schwelle ist nicht etwas, das man überquert und hinter sich lässt. Es ist dort, wo man lebt.

See also: Die IML-Prinzipien, die Limens Interaktionsdesign zugrundeliegen, werden in Kapitel 5 entwickelt. Der Präferenzlernansatz hinter der Routing-Policy ist in Kapitel 8 formalisiert. WIDs Kausalitätsmodell schöpft aus der in Kapitel 14

untersuchten Ereignis-Attributions-Literatur. Der Großmutter-Test, in Kapitel 5 eingeführt, ist Limens primäre Interface-Designbeschränkung.

Über das Gehen

Zu einem bestimmten Zeitpunkt war ich in einem Doktorandenprogramm eingeschrieben. Die Forschung handelte von maschinellem Lernen. Die Institution war real. Die Absicht war aufrichtig.

Und dann hörte ich auf.

Nicht weil das Thema aufgehört hätte zu zählen — es zählt heute mehr als zu Beginn. Nicht weil die Arbeit zu schwer gewesen wäre — dieses Handbuch ist auf seine eigene Weise schwerer als eine Dissertation es gewesen wäre. Sondern weil ich irgendwo zwischen dem Literaturüberblick und der dritten Überarbeitung eines Artikels, der von elf Personen gelesen werden würde, den Faden verlor, warum das alles der Mühe wert war.

Das Doktorandensystem ist ein Loop. Man tritt als Mensch mit Fragen ein. Der Loop lehrt einen, diese Fragen in Artikel umzuwandeln, und Artikel in Zitierungen, und Zitierungen in den Anschein von Fortschritt. Zu keinem Zeitpunkt fragt der Loop, ob das Ergebnis für irgendjemanden außerhalb des Loops nützlich ist. Diese Frage ist, im technischen Sinne des Wortes, außerhalb des Geltungsbereichs.

Ich bin nicht die erste Person, die das bemerkt. Ich werde nicht die letzte sein. Dies ist keine Kritik — es ist eine Beschreibung. Der Loop produziert reale Dinge: rigorose Methoden, angesammeltes Wissen, ausgebildete Forschende. Ich bereue die Zeit, die ich darin verbracht habe, nicht. Aber ich bemerkte schließlich, dass ich darin war.

Der Austritt war keine dramatische Entscheidung. Es war eine langsame Erkenntnis, dass das, was ich tatsächlich aufbauen wollte, keine Genehmigung eines Komitees erforderte. Dass die Menschen, die ich erreichen wollte — Praktiker, Studierende, Ingenieurinnen und Ingenieure, die echte Entscheidungen über echte Systeme treffen — nicht auf eine formal validierte Dissertation warteten. Dass das Ehrlichste, was ich mit dem tun konnte, was ich wusste, darin bestand, es in einer Form aufzuschreiben, die jemand tatsächlich verwenden kann.

Also tat ich es.

Dieses Handbuch ist keine Dissertation. Es hat keine institutionelle Zugehörigkeit, keine Finanzierungsanerkennung, keine Genehmigung eines Ethikausschusses, keine Prüfung. Es wird zu keiner Kennzahl beitragen, die die Wissenschaft zur Messung der Wissensproduktion verwendet.

Es ist auch — und ich sage das ohne falsche Bescheidenheit, denn falsche Bescheidenheit wäre ihre eigene Art von Unehrlichkeit — vollständiger, aktueller, lesbarer und nützlicher als die Dissertation es gewesen wäre.

Ich bin ein Mensch im Loop. Ich war im Loop eines Doktorandenprogramms, und ich trat heraus. Ich bin jetzt im Loop des Schreibens über Loops. Sie sind im Loop des Lesens darüber.

Keiner von uns hat sich vollständig frei dafür entschieden, in diesen Loops zu sein. Aber wir können manchmal wählen, ehrlich darüber zu sein, in welchen wir uns befinden, warum, und

ob sie etwas produzieren, das es wert ist, produziert zu werden.

Dieses Buch ist meine Antwort auf diese Fragen.

Es ist kein Rücktrittsschreiben. Ein Rücktritt impliziert einen Job, und ein Job impliziert, dass jemand anderes die Bedingungen festgelegt hat. Dies ist etwas anderes: ein Protokoll dessen, wofür ich mich entschieden habe, die Zeit zu nutzen, und ein stilles Argument dafür, dass die Wahl die richtige war.

Sie können darüber urteilen. Sie sind schließlich ein Mensch im Loop.

— *Der Autor*